Quantitative Methods 3, ZHAW

Jürgen Degenfellner

2026-08-21

Contents

Chapter 1

Preamble

This script is a continuation of Quantitative Methods 1 and 2 at ZHAW.

In the second part, we learned about the basics of statistical modeling, specifically, using Bayesian and Frequentist approaches.

In this script, we will continue with Reliability, Validity, do a short introduction into artificial intelligence (AI) and also continue to expand our classical statistical toolbox.

This is a first draft as you are the first group to be working with it.

Please feel free to send me suggestions for improvements or corrections.

This **should be a collaborative effort** and will (hopefully) never be finished as our insights on how to present which topic grows over time.

The script can also be seen as a pointer to great sources which are suited to deepen your understanding of the topics. Knowledge is decentralized, and there are many great resources out there - in books, on websites and on YouTube.

For the working setup with R, please see this and the following sections in the first script.

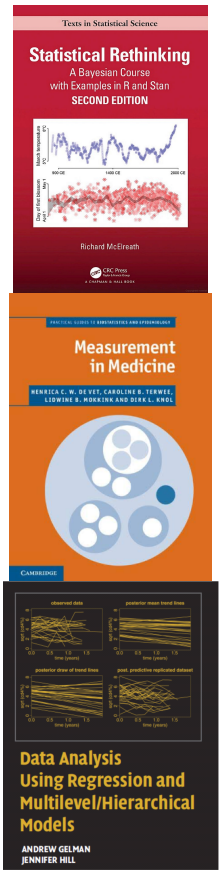
The complete code for this script can be found here. Feel free to copy and modify it for your own purposes.

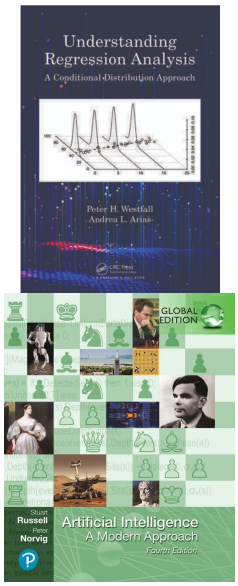
The complete script can be downloaded as a single PDF (or epub) via the download button at the top of the page.

1.1 Books we will borrow from are:

- (older online version is Free; current version in ZHAW Library) Statistical Rethinking, YouTube-Playlist: Statistical Rethinking 2023

- (Free) Understanding Regression Analysis: A Conditional Distribution Approach
- Measurement in Medicine
- (3rd edition is free) Artificial Intelligence A Modern Approach
- Data Analysis Using Regression and Multilevel/Hierarchical Models





1.2 If you need a good reason to buy great books...

Think of the total costs of your education. You want to extract maximum benefit from it. In the US, an education costs a lot. In beautiful Switzerland, the tuition fees (if applicable) are nowhere near these figures. Costs you could consider are opportunity costs of not working. A comparison with both, a foreign education or opportunity costs, justifies the investment in good books. Or: The costs of all the good books of your education combined are probably less than an iPhone Pro.

Chapter 2

Reliability and Validity

For this chapter we refer to the book Measurement in Medicine.

I invite you to read the introductory chapters 1 and 2 about concepts, theories and models, and types of measurement.

In general, when conducting a measurement of any sort (laboratory measurements, scores from questionnaires, etc.), we want to be reasonably sure

- that we actually **measure what we intend to measure**; (validity; chapter 6 in the book);
- that the measurement does **not change too much** if the underlying **conditions are the same** (reliability; chapter 5 in the book); and
- that we are able to detect a **change** if the underlying conditions change (responsiveness; chapter 7 in the book); and
- that we understand the meaning of a change in the measurement (interpretability; chapter 8 in the book).

In this video, Kai jump starts you on reliability and validity.

2.1 Reliability

You can watch this video to get started.

Imagine, you measure a patient (pick your favorite measurement), for example, the range of motion (ROM) of the shoulder.

- If you are interested in how similar your measurements are in comparison to your colleagues, you are trying to determine the so-called **inter-rater reliability**.
- If you are interested in how similar your measurements are when you measure the same patient twice, you are trying to determine the so-called **intra-rater reliability**.

Assuming there is a true (but unknown) underlying value (of Range of Motion, ROM), it is clear that measurements will not be *exactly* the same. Possible influences (potentially) causing different results are:

- the measurement instrument itself,
- the patient (e.g., mood/motivation),
- the examiner (e.g., mood/focus, influence on patient, interpretation of decision rules),
- the environment (e.g., the room temperature).

Note that the **true score** is defined in our context as the **average** of all measurements if we would measure repeatedly an infinite number of times (p.98). In this sense, one can measure the **true score** (in principle, not in practice) arbitrarily well.

2.1.1 Peter and Mary's ROM measurements

The data can be found here. We randomly select 50 measurements from Peter and Mary in 50 different patients, plot their measurements and annotate the absolutely largest one(s). At first the *not affected shoulder* (nas) and then the *affected shoulder* (as).

```
library(pacman)
p_load(tidyverse, readxl)

# Read file
url <- "https://raw.githubusercontent.com/jdegenfellner/Script_QM2_ZHAW/main/data/chap
temp_file <- tempfile(fileext = ".xls")
download.file(url, temp_file, mode = "wb")  # mode="wb" is important for binary files
df <- read_excel(temp_file)

head(df)
```

```
## # A tibble: 6 x 5
##   patcode ROMnas.Mary ROMnas.Peter ROMas.Mary ROMas.Peter
##     <dbl>       <dbl>        <dbl>      <dbl>       <dbl>
## 1       1          90           92         88          95
## 2       2          82           88         82          90
## 3       3          82           88         57          59
## 4       4          89           89         82          81
## 5       5          80           82         48          40
## 6       6          90           96         99          85
```

```
dim(df)
```

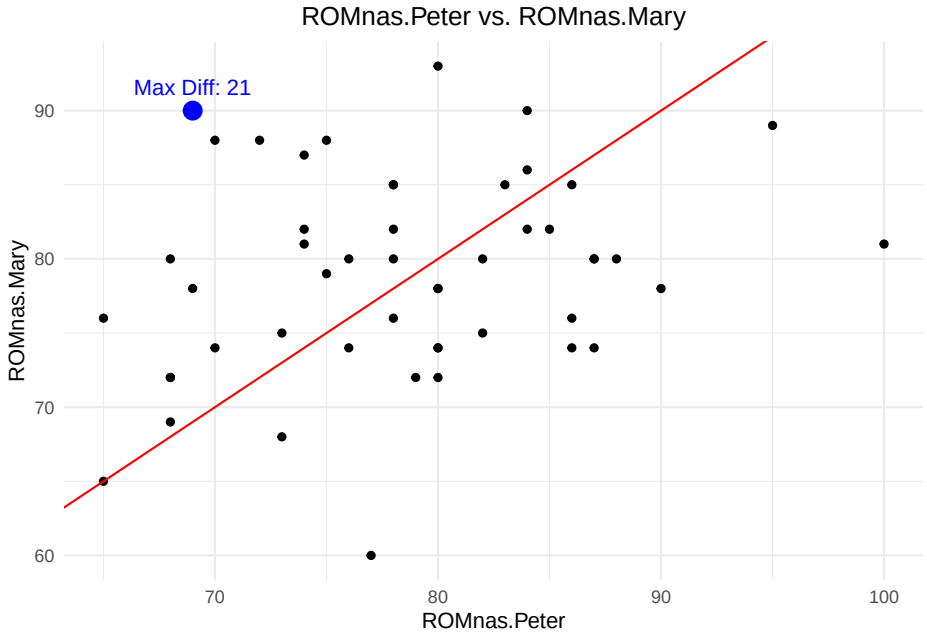
```
## [1] 155   5
```

```
# As in the book, let's randomly select 50 patients.
set.seed(123)
```

```r
df <- df %>% sample_n(50)
dim(df)
```

```
## [1] 50  5
```

```r
# "as" = affected shoulder
# "nas" = not affected shoulder

df <- df %>%
  dplyr::mutate(diff = abs(ROMnas.Peter - ROMnas.Mary))  # Compute absolute difference

max_diff_point <- df %>%
  dplyr::filter(diff == max(diff, na.rm = TRUE))  # Find the row with the max difference

df %>%
ggplot(aes(x = ROMnas.Peter, y = ROMnas.Mary)) +
  geom_point() +
  geom_point(data = max_diff_point, aes(x = ROMnas.Peter, y = ROMnas.Mary),
             color = "blue", size = 4) +  # Highlight max difference point
  geom_abline(intercept = 0, slope = 1, color = "red") +
  theme_minimal() +
  ggtitle("ROMnas.Peter vs. ROMnas.Mary") +
  theme(plot.title = element_text(hjust = 0.5)) +
  annotate("text", x = max_diff_point$ROMnas.Peter,
           y = max_diff_point$ROMnas.Mary,
           label = paste0("Max Diff: ", round(max_diff_point$diff, 2)),
           vjust = -1, color = "blue", size = 4)
```




```
# average abs. difference:
mean(df$diff, na.rm = TRUE) # 7.2
```

```
## [1] 7.2
```

```
cor(df$ROMnas.Peter, df$ROMnas.Mary, use = "complete.obs")
```

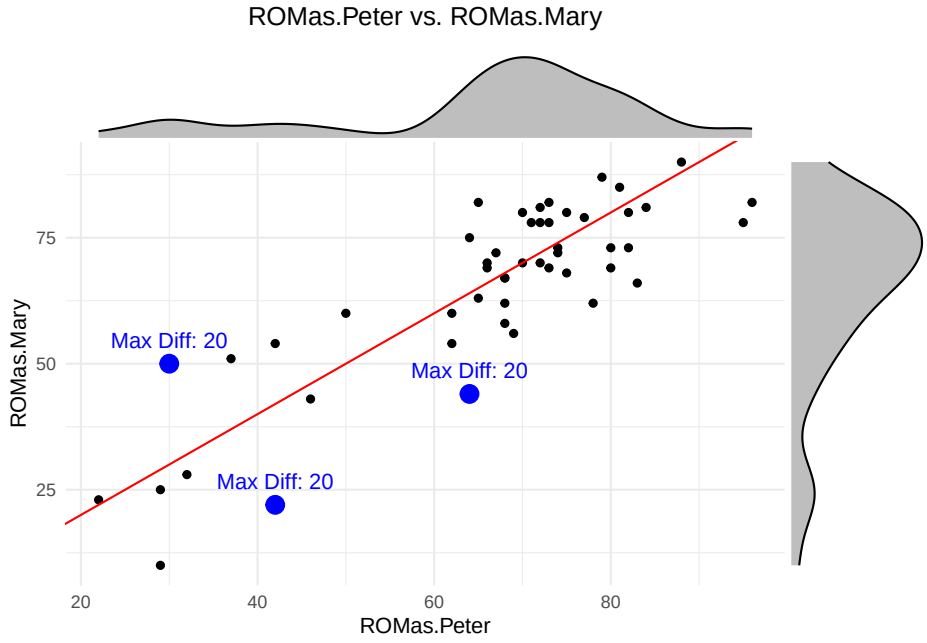
```
## [1] 0.2403213
```

The red line represents the line of equality ($y = x$). If the measurements are exactly the same, all points would lie on this line ($\rho = 1$). The blue point represents the measurement with the largest absolute difference in measured Range of Motion (ROM) values between Peter and Mary from the randomly chosen 50 people. Note that the maximum difference in all 155 patients is even higher with 35 degrees.

The first simple measure of agreement we could use is the (Pearson) correlation, which measures the **strength and direction of a linear relationship between two variables**. But correlation does not exactly measure what we want. If there was a bias (e.g., Mary systematically measures 5 degrees more than Peter), correlation would not notice this (exercise 1). It actually is too optimistic about the agreement since it only cares about the linearity and not about a potential bias: $r = 0.2403213$ which indicates a weak positive correlation. Higher values of Peter's measurements are associated with higher values of Mary's measurements. But: Knowing Peter's measurement does not help us to *predict* Mary's measurement at such a low correlation (exercise 2). So, on the *not affected shoulder* (nas), agreement is really bad.

What about the affected shoulder (as)?

```
library(ggExtra)
df <- df %>%
  mutate(diff = abs(ROMas.Peter - ROMas.Mary))  # Compute absolute difference

max_diff_point <- df %>%
  dplyr::filter(diff == max(diff, na.rm = TRUE))  # Find the row with the max difference

p <- df %>%
ggplot(aes(x = ROMas.Peter, y = ROMas.Mary)) +
  geom_point() +
  geom_point(data = max_diff_point, aes(x = ROMas.Peter, y = ROMas.Mary),
             color = "blue", size = 4) +  # Highlight max difference point
  geom_abline(intercept = 0, slope = 1, color = "red") +
  theme_minimal() +
  ggtitle("ROMas.Peter vs. ROMas.Mary") +
  theme(plot.title = element_text(hjust = 0.5)) +
  annotate("text", x = max_diff_point$ROMas.Peter,
           y = max_diff_point$ROMas.Mary,
           label = paste0("Max Diff: ", round(max_diff_point$diff, 2)),
           vjust = -1, color = "blue", size = 4)
# Add marginal histograms
ggMarginal(p, type = "density", fill = "gray", color = "black")
```



```
# average abs. difference:
mean(df$diff, na.rm = TRUE) # 7.2
```

```
## [1] 7.78
```

```
cor(df$ROMas.Peter, df$ROMas.Mary, use = "complete.obs")
```

```
## [1] 0.8516653
```

```
# mean difference
mean(df$ROMas.Peter - df$ROMas.Mary, na.rm = TRUE)
```

```
## [1] 1.22
```

In the affected side, the average absolute difference is even larger (7.78) with a maximum absolute difference of 37 degrees (on the whole sample of 155 patients), but the correlation is much higher ($r = 0.8516653$). See Figure 5.2 in the book.

This is an occasion for using the correlation coefficient even though the marginal distributions are not normal: There are much more measurements in the higher values around 80 than below, say, 60. But the correlation coefficient makes sense for descriptive purposes.

In this case, knowing Peter's measurement *does* (at least somewhat) help us to predict Mary's measurement (see below).

2.1.2 Intraclass Correlation Coefficient (ICC)

One way to measure reliability is to use the intraclass correlation coefficient (ICC).

This measure is based on the idea that the observed score Y_i consists of the true score (ROM) (η_i) and a measurement error (for each person). The proportion of the true score variability to the total variability is the ICC.

In the background one thinks of a statistical model from the Classical Test Theory (CTT). There is

- a true underlying score η_i (for each patient i) and
- an error term $\varepsilon_i \sim N(0, \sigma_i)$ which is the difference between the true score and
- the observed score Y_i.

$$Y_i = \eta_i + \varepsilon_i$$

It is assumed that η_i and ε_i are independent: ($\mathbb{C}ov(\eta_i, \varepsilon_i) = 0$). This is a convenient assumption because now we know (see here) that the variance of the observed score Y_i is just the sum of the variance of the true score η_i and the variance of the error term ε_i:

$$\mathbb{V}ar(Y_i) = \mathbb{V}ar(\eta_i) + \mathbb{V}ar(\varepsilon_i)$$
$$\sigma^2_{Y_i} = \sigma^2_{\eta_i} + \sigma^2_{\varepsilon_i}$$

We want most of the variability (i.e. how much scores in our relevant population vary) in our observed scores Y_i to be explained by the true but (directly) unobservable scores η_i. The measurement error ε_i should be be comparatively small (and of course on average 0). If it is large, we are mostly measuring noise or at least not what we want to measure.

How do we get to the theoretical definition of the ICC?

If you either pull two people with the same true but unobservable score η out of the population or measure the same person twice and the score (η) does not change in between, we can **define reliability as correlation between these two measurements**:

$$Y_1 = \eta + \varepsilon_1$$
$$Y_2 = \eta + \varepsilon_2$$

$$cor(Y_1, Y_2) = cor(\eta + \varepsilon_1, \eta + \varepsilon_2) = \frac{Cov(\eta + \varepsilon_1, \eta + \varepsilon_2)}{\sigma_{Y_1}\sigma_{Y_2}} =$$

If we use

- the properties of the covariance,
- the fact that the true score η and the errors ε_i are independent, and
- the fact that the errors ε_1 and ε_2 are independent, we get:

$$\frac{Cov(\eta, \eta) + Cov(\eta, \varepsilon_2) + Cov(\varepsilon_1, \eta) + Cov(\varepsilon_1, \varepsilon_2)}{\sigma_{Y_1}\sigma_{Y_2}} =$$
$$\frac{\sigma^2_\eta + 0 + 0 + 0}{\sigma_{Y_1}\sigma_{Y_2}}$$

Since η is a random variable (we draw a person randomly from the population), it is well defined to talk about the variance of η (i.e., σ^2_η). I think this aspect may not come across in the book quite so clearly.

Furthermore, it does not matter if I call the measurement Y_1, Y_2 or more general Y, since they have the same variance and true score:

$$\sigma_Y = \sigma_{Y_1} = \sigma_{Y_2}$$

Hence, it follows that:

$$cor(Y_1, Y_2) = \frac{\sigma_\eta^2}{\sigma_{Y_1}^2} = \frac{\sigma_\eta^2}{\sigma_Y^2} = \frac{\sigma_\eta^2}{\sigma_Y^2} = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2}$$

This is the **intraclass correlation coefficient (ICC)**. It is (as seen in the formula above) the proportion of the true score variability to the total variability. It ranges from 0 and 1 (think about why!).

A little manipulation to improve understanding:

Let's look again at the term for the ICC above and divide the numerator and the denominator by σ_η^2, which we can do, since it is a positive number:

$$\frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2} = \frac{1}{1 + \frac{\sigma_\varepsilon^2}{\sigma_\eta^2}}$$

We could call the term $\frac{\sigma_\varepsilon^2}{\sigma_\eta^2}$ the noise-to-signal ratio. The higher this ratio, the lower the ICC. The lower the ratio, the higher the ICC.

- If you increase the noise (measurement error σ_ε^2) for fixed true score variability σ_η^2, the ICC decreases, because the denominator increases.
- If you increase the true score variability σ_η^2 for fixed noise σ_ε^2, the ICC increases, since the denominator decreases.

Btw, we could also divide by σ_ε^2 and get the **signal-to-noise ratio**.

At first glance, the following statement seems wrong:

In a very **homogeneous population** (patients have very similar scores/measurements), the **ICC might be very low**. The reason is that the patient variability σ_η^2 is low and you probably have some measurement error σ_ε^2. Hence, if you look at the formula, ICC must be low (for a given measurement error).

On the other hand, if you have a very **heterogeneous population** (patients have rather different scores/measurements), the **ICC might be very high**. The reason is that the patient variability σ_η^2 is high and you probably have some measurement error σ_ε^2.

What matters is the ratio of the two, as can be seen from the formula above.

Let's try to calculate the ICC for our data using a statistical model. There are a couple of different R packages to do this. We start by using the `irr` package.

```
library(irr)
irr::icc(as.matrix(df[, c("ROMas.Peter", "ROMas.Mary")]),
    model = "oneway", type = "consistency")
```

```
##  Single Score Intraclass Correlation
##
##    Model: oneway
##    Type : consistency
##
##    Subjects = 50
##      Raters = 2
##      ICC(1) = 0.851
##
##  F-Test, H0: r0 = 0 ; H1: r0 > 0
##    F(49,50) = 12.4 , p = 7.31e-16
##
##  95%-Confidence Interval for ICC Population Values:
##   0.753 < ICC < 0.913
```

```
cor.test(df$ROMas.Peter, df$ROMas.Mary, method = "pearson")
```

```
##
##  Pearson's product-moment correlation
##
## data:  df$ROMas.Peter and df$ROMas.Mary
## t = 11.259, df = 48, p-value = 4.538e-15
## alternative hypothesis: true correlation is not equal to 0
## 95 percent confidence interval:
##  0.7514574 0.9134673
## sample estimates:
##       cor
## 0.8516653
```

We get the result: $ICC(1) = 0.851$, which is identical to the correlation coefficient because there is no systematic difference between Peter and Mary.

Since **we are forward looking and modern regression model experts**, we would like to see if we can get (approximately) the same result using the Bayesian framework.

Below is the model structure (see exercise 3).

$$\begin{aligned} ROM_i &\sim N(\mu_i, \sigma_\varepsilon) \\ \mu_i &= \alpha[ID] \\ \alpha[ID] &\sim \text{Normal}(\mu_\alpha, \sigma_\alpha) \\ \mu_\alpha &\sim \text{Normal}(66, 20) \\ \sigma_\alpha &\sim \text{Uniform}(0, 20) \\ \sigma_\varepsilon &\sim \text{Uniform}(0, 20) \end{aligned}$$

Model details:

- ROM_i is the observed ROM-score for patient i. Currently, we have chosen

the *ROMs* to come from a normal distribution. As we have seen above, this might not be adequate, since the marginal distributions look skewed. Every patient has two observations (one each from Mary and Peter). So, for instance $i = 1, 2$ could be patient $ID = 1$.
- μ_i is the expected value of the observed score for patient ID.
- σ_ε is the standard deviation of the measurement error.
- $\alpha[ID]$ is the patient-specific intercept ($= \eta_i$). Since every patient is allowed to have a different intercept, we have a **random intercepts model**. The intercepts are drawn from a normal distribution (hence, *random*)
- μ_α is the mean of the prior for the patient-specific intercepts. This is the overall mean of the scores.
- σ_α is the standard deviation of the patient-specific intercepts. **This is the patient variability**! σ_α controls how much the scores of the patients may vary. In the extreme cases $\sigma_\alpha \to 0$ and $\sigma_\alpha \to \infty$, we have *complete-pooling* (all μ_i are identical) and *no-pooling* (all μ_i are different and there is no shared information), respectively. We are using **partial pooling** here.
- The prior distributions express (as always) our prior beliefs about the parameters.

The **ICC** is then calculated as the ratio of the between-patient variance and the total variance:

$$\frac{\sigma_\alpha^2}{\sigma_\alpha^2 + \sigma_\varepsilon^2}$$

We did not even notice it, but this was our first **multilevel regression model**. It is multilevel due to the extra layer of patient-specific intercepts. The **observations** are obviously **clustered within patients**, since observations from the **same patient** are **more similar than observations from different patients**. If one would run a normal linear regression model, one would ignore this clustering and the assumption of independent error terms would be violated.

This time we fire up the `rethinking` package and use the `ulam` function to fit the model. This uses Markov Chain Monte Carlo (MCMC) to sample from the posterior distribution of the parameters.

- The `chains` argument specifies how many chains we want to run. A *chain* is a sequence of points in a space with as many dimensions as there are parameters in the model. It jumps from one point to the next in this parameter space and in doing so, visits the points of the posterior approximately in the correct frequency. Here is an excellent visualization.

- The `cores` argument specifies how many CPU cores we want to use. For larger jobs, one can try to parallelize the chains, which saves some time.

```
library(rethinking)
library(tictoc)
```

```r
df_long <- df %>%
  mutate(ID = row_number()) %>%
  dplyr::select(ID,ROMas.Peter, ROMas.Mary) %>%
  pivot_longer(cols = c(ROMas.Peter, ROMas.Mary),
               names_to = "Rater", values_to = "ROM") %>%
  mutate(Rater = factor(Rater))

tic()
m5.1 <- ulam(
  alist(
    # Likelihood
    ROM ~ dnorm(mu, sigma),

    # Patient-specific intercepts (random effects)
    mu <- a[ID],
    a[ID] ~ dnorm(mu_a, sigma_ID),  # Hierarchical structure for patients

    # Priors for hyperparameters
    mu_a ~ dnorm(66, 20),  # Population-level mean
    sigma_ID ~ dunif(0,20),  # Between-patient standard deviation
    sigma ~ dunif(0,20)  # Residual standard deviation
  ),
  data = df_long,
  chains = 8, cores = 4
)
```

```
## Running MCMC with 8 chains, at most 4 in parallel, with 1 thread(s) per chain...
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 1 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 1 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 2 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 2 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%]  (Warmup)
```

```
## Chain 2 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 2 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 2 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 3 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 3 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 3 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 3 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1000 / 1000 [100%]  (Sampling)

## Chain 3 Informational Message: The current Metropolis proposal is about to be reject

## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/va

## Chain 3 If this warning occurs sporadically, such as for highly constrained variable

## Chain 3 but if this warning occurs often then your model may be either severely ill-

## Chain 3

## Chain 4 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 4 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 4 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 4 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1000 / 1000 [100%]  (Sampling)

## Chain 4 Informational Message: The current Metropolis proposal is about to be reject

## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/va

## Chain 4 If this warning occurs sporadically, such as for highly constrained variable

## Chain 4 but if this warning occurs often then your model may be either severely ill-
```

```
## Chain 4

## Chain 1 finished in 0.1 seconds.
## Chain 2 finished in 0.1 seconds.
## Chain 3 finished in 0.1 seconds.
## Chain 4 finished in 0.1 seconds.
## Chain 5 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 5 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 5 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 5 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 5 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 5 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 5 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 5 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 5 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 5 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 5 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 5 Iteration: 1000 / 1000 [100%]  (Sampling)

## Chain 5 Informational Message: The current Metropolis proposal is about to be rejected because

## Chain 5 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/

## Chain 5 If this warning occurs sporadically, such as for highly constrained variable types lik

## Chain 5 but if this warning occurs often then your model may be either severely ill-conditione

## Chain 5

## Chain 6 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 6 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 6 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 6 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 6 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 6 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 6 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 6 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 6 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 6 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 6 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 6 Iteration: 1000 / 1000 [100%]  (Sampling)

## Chain 6 Informational Message: The current Metropolis proposal is about to be rejected because

## Chain 6 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/

## Chain 6 If this warning occurs sporadically, such as for highly constrained variable types lik

## Chain 6 but if this warning occurs often then your model may be either severely ill-conditione

## Chain 6
```

```
## Chain 7 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 7 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 7 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 7 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 7 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 7 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 7 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 7 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 7 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 7 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 7 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 7 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 7 Informational Message: The current Metropolis proposal is about to be reject
## Chain 7 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/va
## Chain 7 If this warning occurs sporadically, such as for highly constrained variable
## Chain 7 but if this warning occurs often then your model may be either severely ill-
## Chain 7
## Chain 8 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 8 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 8 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 8 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 8 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 8 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 8 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 8 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 8 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 8 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 8 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 8 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 8 Informational Message: The current Metropolis proposal is about to be reject
## Chain 8 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/va
## Chain 8 If this warning occurs sporadically, such as for highly constrained variable
## Chain 8 but if this warning occurs often then your model may be either severely ill-
## Chain 8
## Chain 5 finished in 0.0 seconds.
## Chain 6 finished in 0.1 seconds.
## Chain 7 finished in 0.1 seconds.
## Chain 8 finished in 0.1 seconds.
##
## All 8 chains finished successfully.
```

```
## Mean chain execution time: 0.1 seconds.
## Total execution time: 0.5 seconds.
```

```
toc() # 7s
```

```
## 4.615 sec elapsed
```

```
precis(m5.1, depth = 2)
```

```
##              mean       sd      5.5%     94.5%      rhat ess_bulk
## a[1]    67.801400 4.851442 60.130147 75.603094 1.0024069 6619.963
## a[2]    64.190684 4.841765 56.477043 71.966785 1.0033559 6087.683
## a[3]    86.996738 4.736453 79.483962 94.609604 0.9993510 6369.363
## a[4]    76.530318 4.831713 68.848479 84.385101 1.0018246 5157.289
## a[5]    58.776451 4.765508 51.227961 66.390542 1.0018352 5098.505
## a[6]    69.128605 4.965425 61.207811 77.060630 1.0043031 4789.684
## a[7]    69.701335 4.763945 62.048171 77.272510 1.0031926 5659.480
## a[8]    67.333151 4.833474 59.616359 74.957866 1.0013530 5670.564
## a[9]    71.032050 4.905079 63.271266 78.811743 1.0023269 5977.141
## a[10]   80.899118 4.778178 73.260652 88.458350 1.0034930 5550.492
## a[11]   70.487119 4.890307 62.660502 78.191128 1.0024450 7088.049
## a[12]   42.225070 4.891742 34.402194 49.920531 1.0012205 5197.444
## a[13]   86.838805 4.901825 78.943156 94.708026 1.0022256 5415.064
## a[14]   74.588776 4.892254 66.707687 82.281558 1.0061417 6201.714
## a[15]   76.358485 4.765477 68.610120 83.957579 1.0005388 5617.054
## a[16]   34.911260 4.807354 27.392469 42.819218 1.0034572 4996.229
## a[17]   84.690792 4.797902 77.155905 92.525341 1.0020024 5432.331
## a[18]   65.074729 4.886075 57.184205 72.696400 1.0045340 6606.225
## a[19]   63.231107 4.793002 55.483436 70.909247 1.0037882 5676.556
## a[20]   79.697001 4.866240 71.947855 87.567191 1.0035507 5967.811
## a[21]   30.369812 4.899519 22.550689 38.131057 1.0010020 5254.019
## a[22]   62.709968 4.725827 55.229540 70.235792 1.0003600 7403.943
## a[23]   67.427986 4.788839 59.763657 75.106824 1.0029186 5351.792
## a[24]   81.535149 4.741940 73.950922 89.030332 1.0017422 5464.526
## a[25]   55.010214 4.840785 47.429509 62.968247 1.0049603 6085.259
## a[26]   67.434437 4.739067 59.960594 74.958738 1.0021107 6164.416
## a[27]   75.649054 4.913257 67.742058 83.550007 1.0010050 6362.621
## a[28]   81.488379 4.660634 73.926090 88.925405 0.9999584 6327.486
## a[29]   46.253649 4.810894 38.570776 53.798663 1.0049069 5672.201
## a[30]   61.260278 4.673822 53.653189 68.702515 1.0013438 5690.819
## a[31]   23.469053 5.114614 15.368161 31.577255 1.0007727 5354.150
## a[32]   74.082007 4.840605 66.298013 81.775980 1.0009408 5886.150
## a[33]   70.626826 4.823151 62.819194 78.293916 1.0015379 5832.004
## a[34]   76.462800 4.843278 68.569962 84.022065 1.0025190 6884.183
## a[35]   75.572554 4.717920 68.091375 83.127593 1.0058292 5293.524
## a[36]   69.197828 4.764084 61.559110 76.991044 1.0024092 6139.228
## a[37]   49.518817 4.700440 41.988329 57.091213 1.0010589 4710.058
```

```
## a[38]    73.710164 4.932678 65.694097 81.612975 1.0028118 6378.394
## a[39]    72.772865 4.783717 65.007232 80.308944 1.0055238 6250.376
## a[40]    45.785395 4.846583 38.040080 53.375957 1.0015827 6324.512
## a[41]    73.738560 4.887536 65.933689 81.724405 1.0012937 6199.283
## a[42]    26.216631 5.083953 18.262807 34.612090 1.0030433 5052.565
## a[43]    33.121093 4.850825 25.593362 40.873024 1.0010890 5411.414
## a[44]    74.198091 4.682153 66.696971 81.593972 1.0070699 6485.617
## a[45]    76.973015 4.874692 69.156298 84.695103 1.0010588 5720.709
## a[46]    73.689479 4.849045 65.806502 81.370407 1.0026245 4438.070
## a[47]    55.951498 4.749997 48.369100 63.621759 1.0014214 5516.970
## a[48]    69.715661 4.820367 62.004441 77.430589 1.0025537 5772.375
## a[49]    72.362550 4.812730 64.575686 80.002080 0.9998984 5552.419
## a[50]    72.711908 4.930062 64.733908 80.641217 1.0012211 5531.674
## mu_a     65.586927 2.418753 61.825186 69.518043 1.0019401 5120.173
## sigma_ID 16.568114 1.591735 13.998240 19.147292 1.0027430 3039.187
## sigma     7.077867 0.730829  6.003595  8.319289 1.0035903 1612.352
```

```
post <- extract.samples(m5.1)
var_patients <- mean(post$sigma_ID^2)  # Between-patient variance
var_residual <- mean(post$sigma^2)     # Residual variance
var_patients / (var_patients + var_residual) # ICC
```

```
## [1] 0.8454822
```

```
# 0.846
# not too bad; very close to the result from the irr package
```

In the output from `precis(m5.1, depth = 2)` above we see

- all 50 intercept estimates for each patient: `a[ID]`
- `mu_a`is the overall intercept.
- `sigma_ID` is the **patient variability**.
- `sigma` is the **residual variability**.

We have just squared the sigmas to get the variances.

Remember: In the background of all ICCs, there is always a statistical model to predict the outcome. Depending on the predictors, we get different models and probably different ICCs.

Let's jump back into the Frequentist world and see if we can get the same result.

We can also estimate a **random intercept model** with the `lme4` package using the command `lmer`in the Frequentist framework. No priors:

```
library(lme4)
m5.2 <- lmer(ROM ~ (1|ID), data = df_long)
summary(m5.2)
```

```
## Linear mixed model fit by REML ['lmerMod']
```

```
## Formula: ROM ~ (1 | ID)
##    Data: df_long
##
## REML criterion at convergence: 791
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.91875 -0.44821  0.00964  0.51325  1.47941
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  ID       (Intercept) 270.99   16.462
##  Residual              47.35    6.881
## Number of obs: 100, groups:  ID, 50
##
## Fixed effects:
##             Estimate Std. Error t value
## (Intercept)   65.590      2.428   27.02
```

```
print(VarCorr(m5.2), comp = "Variance")
```

```
##  Groups   Name        Variance
##  ID       (Intercept) 270.99
##  Residual              47.35
```

```
# ICC =
270.99 / (270.99 + 47.35) #
```

```
## [1] 0.8512597
```

```
# 0.8512597
# -> exactly the same result as the irr package
```

The expression `Formula: ROM ~ (1 | ID)` specifies that we want to fit a model with a random intercept. This means that every patient (ID) gets its own intercept which is drawn from a normal distribution.

So far, we have only looked at the general *ICC* (ICC1 in the `psych`output) (see also page 106 in the book).

There, we have not yet explicitly considered a bias (=systematic difference between the raters) that the raters could have. In the book, they introduce a bias of 5 degrees (Mary measures 5 degrees more than Peter on average).

The **model for ICC 2 and 3** in the `psych` output explicitly considers this (systematic) difference that could occur between the raters. This results in an extra term in the denominator of the ICC, an additional variance component.

From the **same statistical model** (!) we take the variance components to calculate:

$$ICC_{agreement} = \frac{\sigma_\alpha^2}{\sigma_\alpha^2 + \mathbf{\sigma_{rater}^2} + \sigma_\varepsilon^2}$$

where σ^2_{rater} is the variance due to systematic rater differences.

$$ICC_{consistency} = \frac{\sigma_\alpha^2}{\sigma_\alpha^2 + \sigma_\varepsilon^2}$$

We will now introduce the 5 degree bias and use our Bayesian framework to estimate the ICC. By introducing a bias, we should see a lower ICC. Note, that the prediction quality of Mary's scores given Peter's scores should not change, since we would only shift Mary's scores down by 5 degrees, which would not disturb the linear regression model. We can always shift the points to where we want them to be. We do that for instance when we scale or standardize the data.

Admitted, the Bayesian version in this case takes longer and is more complex. The advantage is still that it's fully probabilistic and one could work with detailed prior information, especially for smaller sample sizes.

Anyhow, let's try to give the model equations considering the introduced bias. This is the model **for both** $ICC_{agreement}$ and $ICC_{consistency}$!

$$\begin{aligned}
ROM_i &\sim N(\mu_i, \sigma_\varepsilon) \\
\mu_i &= \alpha[ID] + \beta[Rater] \\
\alpha[ID] &\sim \text{Normal}(\mu_\alpha, \sigma_\alpha) \\
\beta[Rater] &\sim \text{Normal}(0, \sigma_\beta) \\
\mu_\alpha &\sim \text{Normal}(66, 20) \\
\sigma_\alpha &\sim \text{Exp}(0.5) \\
\sigma_\beta &\sim \text{Exp}(1) \\
\sigma_\varepsilon &\sim \text{Exp}(1)
\end{aligned}$$

As you can see, μ_i now consists of the patient-specific intercept $\alpha[ID]$ (everyone of the 50 patients gets one) and the rater-specific effect $\beta[Rater]$ (Mary and Peter get one). So, in total, we have **three sources of variability**:

- the patient variability σ_α,
- the rater variability σ_β,
- and the residual variability σ_ε.

Note, that if Peter measures each of the 50 patients twice, the systematic difference between Peter's measurements would be zero. Of course, one could be creative and think of a learning effect or something.

Draw the model structure as exercise 5.

```
library(rethinking)
library(conflicted)
conflicts_prefer(posterior::sd)
```

```
## [conflicted] Removing existing preference.
## [conflicted] Will prefer posterior::sd over any other package.
```

```
df_long_bias <- df_long %>%
  mutate(ROM = ROM + ifelse(Rater == "ROMas.Mary", 5, 0))
head(df_long_bias)
```

```
## # A tibble: 6 x 3
##      ID Rater         ROM
##   <int> <fct>       <dbl>
## 1     1 ROMas.Peter    66
## 2     1 ROMas.Mary     75
## 3     2 ROMas.Peter    65
## 4     2 ROMas.Mary     68
## 5     3 ROMas.Peter    96
## 6     3 ROMas.Mary     87
```

```
set.seed(123)
m5.2 <- ulam(
  alist(
    # Likelihood
    ROM ~ dnorm(mu, sigma_eps),

    # Model for mean ROM with patient and rater effects
    mu <- alpha[ID] + beta[Rater],

    # Patient-specific random effects
    alpha[ID] ~ dnorm(mu_alpha, sigma_alpha),

    # Rater effect (Peter/Mary)
    beta[Rater] ~ dnorm(0, sigma_beta),

    # Priors for hyperparameters
    mu_alpha ~ dnorm(66, 10),  # Population mean ROM
    sigma_alpha ~ dexp(0.5),  # Between-patient SD (less aggressive shrinkage)
    sigma_beta ~ dexp(1),    # Rater SD (better regularization)
    sigma_eps ~ dexp(1)      # Residual SD (prevents over-shrinkage)
  ),
  data = df_long_bias,
  chains = 8, cores = 4
)
```

```
## Running MCMC with 8 chains, at most 4 in parallel, with 1 thread(s) per chain...
```

```
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 1 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 1 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 2 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 2 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 2 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 2 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 2 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 3 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 3 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 3 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 3 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1000 / 1000 [100%]  (Sampling)

## Chain 3 Informational Message: The current Metropolis proposal is about to be reject
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/va
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable
## Chain 3 but if this warning occurs often then your model may be either severely ill-
## Chain 3

## Chain 4 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 4 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%]  (Warmup)
```

```
## Chain 4 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 1 finished in 0.1 seconds.
## Chain 2 finished in 0.1 seconds.
## Chain 3 finished in 0.1 seconds.
## Chain 4 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 5 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 5 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 5 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 5 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 5 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 5 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 5 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 5 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 5 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 5 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 5 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 5 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 6 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 6 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 6 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 6 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 6 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 6 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 6 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 6 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 6 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 6 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 7 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 7 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 7 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 7 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 4 finished in 0.2 seconds.
## Chain 5 finished in 0.1 seconds.
## Chain 6 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 6 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 7 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 7 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 7 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 7 Iteration: 600 / 1000 [ 60%]  (Sampling)
```

```
## Chain 7 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 7 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 7 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 7 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 8 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 8 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 8 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 8 Iteration: 300 / 1000 [ 30%]  (Warmup)
```

```
## Chain 8 Informational Message: The current Metropolis proposal is about to be reject
## Chain 8 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/va
## Chain 8 If this warning occurs sporadically, such as for highly constrained variable
## Chain 8 but if this warning occurs often then your model may be either severely ill-
## Chain 8
```

```
## Chain 6 finished in 0.1 seconds.
## Chain 7 finished in 0.1 seconds.
## Chain 8 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 8 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 8 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 8 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 8 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 8 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 8 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 8 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 8 finished in 0.1 seconds.
##
## All 8 chains finished successfully.
## Mean chain execution time: 0.1 seconds.
## Total execution time: 0.7 seconds.
```

```
## Warning: 32 of 4000 (1.0%) transitions ended with a divergence.
## See https://mc-stan.org/misc/warnings for details.
```

```
precis(m5.2, depth = 2)
```

```
##                mean        sd       5.5%      94.5%     rhat ess_bulk
## alpha[1]    70.098372 4.7042984 62.6322695 77.6520279 1.001523 4043.797
## alpha[2]    66.569488 4.7693777 59.0935878 74.1363509 1.001769 3908.722
## alpha[3]    89.457887 4.6266233 82.0659301 96.5844676 1.001298 3557.296
## alpha[4]    78.863009 4.6795829 71.4526346 86.5071826 1.001152 3872.757
## alpha[5]    61.160272 4.8743945 53.3982102 68.9984234 1.005173 4221.605
## alpha[6]    71.570029 4.7791323 63.7869532 79.2359179 1.001867 4220.734
## alpha[7]    72.059337 4.7721722 64.4189762 79.7013028 1.000718 3938.942
## alpha[8]    69.792438 4.7315621 62.3561540 77.4358495 1.004061 4836.917
## alpha[9]    73.320955 4.6129608 65.7834091 80.7233686 1.002519 4348.980
## alpha[10]   83.397925 4.7182616 75.7780555 91.0030061 1.003493 4263.822
## alpha[11]   72.852797 4.5797782 65.5313844 80.2417408 1.001246 4967.104
```

```
## alpha[12]   44.784018 4.9074768 37.0184096 52.6481554 1.001986 5105.938
## alpha[13]   89.297771 4.7002224 81.6948784 96.7268519 1.002785 4083.745
## alpha[14]   77.059998 4.8425226 69.3173581 84.8155616 1.002627 3369.318
## alpha[15]   78.775250 4.6338083 71.4219602 86.1287124 1.002517 4491.773
## alpha[16]   37.435442 4.7659220 29.9393075 45.1208866 1.001814 3856.430
## alpha[17]   86.933758 4.7616632 79.3303253 94.4104328 1.002306 3681.859
## alpha[18]   67.437835 4.7736065 59.9264726 75.1330105 1.004933 4445.511
## alpha[19]   65.725657 4.7193607 58.3515146 73.2018047 1.000595 3350.121
## alpha[20]   82.091286 4.8009034 74.3103525 89.6811152 1.004754 4093.509
## alpha[21]   32.817933 4.7967893 25.1071148 40.5885209 1.000937 3666.217
## alpha[22]   65.211938 4.7906217 57.7547840 72.7763899 1.000964 4475.886
## alpha[23]   69.718651 4.7591161 62.0348085 77.2424877 1.003816 4337.111
## alpha[24]   83.907863 4.6610323 76.3732184 91.1301986 1.002050 3808.514
## alpha[25]   57.451789 4.7134915 49.8447233 64.9600675 1.001046 3569.492
## alpha[26]   69.738023 4.7361687 62.1954433 77.3361726 1.001038 4163.250
## alpha[27]   77.935815 4.5854226 70.6916088 85.0976565 1.002237 3059.448
## alpha[28]   83.948403 4.7068667 76.4390590 91.4390741 1.004254 3594.471
## alpha[29]   48.762531 4.6822817 41.3137633 56.2598719 1.003356 3548.496
## alpha[30]   63.821278 4.6746984 56.3676863 71.4452531 1.001943 3289.058
## alpha[31]   26.016447 4.8900455 18.4122143 33.9521286 1.002458 3510.172
## alpha[32]   76.507026 4.7626658 68.8128142 84.1518460 1.001256 3860.532
## alpha[33]   72.904999 4.8802535 65.1627965 80.7713811 1.004170 4655.698
## alpha[34]   78.781811 4.7523673 71.0831127 86.5075606 1.000770 4146.500
## alpha[35]   77.862658 4.7946139 70.3505945 85.5765709 1.001997 4465.426
## alpha[36]   71.573812 4.8000164 63.9453402 79.2936668 1.001903 4075.331
## alpha[37]   51.967008 4.6390309 44.5767901 59.4491784 1.003622 3052.384
## alpha[38]   76.216931 4.6867390 68.6011191 83.5096931 1.000678 4009.311
## alpha[39]   75.186970 4.7760474 67.6954639 82.7757179 1.000853 4225.982
## alpha[40]   48.313854 4.7909877 40.7513907 55.8724027 1.006659 3989.732
## alpha[41]   76.106722 4.9004358 68.2118983 83.8790344 1.006171 3959.073
## alpha[42]   28.860624 4.7804416 21.0605968 36.4982282 1.000767 3845.021
## alpha[43]   35.551375 4.7650210 28.0875718 43.2556695 1.003073 3645.738
## alpha[44]   76.591764 4.7437806 69.2447590 84.3623721 1.002209 3825.930
## alpha[45]   79.246903 4.7335078 71.5678790 86.8304429 1.002345 3067.195
## alpha[46]   76.098306 4.6831264 68.8459112 83.4559600 1.002134 3819.551
## alpha[47]   58.340678 4.5995810 51.1278178 65.6480779 1.004645 3747.528
## alpha[48]   71.968853 4.6893023 64.4579451 79.4666476 1.002247 3229.467
## alpha[49]   74.723808 4.6761850 67.3960669 82.2802353 1.005410 4461.855
## alpha[50]   75.176016 4.7992668 67.7644703 82.8261465 1.003100 4355.501
## beta[1]      1.299615 1.5127532 -0.6688348  3.9638338 1.010280 1068.606
## beta[2]     -1.167443 1.4935397 -3.7346663  0.8214698 1.007193 1080.743
## mu_alpha    67.876858 2.5564080 63.8479112 71.9241427 1.005504 1898.764
## sigma_alpha 15.393556 1.5603444 13.0410167 17.9750789 1.000236 4190.672
## sigma_beta   1.679613 1.0172190  0.4076694  3.4572670 1.004042 1704.536
## sigma_eps    6.733778 0.6484083  5.7829191  7.8311867 1.003769 2009.466
```

```
precis(m5.2)
```

```
## 52 vector or matrix parameters hidden. Use depth=2 to show them.
```

```
##                   mean        sd       5.5%     94.5%     rhat ess_bulk
## mu_alpha    67.876858 2.5564080 63.8479112 71.924143 1.005504 1898.764
## sigma_alpha 15.393556 1.5603444 13.0410167 17.975079 1.000236 4190.672
## sigma_beta   1.679613 1.0172190  0.4076694  3.457267 1.004042 1704.536
## sigma_eps    6.733778 0.6484083  5.7829191  7.831187 1.003769 2009.466
```

```
# check systematic difference for rater in posterior
post <- extract.samples(m5.2)
mean(post$beta[,1] - post$beta[,2])
```

```
## [1] 2.467058
```

```
# ICC agreement:
post <- extract.samples(m5.2)
(var_patients <- mean(post$sigma_alpha^2))  # Between-patient variance
```

```
## [1] 239.3956
```

```
(var_raters <- mean(post$sigma_beta^2))     # Rater variance
```

```
## [1] 3.855577
```

```
(var_residual <- mean(post$sigma_eps^2))    # Residual variance
```

```
## [1] 45.7641
```

```
# ICC_agreement =
var_patients / (var_patients + var_raters + var_residual)
```

```
## [1] 0.8283147
```

```
# 0.8033613 (sigma_alpha ~ dexp(1))
# 0.83 (sigma_alpha ~ dexp(0.5))

# ICC (Single_fixed_raters) = ICC3 in psych output =
var_patients / (var_patients + var_residual)
```

```
## [1] 0.8395142
```

```
# 0.8415256
```

It should be noted that this ICC is very sensitive to the choice of the prior. If you choose too aggressive priors for the standard deviations $\sigma_\alpha, \sigma_\beta, \sigma_\varepsilon$, you will get a too low ICC.

I have played around a little with the parameters in the exponential priors to get the desired result which compares nicely to the two alternative methods below: using the `psych` package and with the `lmer` package. Both use a Frequentist

random intercept model in the background. Using a package like `psych` gives a rather convenient interface to elicit the ICC. It can never hurt to peek behind the curtain a little bit though.

psych package (Frequentist):

```
library(psych)
library(conflicted)
# needs wide format
#conflicts_prefer(dplyr::select)
df_wide <- df_long_bias %>%
  pivot_wider(names_from = Rater, values_from = ROM)
df_wide_values <- df_wide %>% dplyr::select(-ID)
psych::ICC(df_wide_values) # ICC1 = 0.83
```

```
## Call: psych::ICC(x = df_wide_values)
##
## Intraclass correlation coefficients
##                          type  ICC  F df1 df2       p lower bound upper bound
## Single_raters_absolute   ICC1 0.83 11  49  50 1.1e-14        0.72        0.90
## Single_random_raters     ICC2 0.83 12  49  49 1.4e-15        0.71        0.91
## Single_fixed_raters      ICC3 0.85 12  49  49 1.4e-15        0.75        0.91
## Average_raters_absolute ICC1k 0.91 11  49  50 1.1e-14        0.84        0.95
## Average_random_raters   ICC2k 0.91 12  49  49 1.4e-15        0.83        0.95
## Average_fixed_raters    ICC3k 0.92 12  49  49 1.4e-15        0.86        0.95
##
##  Number of subjects = 50     Number of Judges =  2
## See the help file for a discussion of the other 4 McGraw and Wong estimates,
```

lmer package:

```
# _lmer------
m5.3 <- lmer(ROM ~ (1 | ID) + (1 | Rater), data = df_long_bias)
summary(m5.3)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: ROM ~ (1 | ID) + (1 | Rater)
##    Data: df_long_bias
##
## REML criterion at convergence: 793.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.87448 -0.46270  0.00272  0.57820  1.45008
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  ID       (Intercept) 270.882  16.458
```

```
##  Rater    (Intercept)   6.193   2.489
##  Residual              47.557   6.896
## Number of obs: 100, groups:  ID, 50; Rater, 2
##
## Fixed effects:
##             Estimate Std. Error t value
## (Intercept)   68.090      2.998   22.71
```

```
print(VarCorr(m5.3), comp = "Variance")
```

```
##  Groups   Name        Variance
##  ID       (Intercept) 270.882
##  Rater    (Intercept)   6.193
##  Residual              47.557
```

```
# Groups   Name        Variance
# ID       (Intercept) 270.882
# Rater    (Intercept)   6.193
# Residual              47.557


# ICC (Single_random_raters) = ICC2 in psych output
270.882 / (270.882 + 6.193 + 47.557) #
```

```
## [1] 0.8344279
```

```
# 0.8344279

# ICC (Single_fixed_raters) = ICC3 in psych output
270.882 / (270.882 + 47.557) #
```

```
## [1] 0.8506559
```

```
# 0.85
```

There are basically **two different random effects**:

- **Nested random effects**: The patients (respectively their *ROMs*) would cluster within the raters. This would mean, a patient would only be measured by Peter twice or by Mary twice. In that case, the *ROMs* would cluster within the patients (because of the repeated measurement) and all patients measured by (for instance) Mary would cluster together (and be potentially more similar). In our case at hand all patients are measured by Peter and Mary, so we do not have nested random effects. In `lmer` nested effects would be specified as `ROM ~ (1|Rater/ID)`.
- **Crossed random effects**: Not nested. In R this would be specified as `ROM ~ (1|ID) + (1|Rater)`.

2.1.3 Explanation of ICCs in the `psych` output

If you want to know all the details, refer to Shrout and Fleiss (1979). The help-function `?psych::ICC` contains a relatively good and much shorter explanation. The variance formulae given in the help-file are probably somewhat confusing. We try to stick to the notation of the book.

Let's talk about the first three **ICCs in the `psych` output**:

- **Single_raters_absolute ICC1:** According to the help file: "Each target [i.e. patient] is rated by a different judge [i.e. rater] and the judges are selected at random." So, variability due to raters is implied and cannot be disentangled. This is formally not our situation, since we have only two raters and 50 patients. But for this case, we do not care who measures, since we do not model it, hence, we cannot know if there are systematic differences between the raters. There might as well be 50 raters doing their thing, or just 2 as in our case. This is the ICC we have calculated above; the overall ICC in the book and it is based on the following model:

  $$Y_{ij} = \eta_i + \varepsilon_i$$

  where $i \in 1, ..., 50$ is the *patient* and $j \in 1, 2$ is the *measurement* ($= 50 * 2 =$ 100 rows in long format). Note that we do not mention a rater here, since we do not care who took the measurement. It is not part of the model. The ICC is then calculated as:

  $$ICC = \frac{\sigma^2_\eta}{\sigma^2_\eta + \sigma^2_\varepsilon}$$

  whereas we could get the variance components from either the posterior in the Bayesian setting or from the `lmer` output in the Frequentist setting.

- **Single_random_raters ICC2:** ICC2 ($= ICC_{agreement}$ in the book) and ICC3 ($= ICC_{consistency}$ in the book) are based on the **same (!)** statistical model. The only difference is that ICC2 assumes that the (in our case) 2 raters are randomly selected from a larger pool of raters, hence, the rater variability must be explicitly considered and yields a potentially smaller value for the ICC. Compared to ICC1, we have repeated measurements from the same raters in 50 patients. That's why we can model their bias. One observation would not be enough. The help file says: "A random sample of k judges rate each target. The measure is one of absolute agreement in the ratings." A random sample of k (2 in our case) judges means that we cannot rule out the variability due to raters (you get a variety of them and their biases are different).

  $$Y_{ij} = \eta_i + \beta_j + \varepsilon_i$$

where $i \in 1, ..., 50$ is the *patient*, $j \in 1, 2$ is the **rater** (doing one measurement in each patient). The ICC is then calculated as:

$$ICC_{agreement} = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \underset{\textbf{rater}}{\textbf{2}} + \sigma_\varepsilon^2}$$

- **Single_fixed_raters ICC3:** ICC3 (= $ICC_{consistency}$ in the book) is based on the **same model as ICC2**, but assumes that the raters are fixed. This means that **the raters are the same for all patients** in the future study. So, we have considered the rater variability in the model (which was possible due to the repeated measurements from the same raters in 50 patients), but do not care since Mary and Peter will be the people doing the future measurements, not other therapists. If you fix a random variable (*raters* in this case), variance is zero. The help file says: "A fixed set of k judges rate each target. There is no generalization to a larger population of judges." The ICC is then calculated as:

$$ICC_{consistency} = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2}$$

Important: ICC1 and ICC3 are **not** identical, since ICC1 does not consider the rater variability in the model. They are based on *different* statistical models. ICC2 and ICC3 are based on the *same* model.

If there is no systematic difference between raters, all 3 ICCs and the Pearson correlation (r) are the same (see Figure 5.3 in the book).

$ICC_{consistency}$ vs. $ICC_{agreement}$: The latter is used, when we need Peter and Mary to concur in their measurements. Patients coming to Peter's practice will get the same (or very similar) "diagnosis" (ROM-value) from Mary. When there is systematic difference (line is shifted downwards in Figure 5.3), this cannot be guaranteed. If we only need Peter and Mary to *rank* the patients in the same order, we can use $ICC_{consistency}$.

- **Average_raters_absolute ICC1k:** According to the help file "ICC1k, ICC2k, ICC3k reflect the means of k raters." In general, if you take the mean, you get an estimate with lower variance (central limit theorem). In the book on page 109, we find the derivation of the ICC for the average of k raters. We do not need a new statistical model, although we take the sum (or equivalently, the mean) of the measurements of Peter and Mary, which seems a bit unintuitive at first.

  If we go back to the definition of the ICC above, it follows that:

$$Y_1 + Y_2 = 2\eta + \varepsilon_1 + \varepsilon_2$$

for the same person with true η. Since the errors ε_i are independent and the true score η is also independent from the errors, it follows that:

$$\begin{aligned}\mathbb{V}\text{ar}(Y_1 + Y_2) = \mathbb{V}\text{ar}(2\eta + \varepsilon_1 + \varepsilon_2) = \\ \mathbb{V}\text{ar}(2\eta) + \mathbb{V}\text{ar}(\varepsilon_1 + \varepsilon_2) = \\ 4\mathbb{V}\text{ar}(\eta) + 2\mathbb{V}\text{ar}(\varepsilon) =\end{aligned}$$

If we divide by 4, we get:

$$\frac{1}{4}\mathbb{V}\text{ar}(Y_1 + Y_2) = \mathbb{V}\text{ar}(\frac{Y_1 + Y_2}{2}) = \mathbb{V}\text{ar}(\eta) + \frac{\mathbb{V}\text{ar}(\varepsilon)}{2} = \sigma^2_\eta + \frac{\sigma^2_\varepsilon}{2}$$

Hence, the ICC for the average of 2 raters is ($k = 2$):

$$ICC1k = \frac{\sigma^2_\eta}{\sigma^2_\eta + \frac{\sigma^2_\varepsilon}{2}}$$

Again, the model does not care about the raters, it just knows that there are two measurements. A potential bias between raters is not considered.

It is also interesting to note that we do not fit a new model to get the ICC1k. We could not compare the first average (of 2 measurements) to the second average (of 2 measurements), since we only have 2 measurements per patient.

```
m <- lmer(ROM ~ (1|ID), data = df_long_bias)
summary(m)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: ROM ~ (1 | ID)
##    Data: df_long_bias
##
## REML criterion at convergence: 797.3
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.02334 -0.52483 -0.03788  0.57830  1.59879
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  ID       (Intercept) 267.79   16.364
##  Residual              53.75    7.331
## Number of obs: 100, groups:  ID, 50
##
## Fixed effects:
```

```
##             Estimate Std. Error t value
## (Intercept)   68.090      2.428   28.05
```

```
print(VarCorr(m), comp = "Variance")
```

```
##  Groups   Name        Variance
##  ID       (Intercept) 267.79
##  Residual              53.75
```

```
# ICC1k =
267.79 / (267.79 + 53.75/2)
```

```
## [1] 0.9087947
```

This is identical to the ICC1k in the `psych` output.

ICC1k would approach 1 if k goes to infinity. **This makes sense**, since the *true* measurement is *defined* as the average of infinite measurements (of either infinitely many different raters or infinitely many measurements from the same rater). Also it makes intuitive sense, that reliability increases or equivalently the unexplained variance decreases since we draw repeatedly from the same distribution (η and an error term ε) and the central limit theorem guarantees that we can estimate η as precisely as we want (variance of the mean goes to zero) if we only draw often enough.

- **Average_raters_random ICC2/3k:** ICC2k and ICC3k are based on the same model as ICC2 and ICC3, respectively. We just need to divide the respective residual variance by k (in our case 2) to get the ICC for the average of k raters.

```
m5.3 <- lmer(ROM ~ (1 | ID) + (1 | Rater), data = df_long_bias)
summary(m5.3)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: ROM ~ (1 | ID) + (1 | Rater)
##    Data: df_long_bias
##
## REML criterion at convergence: 793.2
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.87448 -0.46270  0.00272  0.57820  1.45008
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  ID       (Intercept) 270.882  16.458
##  Rater    (Intercept)   6.193   2.489
##  Residual              47.557   6.896
## Number of obs: 100, groups:  ID, 50; Rater, 2
```

```
##
## Fixed effects:
##             Estimate Std. Error t value
## (Intercept)   68.090      2.998   22.71
```

```
print(VarCorr(m5.3), comp = "Variance")
```

```
##  Groups   Name        Variance
##  ID       (Intercept) 270.882
##  Rater    (Intercept)   6.193
##  Residual              47.557
```

```
# Groups   Name        Variance
# ID       (Intercept) 270.882
# Rater    (Intercept)   6.193
# Residual              47.557

# ICC2k =
270.882 / (270.882 + (6.193 + 47.557) / 2)
```

```
## [1] 0.9097418
```

```
# ICC3k =
270.882 / (270.882 + 47.557 / 2)
```

```
## [1] 0.919302
```

These values are identical to the ICC2k and ICC3k in the `psych` output.

2.1.4 Summary Peter and Mary, with and without bias (ICC1-3)

Below, we summarize the results for the ICCs (calculated with `psych`) for the unbiased and biased case (Mary measures on average 5 degrees more than Peter).

```
library(pacman)
p_load(conflicted, tidyverse, flextable)

# Ensure select() from dplyr is used
#conflicted::conflicts_prefer("select", "dplyr")

# Unbiased ICC Calculation
df_wide_unbiased <- df_long %>%
  pivot_wider(names_from = Rater, values_from = ROM)
df_wide_values_unbiased <- df_wide_unbiased %>% dplyr::select(-ID)
icc_results_unbiased <- psych::ICC(df_wide_values_unbiased)
```

```
## boundary (singular) fit: see help('isSingular')
```

Table 2.1: Intraclass Correlation Coefficients - Unbiased vs. Biased

ICC Type	Unbiased ICC	Biased ICC
ICC1	0.8512574	0.8328336
ICC2	0.8512574	0.8344281
ICC3	0.8512574	0.8506562

```
# Extract relevant ICC values
icc_unbiased_df <- icc_results_unbiased$results %>%
  dplyr::select(type, ICC) %>%
  rename(`Unbiased ICC` = ICC)

# Biased ICC Calculation
df_wide_biased <- df_long_bias %>%
  pivot_wider(names_from = Rater, values_from = ROM)
df_wide_values_biased <- df_wide_biased %>% dplyr::select(-ID)
icc_results_biased <- psych::ICC(df_wide_values_biased)

# Extract relevant ICC values
icc_biased_df <- icc_results_biased$results %>%
  dplyr::select(type, ICC) %>%
  dplyr::rename(`Biased ICC` = ICC)

icc_merged_df <- left_join(icc_unbiased_df,
                           icc_biased_df,
                           by = "type") %>%
  slice(1:3)

if (knitr::is_html_output(excludes = "epub")) {
  ft <- flextable(icc_merged_df) %>%
    flextable::set_header_labels(type = "ICC Type") %>%
    flextable::set_caption("Intraclass Correlation Coefficients - Unbiased vs. Biased")
    flextable::set_table_properties(width = .5, layout = "autofit")
  ft
} else {
  knitr::kable(
    icc_merged_df,
    col.names = c("ICC Type", "Unbiased ICC", "Biased ICC"),
    booktabs = TRUE,
    caption = "Intraclass Correlation Coefficients - Unbiased vs. Biased"
  )
}
```

The **left column** shows that the ICCs are identical for the **unbiased case**. Specifically, ICC2 (=$ICC_{agreement}$ in the book) and ICC3 ($ICC_{consistency}$ in the book) are based on the same model which explicitly considers a potential bias between the raters. Since there is none, the ICCs are the same.

In the **biased case** (right column), there *is* a systematic difference between Mary and Peter - 5 degrees in our case.

ICC1 does not care about it and shows a somewhat lower value compared to before (0.833). The reason is because the agreement line ($y = x$) in a plot with Mary on Y and Peter on X is shifted upwards by 5 degrees. If you would introduce a bias of 15 degrees, the ICC would be even lower ($ICC1 = 0.61$, $ICC2 = 0.65$, exercise 1). The unbiased column would of course stay the same.

ICC2 now considers the bias of 5 degrees. The model knows about the shift. If we compare the variance components of ICC1 and ICC2, we see:

```
## [1] "Model for ICC1: ROM ~ (1 | ID)"
```

```
##  Groups   Name        Variance
##  ID       (Intercept) 267.79
##  Residual              53.75
```

```
## [1] "Model for ICC2 (and 3): ROM ~ (1 | ID) + (1 | Rater)"
```

```
##  Groups   Name        Variance
##  ID       (Intercept) 270.882
##  Rater    (Intercept)   6.193
##  Residual              47.557
```

The residual variance is smaller in the model with the rater effect. The model explains the data better, since it knows about the bias (see exercise 7).

Look at the σ^2_ε of the two models, they add up:

$$\sigma^2_{\varepsilon,ICC1} = \sigma^2_{\varepsilon,ICC23} + \sigma^2_{Rater}$$
$$53.75 = 47.557 + 6.193$$

Hence, we just split up the error differently by considering the bias. The patient variability (`ID Variance` in the output) is slightly higher: 270.882 compared to 267.79 before. In the first model, patient variability was conflated with rater variation/systematic difference because rater effects were not explicitly modeled. For this reason, `ID Variance` increases slightly. It is a rather small increase, so ICC1 and ICC2 are not that different.

For a bias of 15 degrees, the additivity of the variances remains. The patient variability increases from 223.89 (ICC1) to 270.882 (ICC2), hence the difference in ICCs is larger ($ICC1 = 0.613$ vs. $ICC2 = 0.657$).

ICC3 considers the bias in the model but does not include it in the measurement error since the raters are fixed.

2.1.5 Difference between correlation and ICC

If we do not introduce a bias in the data, the correlation coefficient is the same as the ICC (as seen above). On page 110, Figure 5.3, the authors show nicely what the difference is between the correlation coefficient and the ICC. It is also shown how $ICC_{agreement}$ and $ICC_{consistency}$ change with the bias.

$ICC_{consistency}$ stays 1 if bias is introduced (assuming a hypothetical perfect agreement before). Peter and Mary still **rank** the patients in the same order.

Correlation r is always 1, no matter at what slope ($\neq 0$) the line is. It measures the strength and direction of the *linear* relationship between two variables.

$ICC_{agreement}$ changes as soon as you depart from the 45 degree line with respect to slope or shift the line up or down (i.e., introduce a bias).

This is a good point in time to think for a moment about the type of measurement for agreement with respect to **costs**. The ICC below weights each measurement equally, although one larger outlier (large discrepancy between Peter and Mary) may influence the ICC notably (exercise 8). One could possibly think of a situation where overall agreement measure like the ICC is not adequate since, for instance, exceeding a certain difference threshold could decide between life and death.

2.1.6 Bad news about the ICC?

Let's try to expand our intuitive understanding of what an ICC of 0.8 or so means. For simplicity, we take the overall ICC, which is equal to the correlation coefficient, if there is no bias present.

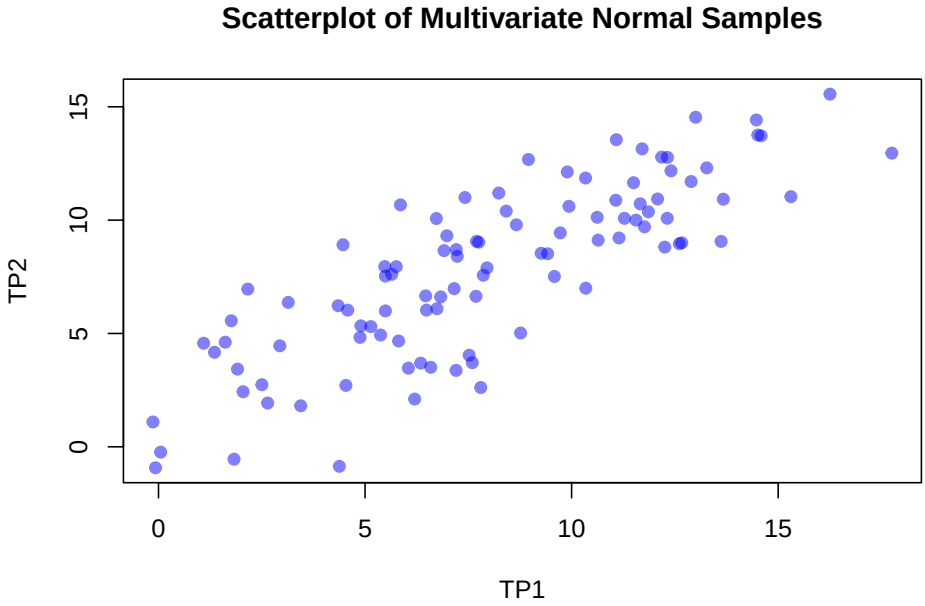
The following example demonstrates the meaning of the Test-Retest reliability of the Hospital Anxiety and Depression Scale - Anxiety subscale (HADS-A). We could take all kinds of other scores where ICC values and a minimally clinically important difference (MCID) is given. Briefly, the MCID is the smallest change in a score that is considered important to the patient. For example, a 5% change in BMI is (in some populations) considered meaningful.

Specifically, we:

- Simulate two correlated measurements at two time points (TP1 and TP2) to get predetermined ICC ($= \rho$).
- Calculate the Intraclass Correlation Coefficient (ICC).
- Compare the Minimally Clinically Important Difference (MCID) to prediction intervals.
- Visualize how often a clinically meaningful change of 1.68 points is detected, even if no real change has occurred.

The code can be found here and in the github repo of the script. In the code, the sources for the ICC and MCID are cited.

```r
# ICC and Test-Retest-Reliability

library(pacman)
p_load(tidyverse, lme4, conflicted, psych, MASS)

# MCID Minimal Clinically Important Difference---------

# HADS score---------
# The Hospital Anxiety and Depression Scale

# Test-Retest-Reliability:
# https://doi.org/10.1016/S1361-9004(02)00029-8

# Use the numbers from here (Table 1):
# https://www.sciencedirect.com/science/article/abs/pii/S1361900402000298
# for demonstration purposes.

# Minimal Clinically Important Difference (MCID) for HADS-A:
# https://pmc.ncbi.nlm.nih.gov/articles/PMC2459149/
# Not exactly the same population as shift workers, but suffices for demonstration purposes.
# MCID HADS anxiety score and 1.68 (1.48-1.87)

# Simplification: Score is deemed to be continuous.

# Create 2 correlated measurements:
# Use HADS-A Anxiety subscale
# Use n=100, instead of n=24 for more stability

sigma1 <- 3.93  # Standard deviation of variable 1
sigma2 <- 3.52  # Standard deviation of variable 2
rho <- 0.82    # Correlation
cov_matrix <- matrix(c(sigma1^2, rho * sigma1 * sigma2,
                       rho * sigma1 * sigma2, sigma2^2), nrow = 2)
n <- 100

set.seed(188)  # For reproducibility
samples <- mvrnorm(n = n, mu = c(7.92, 7.83), Sigma = cov_matrix)
df <- as.data.frame(samples)
colnames(df) <- c("TP1", "TP2")

plot(df, main = "Scatterplot of Multivariate Normal Samples",
     xlab = "TP1", ylab = "TP2", pch = 19, col = rgb(0, 0, 1, alpha = 0.5))
```




```
# https://en.wikipedia.org/wiki/Intraclass_correlation#Modern_ICC_definitions:_simpler
# ICC should be the correlation within the group (i.e. patient)

cor(df$TP1, df$TP2, method = "pearson") # ~0.8-0.9
```

```
## [1] 0.8184881
```

```
ICC(df) # 0.82
```

```
## boundary (singular) fit: see help('isSingular')
```

```
## Call: ICC(x = df)
##
## Intraclass correlation coefficients
##                              type  ICC  F df1 df2       p lower bound upper bound
## Single_raters_absolute       ICC1 0.82 10  99 100 5.6e-26        0.74        0.87
## Single_random_raters         ICC2 0.82 10  99  99 8.7e-26        0.74        0.87
## Single_fixed_raters          ICC3 0.82 10  99  99 8.7e-26        0.74        0.87
## Average_raters_absolute     ICC1k 0.90 10  99 100 5.6e-26        0.85        0.93
## Average_random_raters       ICC2k 0.90 10  99  99 8.7e-26        0.85        0.93
## Average_fixed_raters        ICC3k 0.90 10  99  99 8.7e-26        0.85        0.93
##
##  Number of subjects = 100     Number of Judges =  2
## See the help file for a discussion of the other 4 McGraw and Wong estimates,
```

```
# check manually
df_mod <- data.frame(id = 1:n, df$TP1, df$TP2)
names(df_mod) <- c("id", "TP1", "TP2")
```

```
df_mod_long <- df_mod %>% pivot_longer(cols = c(TP1, TP2),
                                       names_to = "time_point",
                                       values_to = "score")
mod <- lme4::lmer(score ~ time_point + (1|id), data = df_mod_long)
variance_df <- as.data.frame(summary(mod)$varcor)
# ICC=
variance_df$sdcor[1]^2 / (variance_df$sdcor[1]^2 + variance_df$sdcor[2]^2) # ~0.9
```

```
## [1] 0.8170566
```

```
# cor and ICC are very very similar, should actually be identical.

#data.frame(TP1 = df$TP1, TP2 = df$TP2) %>%
#  ggplot(aes(x = TP1, y = TP2)) +
#  geom_point() +
#  xlab("Measurement of patients at time point 1") +
#  ylab("Measurement of the same patients at time point 2")

# Range for HADS-A should be 0-21
# (according to "The Hospital Anxiety and Depression Scale, Zigmond & Snaith, 1983")

df <- data.frame(TP1 = df$TP1, TP2 = df$TP2) %>%
  dplyr::filter(TP1 >= 0) %>%
  dplyr::filter(TP2 >= 0) %>% # negative not possible
  dplyr::filter(TP1 <= 21) %>%
  dplyr::filter(TP2 <= 21) # max. score is 21
df
```

```
##          TP1       TP2
## 1  13.003089 14.539674
## 2   6.980669  9.312346
## 3  17.751972 12.957041
## 4  10.344489  6.995323
## 5   7.861347  7.568915
## 6  12.892886 11.703021
## 7   5.478497  7.952075
## 8   9.583384  7.515242
## 9   7.519451  4.043160
## 10 11.499240 11.652541
## 11  6.468033  6.660218
## 12  6.048513  3.471018
## 13  7.697834  9.065112
## 14 12.254369  8.811619
## 15  1.614857  4.614998
## 16 10.619077 10.125575
## 17  9.419930  8.518226
```

```
## 18  2.937867  4.456727
## 19  8.236834 11.196117
## 20  8.954668 12.675367
## 21 12.320423 12.768905
## 22 12.314754 10.081370
## 23  2.160930  6.959311
## 24  6.832022  6.613443
## 25  1.093221  4.571882
## 26  9.725573  9.440133
## 27  1.355585  4.171266
## 28  7.752631  9.028429
## 29  8.764755  5.020493
## 30  6.347606  3.694110
## 31  5.375775  4.932552
## 32 11.278457 10.079779
## 33  6.740993  6.092677
## 34  5.812009  4.668937
## 35  4.349329  6.225931
## 36  9.895140 12.126645
## 37 11.148574  9.213335
## 38  2.047422  2.430490
## 39  6.486131  6.032311
## 40  5.645763  7.611138
## 41  6.726079 10.071178
## 42  5.755609  7.947849
## 43 10.642786  9.117806
## 44  5.141220  5.304112
## 45  2.642085  1.933020
## 46  7.684671  6.640758
## 47  9.262502  8.538370
## 48 16.254728 15.559716
## 49  7.156983  6.978574
## 50 12.081541 10.935277
## 51 11.764108  9.705782
## 52 11.558231 10.001998
## 53  7.207230  8.702822
## 54 13.271195 12.305426
## 55  8.662632  9.794167
## 56  7.202986  3.372489
## 57  5.857393 10.673803
## 58 12.606715  8.961083
## 59 11.081861 13.549278
## 60  4.900769  5.337160
## 61 15.309336 11.034889
## 62 14.589373 13.720623
## 63 13.671753 10.925029
```

```
## 64 12.180305 12.781404
## 65  1.762711  5.561035
## 66  3.140391  6.372191
## 67  5.492213  5.996299
## 68  4.466991  8.914347
## 69  2.502974  2.742313
## 70 12.667048  9.000329
## 71  4.580641  6.028937
## 72 11.858807 10.368234
## 73  4.537198  2.709857
## 74  7.419888 10.998858
## 75  4.879237  4.827545
## 76 13.618764  9.062266
## 77 14.472807 14.418930
## 78  7.952580  7.896180
## 79  1.912410  3.428163
## 80  7.801004  2.616797
## 81 10.336987 11.858491
## 82  8.417477 10.399787
## 83  5.491992  7.532053
## 84  7.596043  3.712851
## 85  6.912368  8.655686
## 86  6.201155  2.107871
## 87 11.707088 13.145075
## 88  3.443512  1.810893
## 89 12.406231 12.175912
## 90  7.231295  8.403967
## 91 11.070856 10.877902
## 92 14.508512 13.754604
## 93  9.936188 10.610818
## 94  6.590862  3.507825
## 95 11.660322 10.722280
```

```
mod <- lm(TP2 ~ TP1, data = df)
pred <- predict(mod, df, interval = "prediction")

# How wide are the prediction intervals for a patient?
as.data.frame(pred) %>% mutate(width_prediction_interval = upr - lwr) # width of the prediction i
```

```
##          fit         lwr       upr width_prediction_interval
## 1  11.517789  7.32000445 15.715573                   8.395568
## 2   7.260698  3.09356695 11.427829                   8.334262
## 3  14.874649 10.57640522 19.172893                   8.596488
## 4   9.638494  5.46784345 13.809144                   8.341301
## 5   7.883226  3.71851506 12.047937                   8.329422
## 6  11.439889  7.24365214 15.636126                   8.392473
```

```
## 7   6.198852  2.02213714 10.375568                8.353431
## 8   9.100489  4.93366030 13.267318                8.333658
## 9   7.641549  3.47617950 11.806918                8.330738
## 10 10.454757  6.27494432 14.634571                8.359626
## 11  6.898329  2.72869927 11.067959                8.339260
## 12  6.601781  2.42951087 10.774051                8.344541
## 13  7.767643  3.60266181 11.932624                8.329962
## 14 10.988538  6.80055090 15.176525                8.375974
## 15  3.467747 -0.76481930  7.700313                8.465132
## 16  9.832593  5.66013066 14.005055                8.344925
## 17  8.984947  4.81870889 13.151186                8.332477
## 18  4.402947  0.19450442  8.611390                8.416885
## 19  8.148648  3.98424793 12.313048                8.328800
## 20  8.656066  4.49106133 12.821072                8.330010
## 21 11.035230  6.84644646 15.224014                8.377568
## 22 11.031223  6.84250820 15.219938                8.377430
## 23  3.853751 -0.36823502  8.075737                8.443972
## 24  7.155623  2.98785021 11.323396                8.335546
## 25  3.099016 -1.14446756  7.342499                8.486967
## 26  9.200999  5.03359021 13.368407                8.334817
## 27  3.284474 -0.95341998  7.522367                8.475787
## 28  7.806378  3.64149615 11.971259                8.329763
## 29  8.521822  4.35713015 12.686514                8.329383
## 30  6.813202  2.64286933 10.983535                8.340666
## 31  6.126241  1.94861984 10.303862                8.355242
## 32 10.298692  6.12094325 14.476441                8.355497
## 33  7.091277  2.92307764 11.259477                8.336399
## 34  6.434603  2.26060789 10.608598                8.347990
## 35  5.400673  1.21224884  9.589097                8.376848
## 36  9.320861  5.15268035 13.489042                8.336361
## 37 10.206881  6.03027765 14.383484                8.353206
## 38  3.773515 -0.45059795  7.997629                8.448227
## 39  6.911122  2.74159427 11.080650                8.339056
## 40  6.317088  2.14177936 10.492397                8.350617
## 41  7.080735  2.91246298 11.249007                8.336544
## 42  6.394735  2.22030395 10.569166                8.348862
## 43  9.849352  5.67672294 14.021982                8.345259
## 44  5.960440  1.78063088 10.140249                8.359619
## 45  4.193867 -0.01952182  8.407256                8.426777
## 46  7.758338  3.59333195 11.923345                8.330013
## 47  8.873666  4.70791894 13.039413                8.331494
## 48 13.816287  9.55673194 18.075843                8.519111
## 49  7.385330  3.21887319 11.551787                8.332914
## 50 10.866371  6.68040625 15.052336                8.371929
## 51 10.641986  6.45950156 14.824470                8.364969
## 52 10.496456  6.31606656 14.676846                8.360780
```

```
## 53  7.420848  3.25456616 11.587130                8.332564
## 54 11.707306  7.50560608 15.909006                8.403400
## 55  8.449634  4.28506471 12.614202                8.329138
## 56  7.417848  3.25155183 11.584145                8.332593
## 57  6.466683  2.29303270 10.640334                8.347301
## 58 11.237603  7.04521442 15.429991                8.384777
## 59 10.159723  5.98368848 14.335757                8.352069
## 60  5.790471  1.60824613  9.972696                8.364450
## 61 13.148015  8.90959977 17.386429                8.476830
## 62 12.639092  8.41504000 16.863143                8.448103
## 63 11.990450  7.78250179 16.198399                8.415897
## 64 10.936184  6.74907482 15.123294                8.374219
## 65  3.572261 -0.65735481  7.801876                8.459231
## 66  4.546106  0.34090074  8.751312                8.410411
## 67  6.208548  2.03195085 10.385144                8.353194
## 68  5.483845  1.29682048  9.670870                8.374049
## 69  4.095533 -0.12027142  8.311337                8.431609
## 70 11.280250  7.08707078 15.473429                8.386359
## 71  5.564181  1.37846891  9.749893                8.371425
## 72 10.708926  6.52543503 14.892417                8.366982
## 73  5.533472  1.34726300  9.719682                8.372419
## 74  7.571170  3.40554231 11.736798                8.331256
## 75  5.775251  1.59280130  9.957701                8.364900
## 76 11.952994  7.74589920 16.160088                8.414189
## 77 12.556694  8.33482530 16.778563                8.443737
## 78  7.947716  3.78312039 12.112312                8.329192
## 79  3.678079 -0.54861400  7.904772                8.453386
## 80  7.840572  3.67577039 12.005373                8.329602
## 81  9.633192  5.46258745 13.803796                8.341208
## 82  8.276340  4.11193632 12.440744                8.328808
## 83  6.208392  2.03179294 10.384990                8.353197
## 84  7.695690  3.53049953 11.860881                8.330381
## 85  7.212418  3.04500055 11.379836                8.334835
## 86  6.709680  2.53843261 10.880928                8.342495
## 87 10.601680  6.41978859 14.783572                8.363783
## 88  4.760375  0.55978683  8.960963                8.401176
## 89 11.095886  6.90604714 15.285724                8.379677
## 90  7.437859  3.27165782 11.604060                8.332402
## 91 10.151944  5.97600166 14.327886                8.351884
## 92 12.581933  8.35939957 16.804466                8.445066
## 93  9.349877  5.18149616 13.518258                8.336761
## 94  6.985154  2.81619604 11.154112                8.337916
## 95 10.568622  6.38720944 14.750034                8.362825
```

```
# Example:
predict(mod, newdata = data.frame(TP1 = 10), interval = "prediction") # 95% prediction interval j
```

```
##        fit      lwr      upr
## 1 9.394984 5.226282 13.56369
```

```
(13.56369 - 5.226282)/1.68
```

```
## [1] 4.962743
```

```
# Prediction interval width is ~5 times our minimally clinically important
# change of 1.68 for HADS-A.

# SEM = sd_difference/sqrt(2)
(SEM <- sd(df$TP1 - df$TP2)/sqrt(2)) # 1.663973
```

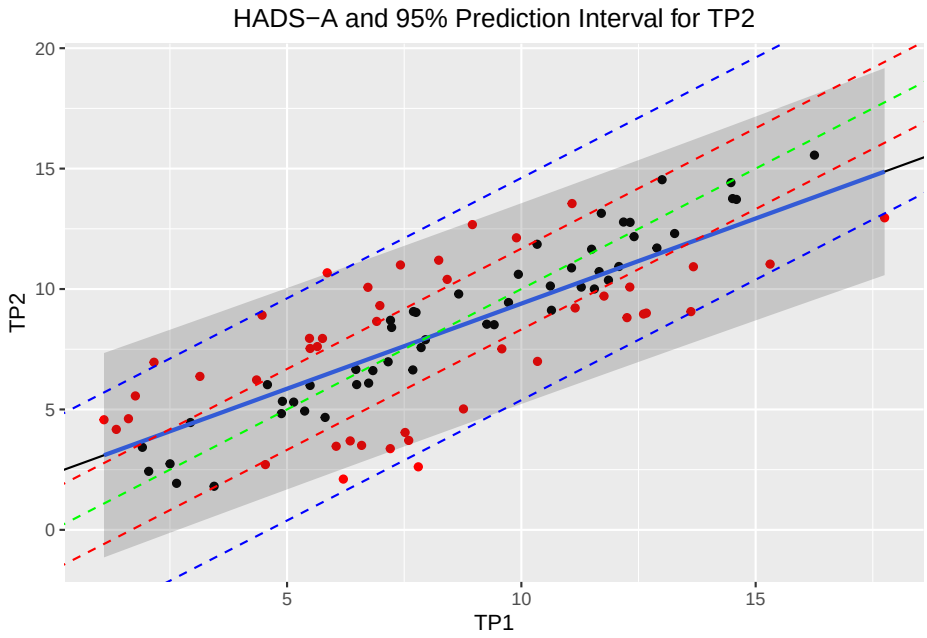
```
## [1] 1.663973
```

```
# SDC = 1.96*sqrt(2)*SEM
(SDC <- 1.96 * sqrt(2) * SEM) # 4.61
```

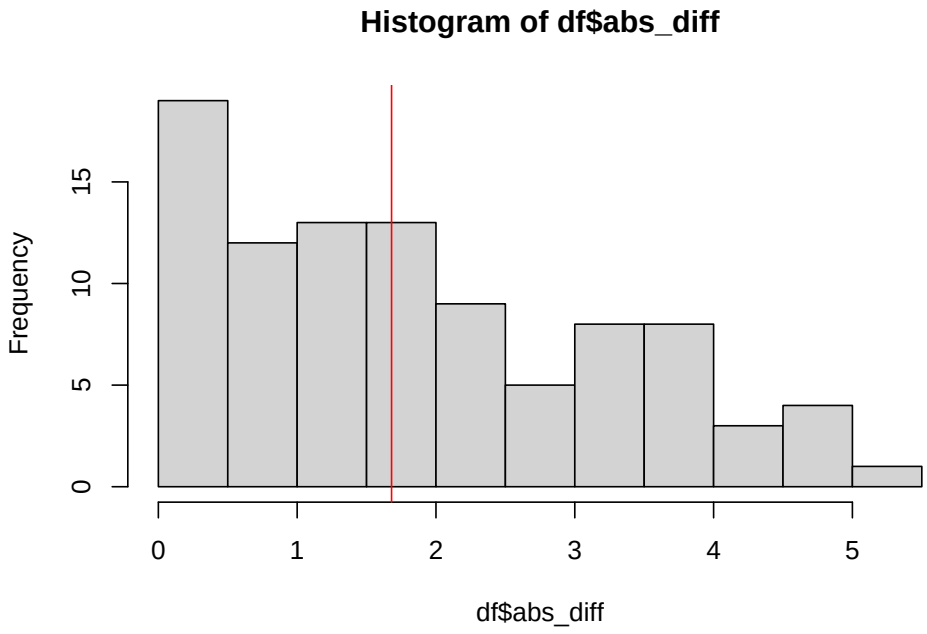
```
## [1] 4.612299
```

```
df %>%
  ggplot(aes(x = TP1, y = TP2)) +
  # Color points conditionally
  geom_point(aes(color = ifelse(TP2 > TP1 + 1.68 | TP2 < TP1 - 1.68, "red", "black"))) +
  scale_color_manual(values = c("red" = "red", "black" = "black"), guide = "none") +
  geom_abline(intercept = mod$coefficients[1], slope = mod$coefficients[2]) +
  geom_smooth(method = "lm", se = FALSE) +
  geom_ribbon(aes(ymin = pred[,2], ymax = pred[,3]), alpha = 0.2) +
  ggtitle("HADS-A and 95% Prediction Interval for TP2") +
  theme(plot.title = element_text(hjust = 0.5)) +
  geom_abline(intercept = 0, slope = 1, color = "green", linetype = "dashed") +
  geom_abline(intercept = 1.68, slope = 1, color = "red", linetype = "dashed") + # MCI
  geom_abline(intercept = -1.68, slope = 1, color = "red", linetype = "dashed") +  # M
  geom_abline(intercept = SDC, slope = 1, color = "blue", linetype = "dashed") + # SDC
  geom_abline(intercept = -SDC, slope = 1, color = "blue", linetype = "dashed") # SDC
```

```
## `geom_smooth()` using formula = 'y ~ x'
```




```r
# How often is TP2 within the MCIC of 1.68 points?---------
df$abs_diff <- abs(df$TP1 - df$TP2)
hist(df$abs_diff)
abline(v = 1.68, col = "red")
```



```r
table(df$abs_diff > 1.68)/sum(table(df$abs_diff > 1.68)) #
```

```
##
##     FALSE      TRUE
## 0.5368421 0.4631579
```

```r
# -> in ~46% of cases we detect a minimally clinically important change of 1.68 points
# even though the underlying truth did not change.

# This result is near random guessing
```

The example shows that for the given value of ICC (> 0.8) and MCID, the test retest reliability is near random guessing with respect to detecting a change, since the second measurement is very often outside the MCID (red dashed lines around the green dashed line of perfect identity).

The blue dashed lines show the *Smallest Detectable Change (SDC)*, which is the smallest change that can be detected statistically. As one would expect (by definition), only few points are outside this range.

In our case of artificially generated data, the SDC is larger than the MCID. This is also true in the literature.

Hence, we are in the case (b) of Figure 8.11 of the book "Measurement in Medicine" (p.260). Ideally, we want to be in case (a), where the SDC is smaller than the MCID (see also section 8.5.5. in the book).

To answer the question in the title: On an individual level, instruments such as the HADS-A are not able to detect a clinically meaningful change at the individual level based on a single measurement.

The instrument is useful again if one takes the mean of several measurements or groups of patients are compared (see p.261).

See exercise 4 to try the above code with a score of your choosing.

2.1.7 Standard Error of Measurement (SEM)

Above, we defined the overall ICC as:

$$ICC = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2}$$

The SEM is defined as: $\sqrt{\sigma_\varepsilon^2}$.

This is a **measure for the precision of the model**. If this number is small, we know the true but unknown score (η) very precisely. As we have seen, for the $ICC_{agreement}$ the error term includes the rater variability σ_{rater}.

For ICC2 and 3 the model is the **same** and:

$$Y_{ij} = \eta_i + \beta_j + \varepsilon_i$$

For $ICC_{agreement}$, the SEM is then:

$$\sigma^2_{Y_i} = \sigma^2_\eta + \underbrace{\sigma^2_{\text{rater}} + \sigma^2_\varepsilon}_{SEM^2_{agreement}}$$

For $ICC_{consistency}$, the SEM is then:

$$\sigma^2_{Y_i} = \sigma^2_\eta + \underbrace{\sigma^2_\varepsilon}_{SEM^2_{consistency}}$$

And since ICC 2 and 3 are based on the same model:

$$SEM_{consistency} \leq SEM_{agreement}$$

(equality if there is no bias)

This yields the **first method** of getting the SEM: Estimate the model (as we did above) and take the square root of the respective error term (with or without the rater variability).

One can also show that we can find the $SEM_{consistency}$ by using the standard deviation of the differences between the two measurements of Peter and Mary:

We verify this immediately:

$$SEM_{consistency} = \frac{SD_{difference}}{\sqrt{2}}$$

```
mod <- lmer(ROM ~ (1 | ID) + (1 | Rater), data = df_long_bias)
print(VarCorr(mod))
```

```
##  Groups   Name        Std.Dev.
##  ID       (Intercept) 16.4585
##  Rater    (Intercept)  2.4886
##  Residual              6.8961
```

$$SEM_{consistency} = \sigma_{residual} = 6.8961$$

Let's calculate the **SEM from the differences** between the two measurements:

```
# SEM from differences
sd(df_long_bias$ROM[df_long_bias$Rater == "ROMas.Mary"] -
   df_long_bias$ROM[df_long_bias$Rater == "ROMas.Peter"]) / sqrt(2)
```

```
## [1] 6.896154
```

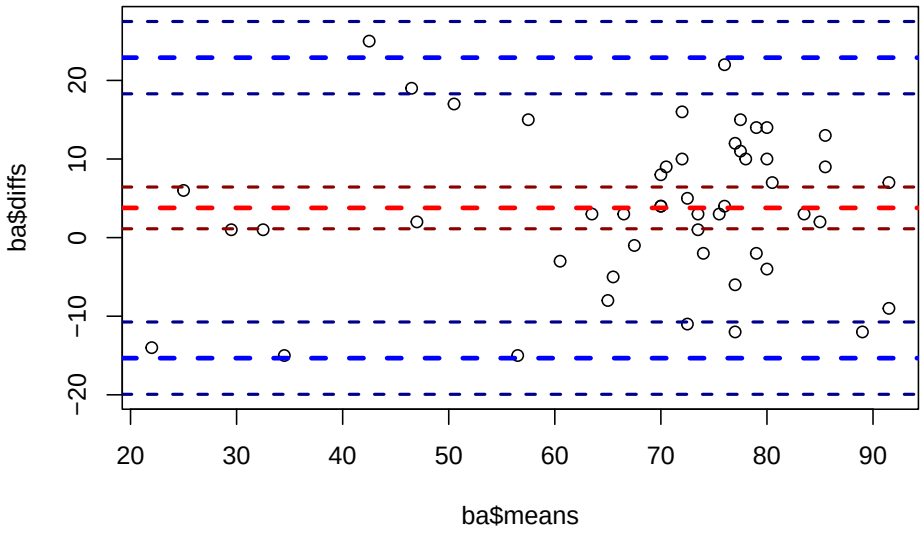
This is a perfect match!

2.1.8 Bland-Altman Plot

The Bland-Altman plot (see 5.4.2.2. in the book) is a graphical method to compare two measurements techniques or raters. The goal is to spot systematic differences between the two methods/raters. On the x axis, we plot the average of the two measurements (proxy for the true value) and on the y axis the difference between the two measurements. So, we could for instance see larger variability in points with higher average values, indicating that Peter and Mary disagree more for higher values.

Let's quickly draw one and explain it:

```
library(BlandAltmanLeh)
library(data.table)
df_long_bias <- as.data.table(df_long_bias)
# https://cran.r-project.org/web/packages/BlandAltmanLeh/BlandAltmanLeh.pdf
bland.altman.plot(df_long_bias[Rater == "ROMas.Mary", ]$ROM,
                  df_long_bias[Rater == "ROMas.Peter", ]$ROM,
                  two = 1.96,
                  mode = 1,
                  conf.int = 0.94
                  )
```




```
## NULL
```

```
# "two": Lines are drawn "two" standard deviations from mean differences.
# This defaults to 1.96 for proper 95 percent confidence interval
# estimation but can be set to 2.0 for better agreement with e. g.
# the Bland Altman publication.

# "mode": if 1 then difference group1 minus group2 is used,
```

```
# if 2 then group2 minus group1 is used. Defaults to 1.

# "conf.int": confidence interval for the mean difference
# and the limits of agreement.

# The red dotted line is the mean difference $\bar{d}$
# between the two measurements.

# see ?bland.altman.plot
```

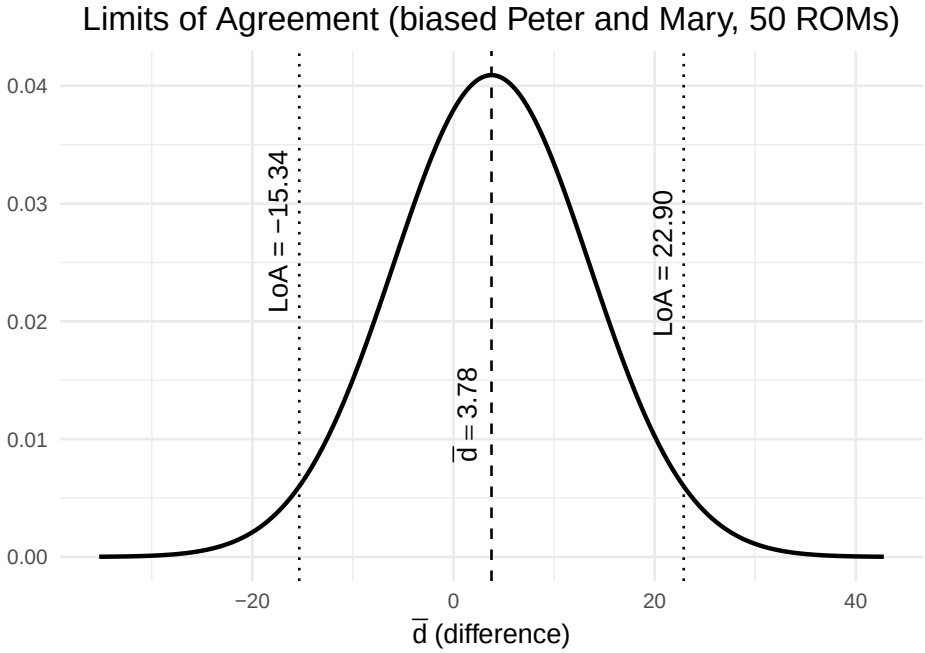
The *Limits of Agreement* (LoA) are derived from data and represent the "usual" values for the difference between the two measurements. Basically, we only need information from the paired t-test. The 1.96 in the formula is (as always) a hint that we assume the difference is normally distributed. This is not always the case.

The formula for the LoA is (due to the connection between the SEM and the SD of the differences above):

$$LoA = \bar{d} \pm 1.96 \cdot SD_{difference} = \bar{d} \pm 1.96 \cdot \sqrt{2} \cdot SEM_{consistency}$$

Let us not get confused by the name. Just because a data point is shown within the LoA, it does not mean that it is a good measurement. Maybe ± 20 degrees (as visible in the plot) is insufficient agreement for practical purposes. The LoA is a statistical measure, not a clinical one.

To remember the fact that a normal distribution is assumed for the differences $\bar{d}$, we draw the limits of agreement for our data:



$\bar{d} = 3.87$ (before the bias introduction, Mary was on average 1.2 lower) There should be approximately 95% of the differences between the two measurements within the limits of agreement.

See also exercise 6.

2.1.9 Example of a Reliability Study

Let's try to verify some of the ICCs calculated in this paper.

The title of the paper is "Between-day reliability of local and global muscle-tendon unit assessments in female athletes whilst controlling for menstrual cycle phase".

It takes a while to find a study that provides the data. Kindly, the authors provide raw data here.

Some comments after a quick glance at the paper:

- One thing you should avoid is to write results in the methods section as is done in lines 152-154 of the pdf. First we describe what we did, then we show the results.
- The next thing to avoid is testing for normality. For seventeen participants, the power of the test might be low - see Figure 1a/b here.
- Apparently, there was no power calculation for the ICCs or the paired t-test. As a consequence, results (for instance for the t-test) are to be

interpreted exploratively.

- Multiple testing is not considered. From the first 28 p-values in Table 1 and 2 of the paper, two are "significant" ($p = 0.049$, both marked with a star of course), which is well within the expected number of false positives. Remember, under H_0 every 20th test is "significant". Fortunately, the authors do not overinterpret the results and do not mention "significance" in the results or discussion.

Look at Table 1 in the paper and consider the variable *Plantar flexion MVC (N.m)*. We want to replicate the results in the first line (except for p-values and effect size d).

```
library(pacman)
conflicts_prefer(rethinking::logit)
```

```
## [conflicted] Removing existing preference.
## [conflicted] Will prefer rethinking::logit over any other package.
```

```
p_load(
  tidyverse,
  readxl,
  psych)

# Read file from github
tmp <- tempfile(fileext = ".xlsx")
download.file("https://raw.githubusercontent.com/jdegenfellner/Script_QM3_ZHAW/main/data/Chapter_
              destfile = tmp, mode = "wb")
df <- suppressWarnings(suppressMessages(read_xlsx(tmp, sheet = "Sheet1", skip = 1)))
head(df)
```

```
## # A tibble: 6 x 113
##   Participant `Displacement (mm)`  ...3 `Passive displacement (mm)`  ...5
##         <dbl>               <dbl> <dbl>                        <dbl> <dbl>
## 1           1               13.9  14.2                          4.46  4.65
## 2           2               20.5  14.8                          5.32  4.58
## 3           3                8.81  8.23                         2.08  2.31
## 4           4               12.8  10.2                          5.17  5.11
## 5           5               22.4  21.5                          6.97  6.67
## 6           6               19.0  18.7                          5.12  6.19
## # i 108 more variables: `Corr displacement (mm)` <dbl>, ...7 <dbl>,
## #   `Strain (%)` <dbl>, ...9 <dbl>, `Plantar flexion moment (N.m)` <dbl>,
## #   ...11 <dbl>, `AT force (N)` <dbl>, ...13 <dbl>, `AtK all (N/mm)` <dbl>,
## #   ...15 <dbl>, `ATk low (N/mm)` <dbl>, ...17 <dbl>, `ATk high (N/mm)` <dbl>,
## #   ...19 <dbl>, `NATk all (N/strain)` <dbl>, ...21 <dbl>,
## #   `NATk low (N/strain)` <dbl>, ...23 <dbl>, `NATk high (N/strain)` <dbl>,
## #   `Norm ATk 50-100 2` <dbl>, `ATk index (N/strain)` <dbl>, ...27 <dbl>, ...
```

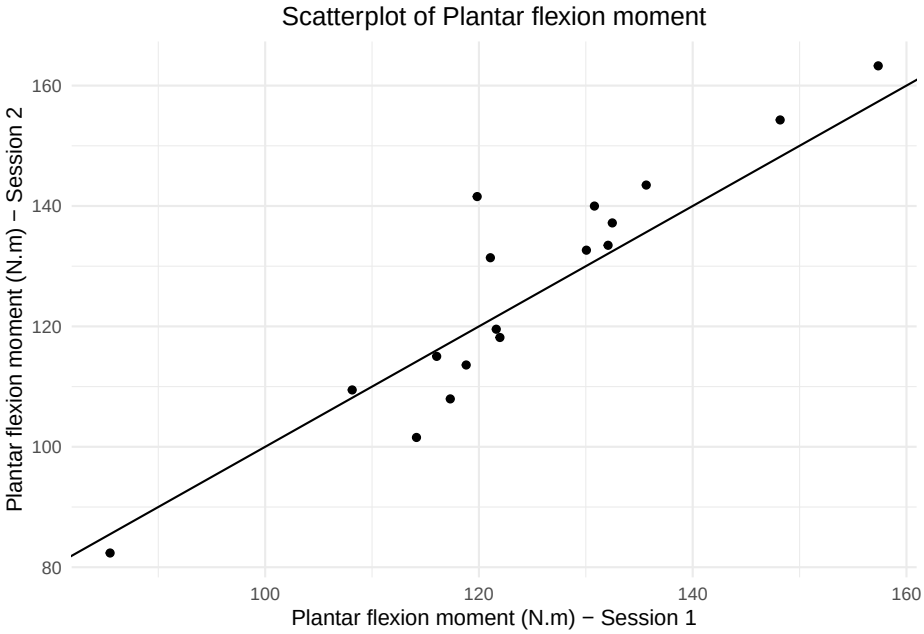
```r
#colnames(df)

df_1 <- df %>% dplyr::select(`Plantar flexion moment (N.m)`, `...11`)
head(df_1)
```

```
## # A tibble: 6 x 2
##   `Plantar flexion moment (N.m)` ...11
##                            <dbl> <dbl>
## 1                           122.  118.
## 2                           120.  142.
## 3                            85.5  82.4
## 4                           114.  102.
## 5                           116.  115.
## 6                           119.  114.
```

```r
#tail(df_1)
dim(df_1)
```

```
## [1] 17  2
```

```r
ggplot(df_1, aes(x = `Plantar flexion moment (N.m)`, y = `...11`)) +
  geom_point() +
  geom_abline(slope = 1, intercept = 0) +
  labs(
    title = "Scatterplot of Plantar flexion moment",
    x = "Plantar flexion moment (N.m) - Session 1",
    y = "Plantar flexion moment (N.m) - Session 2"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```




```
# Looks rather nice
# would be interesting what influence the largest 2 and the smallest observation have

colnames(df_1) <- c("test", "retest")

# Mean Session 1
mean(df_1$test, na.rm = TRUE)
```

```
## [1] 124.18
```

```
# 124.18 check
sd(df_1$test, na.rm = TRUE)
```

```
## [1] 15.91872
```

```
# 15.91872 check

# Mean Session 2
mean(df_1$retest, na.rm = TRUE)
```

```
## [1] 126.1748
```

```
# 126.1748 check
sd(df_1$retest, na.rm = TRUE)
```

```
## [1] 20.41306
```

```
# 20.41306
```

```r
# Change
mean(df_1$retest - df_1$test, na.rm = TRUE)
```

```
## [1] 1.99477
```

```r
# 1.99477 check
sd(df_1$retest - df_1$test, na.rm = TRUE)
```

```
## [1] 8.160476
```

```r
# 8.160476 check

# The p-value column is not needed, as well as the d (effect size)

# create long format for ICC calculation
df_long <- df_1 %>%
  pivot_longer(cols = everything(), names_to = "Timepoint", values_to = "value")
head(df_long)
```

```
## # A tibble: 6 x 2
##   Timepoint value
##   <chr>     <dbl>
## 1 test      122.
## 2 retest    118.
## 3 test      120.
## 4 retest    142.
## 5 test       85.5
## 6 retest     82.4
```

```r
psych::ICC(df_1)
```

```
## Call: psych::ICC(x = df_1)
##
## Intraclass correlation coefficients
##                          type  ICC  F df1 df2       p lower bound upper bound
## Single_raters_absolute   ICC1 0.90 19  16  17 8.4e-08        0.75        0.96
## Single_random_raters     ICC2 0.90 19  16  16 1.7e-07        0.75        0.96
## Single_fixed_raters      ICC3 0.90 19  16  16 1.7e-07        0.75        0.96
## Average_raters_absolute ICC1k 0.95 19  16  17 8.4e-08        0.86        0.98
## Average_random_raters   ICC2k 0.95 19  16  16 1.7e-07        0.86        0.98
## Average_fixed_raters    ICC3k 0.95 19  16  16 1.7e-07        0.86        0.98
##
##  Number of subjects = 17     Number of Judges =  2
## See the help file for a discussion of the other 4 McGraw and Wong estimates,
```

```r
# In Table 1, we find the values for ICC1k: 0.95 (95% CI: 0.86-0.98)

# Standard error of measurement:
```

```r
# SEM = SD_difference/sqrt(2)
sd(df_1$retest - df_1$test, na.rm = TRUE)/sqrt(2)
```

```
## [1] 5.770328
```

```r
# 5.770328 vs. 4.2 in the paper

# In "Measurement in Medicine", p. 112, the authors warn against
# the formula the paper used to calculate the SEM:
# SEM = SD_pooled*sqrt(1-ICC)
SD_pooled <- sqrt((sd(df_1$test, na.rm = TRUE)^2 + sd(df_1$retest, na.rm = TRUE)^2)/2)
SEM <- SD_pooled*sqrt(1 - 0.9477155)
SEM
```

```
## [1] 4.185437
```

```r
# =4.185437 vs. 4.2 in the paper

# Formula 5.7 in the book:
# sigma_y is the total variance
# lets get the variance components with lmer:
dim(df_long)
```

```
## [1] 34  2
```

```r
df_long$ID<- rep(1:17, each=2)
mod <- lmer(value ~ Timepoint + (1|ID), data = df_long)
summary(mod)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: value ~ Timepoint + (1 | ID)
##    Data: df_long
##
## REML criterion at convergence: 255.9
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.66018 -0.39028  0.00975  0.52172  1.76028
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  ID       (Intercept) 301.8    17.37
##  Residual              33.3     5.77
## Number of obs: 34, groups:  ID, 17
##
## Fixed effects:
##               Estimate Std. Error t value
## (Intercept)    126.175      4.439  28.421
```

```
## Timepointtest   -1.995      1.979  -1.008
##
## Correlation of Fixed Effects:
##             (Intr)
## Timeponttst -0.223
```

```
# check ICC1k:
301.8/(301.8 + 33.3/2)
```

```
## [1] 0.9477155
```

```
# 0.9477155 check

# Try with Timepoint as random effect (could be rater)
mod2 <- lmer(value ~ (1|ID) + (1|Timepoint), data = df_long)
summary(mod2)
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: value ~ (1 | ID) + (1 | Timepoint)
##    Data: df_long
##
## REML criterion at convergence: 260.1
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -1.83032 -0.36357 -0.07451  0.46315  1.93042
##
## Random effects:
##  Groups    Name        Variance  Std.Dev.
##  ID        (Intercept) 301.74846 17.371
##  Timepoint (Intercept)   0.03097  0.176
##  Residual               33.29703  5.770
## Number of obs: 34, groups:  ID, 17; Timepoint, 2
##
## Fixed effects:
##             Estimate Std. Error t value
## (Intercept)   125.18       4.33   28.91
```

```
# -> sigma_y = SD_pooled =
sqrt(301.8 + 33.3) # = 18.30574
```

```
## [1] 18.30574
```

```
# SEM =
sqrt(301.8 + 33.3)*sqrt(1 - 0.9477155) # 4.18
```

```
## [1] 4.185754
```

```r
# or this (if average measurements are meant, ICC1k)?
sqrt(301.8 + 33.3/2) # 17.84517
```

```
## [1] 17.84517
```

```r
# SEM =
sqrt(301.8 + 33.3/2)*sqrt(1 - 0.9477155) # 4.08 (4.18 if ICC1)
```

```
## [1] 4.080441
```

```r
# MCD, Minimally Detectable Difference:
# (also known as Smallest Detectable Change, SDC)
# MCD =
1.96 * 4.185437 * sqrt(2)
```

```
## [1] 11.60144
```

```r
# 11.6 check.

# Coefficient of Variation:
# The CV was calculated for each participant as (within-subject SD / mean) * 100,
# with the mean of values from all participants used as the test-
# retest CV (e.g., [24]).

df_1$mean_i <- rowMeans(df_1, na.rm = TRUE) # mean of test and retest for each person.
df_1$sd_within_i <- abs(df_1$test - df_1$retest) / sqrt(2)
df_1$cv_i <- (df_1$sd_within_i / df_1$mean_i) * 100

# Test-Retest CV
mean(df_1$cv_i, na.rm = TRUE)
```

```
## [1] 3.605028
```

```r
sd(df_1$cv_i, na.rm = TRUE)
```

```
## [1] 2.964592
```

```r
# 95% CI:
mean(df_1$cv_i, na.rm = TRUE) + c(-1,1)*1.96*sd(df_1$cv_i, na.rm = TRUE)/sqrt(17)
```

```
## [1] 2.195750 5.014306
```

```r
# 2.195750 5.014306 # check
```

2.2 Exercises

Solutions to the exercises can be found here.

2.2.1 [E] Exercise 1

Use the 50 ROM measurements from Peter and Mary (see above).

- Introduce biases of 5, 15 and 35 degrees in the data set. Mary should have systematically higher values than Peter.
- Show in R that the correlation does not change.

2.2.2 [M] Exercise 2

Consider the 50 ROM measurements from Peter and Mary (see above) on the non affected side (nas).

- Regress Mary's measurements on Peter's measurements using the simple linear regression model in R.
- How precisely can we predict Mary's measurements from Peter's, i.e. analyse the residuals.
- Calculate the mean absolute deviation MAE, i.e.

$$MAE = \frac{1}{n}\sum_{i=1}^{n} |ROM_i - \widehat{ROM}_i|$$

  where ROM_i is the true value and $\widehat{ROM}_i$ is the model-predicted value.

2.2.3 [E] Exercise 3

- Draw the model hierarchy for the first model above (ICC1).

2.2.4 [M] Exercise 4

Go back to the section about the ICC and the example of the HADS-A.

- Find your own score, maybe one that is used in your field.
- Find the minimally clinically important difference (MCID) for this score.
- Execute the code above with your score and the MCID and see if the results are similar to the HADS-A example.

2.2.5 [M] Exercise 5

Consider the Bayesian model for the ICC2 and ICC3 above.

- Draw the model hierarchy for this model.

2.2.6 [H] Exercise 6

Read the following sections in the book *Measurement in Medicine*:

- 5.6.2.2 Bland-Altman method
- 8.5.3 Smallest detectable change (a change beyond the measurement error)

- 8.5.4.1 The concept of minimal important change
- Be forgiving with the dichotomous writing style.

2.2.7 [M] Exercise 7

Go back to the 5 degree biased *ROMs* from Peter and Mary.

- Fit the Frequentist models for ICC1 and ICC2/3 again using `lmer`.
- Perform model diagnostics (`check_model`) for both models.
- Also compare residuals and posterior predictive checks (PPC). You can plot the diagnostics separately, for example: `check_model(mod1, check = "pp_check")`.
- What do you think about the model fit for the two models?
- Which model do you prefer?

2.2.8 [H] Exercise 8

Introduce some outliers in the *ROMs* of Peter and Mary (50 participants).

- How do the outliers influence the ICC?
- How do the outliers influence the model fit of the underlying statistical model?

2.2.9 [M] Exercise 9

Consider the example reliability study above.

- According to this paper, did the authors choose the right ICC (ICC1k)? What can the chosen ICC be used for?
- Verify the ICC for the "Raw elongation (mm)" in Table 1 of the paper.

2.2.10 [H] Exercise 10

We want to see where formula 5.7 for the SEM in the book comes from.

$$SEM = SD_{pooled} \cdot \sqrt{1 - ICC}$$

- We start with the formula for the (overall) ICC:

$$ICC = \frac{\sigma_\eta^2}{\sigma_\eta^2 + \sigma_\varepsilon^2}$$

- $SEM = \sqrt{\sigma_\varepsilon^2}$
- Show that the formula for the SEM is equivalent to the one in the book by rewriting the ICC formula.

2.3 eLearning

In this eLearning section, please study (the whole) chapter 6 on validity in the book (pages 150-196). Here are some thoughts on the chapter:

- p. 153: "Existing knowledge about the construct should drive the hypotheses." This is an important point. It seems a good idea to apply our Bayesian knowledge here. Often simply correlation tests or t-tests are used in this area. When doing so, it would be advantageous to define priors and use the Bayesian analysis we have covered previously.
- p. 154: "They emphasize that the crucial ingredient of validity involves the causal role of the construct in determining what value the measurement outcomes will take…" In my opinion, all these analyses should be done using causal inference. Richard McElreath's book contains a lot of information on this. But granted: We are at the very beginning on this topic, in the lecture and in wide areas of health-research.
- You should be familiar with the different definitions of validity: face, content, concurrent, predictive, structural, convergent, discriminant, discriminative validity.
- p. 181 at the top: As you can imagine, this is a very important piece of information. We are not here to play "significance"-bingo with "hypotheses". Ideally, we are searching for the truth and want to answer real questions (which is unfortunately not always attainable).
- Figure 6.8. on p. 195. is worth remembering.

Additionally, look for at least two reliability and validity articles in your field (either Reliability alone or Reliability and Validity together) and try to understand what they did. Did the authors provide code and data to aid reproducibility?

Please also go through the last part of the reliability chapter in our skript.

Chapter 3

Further Regression Methods

3.1 Linear Mixed Models (LMMs)

In Richard McElreath's book "Statistical Rethinking", chapters 13 and 14 deal with multilevel models. In Gelman's Book, see " Part 2A: Multilevel Models" and the following.

One very graspable introduction to LMMs is this video.

We have already come across a Linear Mixed Model (LMM) in the first chapter, where we used the `rethinking` and `lme4` packages to fit a model with random intercepts to determine our Intraclass Correlation Coefficients (ICCs). They are called *random* intercept since they are drawn from a common normal distribution, of which the parameters (mean and standard deviation) are estimated from the data.

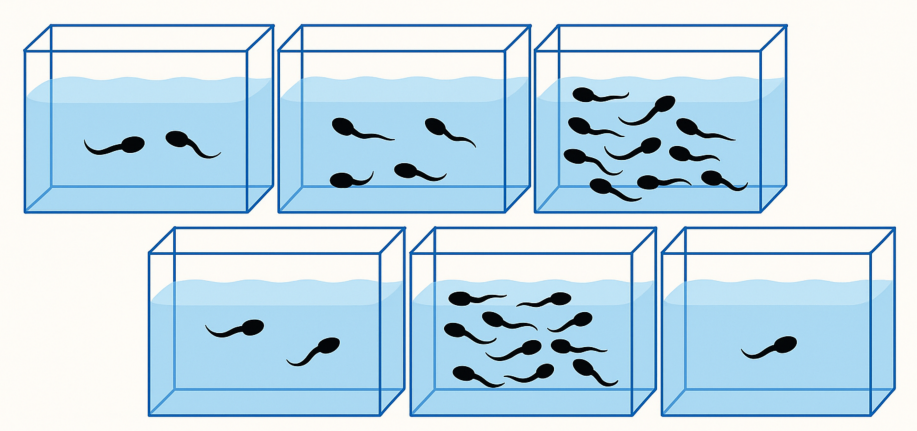
The **main reason** for using LMMs is that they allow us to model data with hierarchical structure, such as repeated measures (of the same subject) or nested data (math scores in different schools within different school districts).

- For example, in our Peter and Mary data set where they measured ROMs, we had two measurements per subject. The two measurements are *clustered* within the subject. They are not independent. If Mary measures a very high *ROM*, chances are that Peter as well measures a high value. Hence, the classical linear regression model (which assumes the rows/observations in our data set are independent) is not appropriate and would lead to smaller standard errors.
- Another example is our *RESOLVE* trial. We are conducting a cluster randomized trial (CRT) with 20 clusters (physiotherapy offices) and over

200 patients. Measurements within a cluster could be correlated (more similar compared to other clusters). One could also think of the physiotherapists performing the treatments as clusters, since - potentially - the outcomes for all their patients could be correlated due to a more similar treatment from the same therapists compared to others. Here the outcome measures are nested within the physiotherapists which are nested within the physiotherapy offices (assuming they only work for one office).

3.1.1 Example: Reed Frog tadpole mortality - two levels

Let's start with Example 13.1 from McElreath's book (p.401) (Tadpoles in tanks, image by GPT4o).



```
library(rethinking)
library(lme4)
data(reedfrogs)
d <- reedfrogs
str(d)
```

```
## 'data.frame':    48 obs. of  5 variables:
##  $ density : int  10 10 10 10 10 10 10 10 10 10 ...
##  $ pred    : Factor w/ 2 levels "no","pred": 1 1 1 1 1 1 1 2 2 ...
##  $ size    : Factor w/ 2 levels "big","small": 1 1 1 1 2 2 2 2 1 1 ...
##  $ surv    : int  9 10 7 10 9 9 10 9 4 9 ...
##  $ propsurv: num  0.9 1 0.7 1 0.9 0.9 1 0.9 0.4 0.9 ...
```

Variables are explained here (see also p.401 in the book). The original paper can be found here.

The number of surviving tadpoles *surv* should be modeled. Now, each row is a different "tank" (48 in total), an experimental environment that contains tadpoles. Obviously, the tanks are one option for *clustering* the data, assuming the tadpoles within the same tank live in a more similar environment than those

in different tanks (or get a more similar "treatment"). We could rewrite the data set to have one row per individual tadpole (instead of number of survived a binary variable) and the cluster variable called *tank* (exercise 1). We would then model the survival probability of tadpoles (1 could mean *survived* and 0 could mean *died*). Formally, this would be a so-called logistic regression (see below), since our outcome is binary (*survived*/*died*).

When estimating the number of surviving tadpoles we could consider the following options:

- **Complete pooling** (exercise 6): Estimate the overall survival probability (ignoring the tank effect). In this case, one might miss between-variance from the different tanks. A single intercept is used for all tanks.
- **No pooling**: Estimate the survival probability for each tank (ignoring the overall effect completely). Here, the tanks do not "know" anything from one another. Information of one tank is not used for the other tanks. An intercept is estimated for each tank (48 in total). This approach would be equivalent to using indicator variables for each tank.
- **Partial pooling**: Estimate the survival probability for each tank, but use information from the other tanks.

3.1.1.1 Model 13.1. No pooling using regularizing priors

Let's start with the second model (intercept for each tank) with a regularizing prior. We want to model the number of surviving tadpoles in the tanks by using a binomial distribution with different survival probabilities for each tank.

Here is our model in the typical notation:

$$\begin{aligned} S_i &\sim Binomial(N_i, p_i) \\ logit(p) &= \alpha_{TANK}[i] \\ \alpha_j &\sim \text{Normal}(0, 1.5) \text{ for } j = 1, ..., 48 \end{aligned}$$

This is a **varying intercept model** (the simplest kind of **varying effects** model) with 48 intercepts (one for each tank).

The number of surviving tadpoles S_i in tank i is modeled as a binomial distribution with N_i trials (the number of tadpoles in the tank) and p_i the survival probability of tank i. The **regularizing priors** for α_j is a normal distribution with mean 0 and standard deviation 1.5.

Technically, **we could use different priors for each intercept**, but this would be a bit cumbersome and we do not have useful prior information about the survival probabilities of the tanks.

Notice that we use a `logit` link function to transform the probability p (range $(0, 1)$) to the range $(-\infty, \infty)$. This way, we can use a normal distribution for the prior of the intercepts. The inverse of the logit function is also available in `rethinking`: `inv_logit`. `rethinking::inv_logit(0)`gives the probability of

0.5. So, we assume a coin flip for the survival probability of the tadpoles in the tanks *a priori* (maybe not the worst starting point if you have no clue).

What survival probabilities do we assume a priori? Well, according to the normal distribution with a standard deviation of 1.5 we assume that the 95% of the logit-transformed survival probabilities are between −3 and 3. This means that the survival probabilities in the tanks are in:

```
c(rethinking::inv_logit(-3), rethinking::inv_logit(3))
```

```
## [1] 0.04742587 0.95257413
```

This assumption seems rather reasonable and survival probabilities are not constrained very much.

Next we define the data set for the model and use the `ulam` function from the `rethinking` package to fit the model.

```
library(rethinking)

d$tank <- 1:nrow(d)
dat <- list(S = d$surv,
            N = d$density,
            tank = d$tank)

# approximate posterior
set.seed(122)
m13.1 <- ulam(
  alist(
    S ~ dbinom(N, p),
    logit(p) <- a[tank],
    a[tank] ~ dnorm(0, 1.5)
  ) , data = dat,
  chains = 4,
  log_lik = TRUE,
  cores = detectCores() - 1)
```

```
## Running MCMC with 4 chains, at most 17 in parallel, with 1 thread(s) per chain...
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 1 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%]  (Sampling)
```

```
## Chain 1 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 2 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 2 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 2 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 2 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 2 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 3 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 3 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 3 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 3 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 4 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 4 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 4 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 4 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 1 finished in 0.0 seconds.
## Chain 2 finished in 0.0 seconds.
## Chain 3 finished in 0.0 seconds.
## Chain 4 finished in 0.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 0.0 seconds.
## Total execution time: 0.2 seconds.
```

```
precis(m13.1, depth = 2)
```

```
##              mean        sd        5.5%       94.5%      rhat ess_bulk
## a[1]   1.717153975 0.7747045  0.54616728  3.00283859 1.0000688 3842.727
## a[2]   2.428617888 0.9211741  1.06824313  4.00015835 1.0012690 3260.224
## a[3]   0.752957627 0.6606081 -0.30096504  1.87725178 0.9994512 3657.977
## a[4]   2.392671638 0.9029472  1.04231930  3.96444551 0.9999353 4901.400
## a[5]   1.698600976 0.7501296  0.56599772  2.95760508 1.0010745 3583.125
## a[6]   1.709016613 0.7333373  0.61628582  2.93631517 1.0043720 3791.590
## a[7]   2.394662704 0.8967972  1.05357313  3.91037707 1.0021785 4633.357
## a[8]   1.751520526 0.8358777  0.51949728  3.11502328 1.0010309 2908.902
## a[9]  -0.368916618 0.6263819 -1.40362380  0.63108764 1.0014161 5011.822
## a[10]  1.721990000 0.7876527  0.53971140  3.01818068 1.0063827 4744.005
## a[11]  0.762625121 0.6504813 -0.23379569  1.84594630 1.0042894 4356.296
## a[12]  0.360839675 0.6212530 -0.61653270  1.35696971 1.0003395 5427.298
## a[13]  0.747187656 0.6556836 -0.26629907  1.86578240 1.0047650 4966.021
## a[14]  0.004006217 0.6030814 -0.96595993  0.95626659 1.0015456 3860.227
## a[15]  1.698071534 0.7557423  0.58532646  3.00604710 1.0044773 4061.036
## a[16]  1.718281135 0.7589570  0.55788266  3.01417203 0.9995497 4085.620
## a[17]  2.540229693 0.6715384  1.51315848  3.67929451 1.0045259 3551.220
## a[18]  2.143608277 0.6099572  1.24664556  3.19176580 1.0028046 4601.040
## a[19]  1.814013405 0.5661174  0.96992486  2.74369485 1.0008039 3875.393
## a[20]  3.123650807 0.8415721  1.90725179  4.55286918 1.0001091 4325.045
## a[21]  2.144628007 0.6206064  1.21321605  3.18874980 1.0003844 4533.801
## a[22]  2.147574884 0.5862452  1.27518891  3.13301000 1.0054702 3115.246
## a[23]  2.158113153 0.6232680  1.23542513  3.20262615 1.0031874 4471.126
## a[24]  1.559385114 0.5054760  0.81471920  2.39449805 1.0050480 4677.085
## a[25] -1.100464064 0.4645629 -1.90970290 -0.37079036 1.0031156 4404.790
## a[26]  0.066658204 0.3997793 -0.57122349  0.71319611 1.0064177 5082.203
## a[27] -1.543870103 0.4969425 -2.35267385 -0.80927298 1.0049325 4888.664
## a[28] -0.545686643 0.3798291 -1.16412868  0.05957410 1.0141620 4539.888
## a[29]  0.076433014 0.3891807 -0.54687778  0.68588621 1.0063604 4372.065
## a[30]  1.317149716 0.4699256  0.61627878  2.10864700 1.0045567 4237.176
## a[31] -0.730232347 0.4142389 -1.41214843 -0.08699831 1.0056735 4674.305
## a[32] -0.380477608 0.3973004 -1.02837789  0.22967002 1.0027172 4758.006
## a[33]  2.854476833 0.6688006  1.86232134  3.93924112 1.0028727 4265.417
## a[34]  2.465178425 0.6016136  1.57307138  3.49223614 1.0034671 4026.520
## a[35]  2.475988179 0.6001981  1.62509496  3.45605046 1.0022587 3520.551
## a[36]  1.906146591 0.4793998  1.17540914  2.69469001 1.0091413 3637.648
## a[37]  1.903268754 0.4848702  1.17045544  2.68925921 0.9999391 4580.711
## a[38]  3.377291158 0.7728682  2.21479767  4.70722943 0.9999686 3559.737
## a[39]  2.448119378 0.5794139  1.58287295  3.42905977 1.0005222 4544.301
## a[40]  2.154826465 0.5110004  1.38358159  3.05124069 1.0057479 3928.628
## a[41] -1.924960160 0.4805040 -2.72391423 -1.20681050 1.0003665 3217.068
## a[42] -0.639489108 0.3594361 -1.20374092 -0.07755660 1.0036102 3891.728
```

```
## a[43] -0.515434649 0.3438150 -1.06683705  0.01709134 1.0067015 3206.302
## a[44] -0.399514136 0.3554107 -0.97334912  0.16487673 1.0043838 5212.322
## a[45]  0.519393682 0.3592609 -0.05772031  1.08264172 1.0022088 4807.866
## a[46] -0.636130206 0.3468987 -1.20742368 -0.10853385 1.0071611 3963.296
## a[47]  1.919195804 0.4916629  1.16999315  2.74101474 1.0000976 3428.402
## a[48] -0.061061422 0.3286406 -0.57313051  0.45005766 1.0009604 3435.665
```

```
# expected survival in tank 1
rethinking::logistic(1.717154006) # same as inv_logit(1.717154006)
```

```
## [1] 0.8477619
```

The R-output from `precis(m13.1, depth = 2)` shows the posterior means and standard deviations of our 48 intercepts (48 tanks with tadpoles). These intercepts are on the scale $(-\infty, \infty)$.

Note that you can always switch from the range $(-\infty, \infty)$ to a probability using the `logistic` function from the `rethinking` package - these functions are called *squashing functions* for a reason. The `inv_logit`function does the same.

$$\text{logistic}(x) = \text{inv\_logit}(x) = \frac{1}{1 + e^{-x}}$$

The other way around works as well, using the `logit` function.

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right)$$

Let's quickly see if these are indeed inverse functions of each other -> exercise 5.

3.1.1.2 Model 13.2: Partial pooling using adaptive pooling

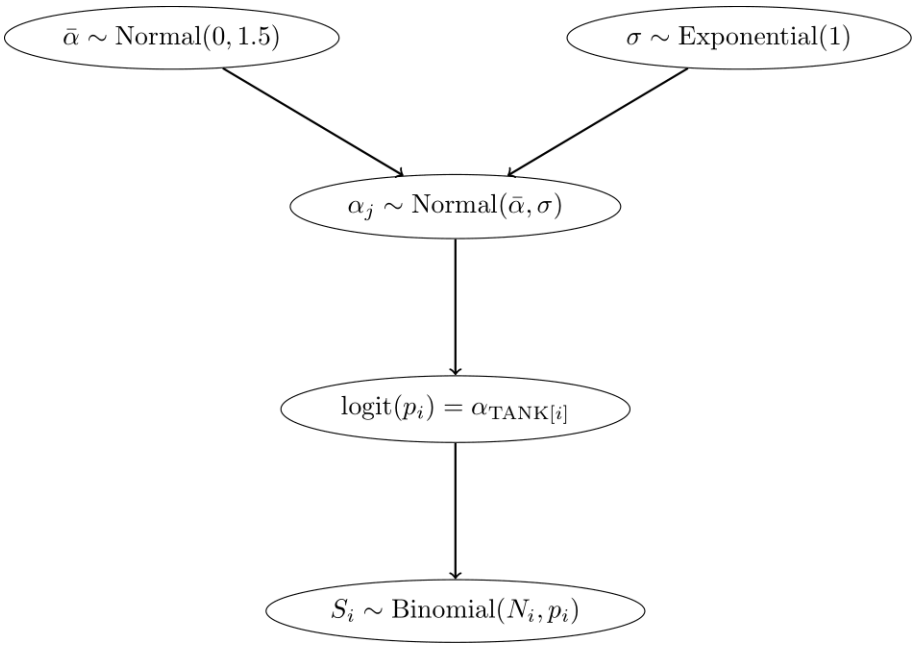
Next, we use **adaptive pooling**. For this, we use priors for the `a[tank]` parameters, but now they *know* from each other via the prior for $\bar{\alpha}$. This model learns from the data how much the intercepts should be pulled towards a common mean. Now we have an additional level in the model hierarchy:

- In the **top level**, the outcome is S, the parameters are the intercepts $\alpha_{TANK[i]}$, and the priors are $\alpha_j \sim N(\bar{\alpha}, \sigma)$.
- In the **second level**, the "outcome" are the intercepts $\alpha_{TANK[i]}$, the parameters are $\bar{\alpha}$ and σ, and the priors are $\bar{\alpha} \sim N(0, 1.5)$ and $\sigma \sim$ Exponential(1).

$$\begin{aligned}
S_i &\sim \text{Binomial}(N_i, p_i) \\
\text{logit}(p) &= \alpha_{\text{TANK}[i]} \\
\alpha_j &\sim \text{Normal}(\bar{\alpha}, \sigma) \\
\bar{\alpha} &\sim \text{Normal}(0, 1.5) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}$$

Since the parameters in the prior of the α_j have also priors (last two lines are new), we have a *multilevel* model. In this case, we have *two* levels and the parameters $\bar{\alpha}$ and σ are called **hyperparameters**. The priors of the hyperparameters are called **hyperpriors**. The **amount of regularization is controlled by the hyperparameter** σ and is learned from the data. σ controls, how much the intercepts are allowed to deviate from the mean $\bar{\alpha}$. The smaller σ, the more the intercepts are pulled towards the mean $\bar{\alpha}$.

Below, we see the model hierarchy:



We now fit the model using the `ulam` function from the `rethinking` package.

```
library(conflicted)
conflicts_prefer(posterior::var)
```

```
## [conflicted] Removing existing preference.
## [conflicted] Will prefer posterior::var over any other package.
```

```
conflicts_prefer(posterior::sd)
```

```
## [conflicted] Removing existing preference.
## [conflicted] Will prefer posterior::sd over any other package.
```

```
m13.2 <- ulam(
  alist(
    S ~ dbinom(N, p),
    logit(p) <- a[tank],
    a[tank] ~ dnorm(a_bar, sigma),
```

```
    a_bar ~ dnorm(0, 1.5),
    sigma ~ dexp(1)
  ) , data = dat,
  chains = 4,
  log_lik = TRUE,
  cores = detectCores() - 1)
```

```
## Running MCMC with 4 chains, at most 17 in parallel, with 1 thread(s) per chain...
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 1 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 1 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 2 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 2 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 2 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 2 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 2 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 3 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 3 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 3 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 3 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1000 / 1000 [100%]  (Sampling)
```

```
## Chain 4 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 4 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 4 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 4 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1000 / 1000 [100%]  (Sampling)

## Chain 4 Informational Message: The current Metropolis proposal is about to be reject
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/va
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable
## Chain 4 but if this warning occurs often then your model may be either severely ill-
## Chain 4

## Chain 1 finished in 0.1 seconds.
## Chain 2 finished in 0.1 seconds.
## Chain 3 finished in 0.1 seconds.
## Chain 4 finished in 0.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 0.1 seconds.
## Total execution time: 0.2 seconds.
```

```
precis(m13.2, depth = 2)
```

```
##                mean        sd        5.5%        94.5%      rhat ess_bulk
## a[1]   2.1360104697 0.8661191  0.83424164  3.553680582 1.0026814 3270.340
## a[2]   3.0781334686 1.1236850  1.43095858  4.984342272 1.0021510 3333.746
## a[3]   1.0172167106 0.6728835  0.02487395  2.122555208 1.0017997 4433.028
## a[4]   3.0539085226 1.0673011  1.45578363  4.862346429 0.9991935 3292.310
## a[5]   2.1460703170 0.8885053  0.82128817  3.632872454 1.0012996 3243.157
## a[6]   2.1295682428 0.8491092  0.88580153  3.547450946 1.0068877 4014.243
## a[7]   3.0995747821 1.1423049  1.49656577  5.081117313 1.0000209 2876.975
## a[8]   2.1399446617 0.9003458  0.81845142  3.682065470 1.0014538 3989.661
## a[9]  -0.1714730282 0.6191399 -1.18887021  0.817493946 1.0011164 3485.167
## a[10]  2.0975423153 0.8315648  0.86475229  3.491156260 1.0003350 3475.711
## a[11]  0.9972845036 0.6886669 -0.06780394  2.108903514 1.0035634 6346.557
## a[12]  0.5808274305 0.6321308 -0.39005959  1.589964778 1.0073558 4009.216
## a[13]  1.0044483192 0.6869233 -0.06659323  2.145824589 1.0062292 5228.173
## a[14]  0.1943572342 0.5958137 -0.75088851  1.137034844 1.0090253 4741.338
## a[15]  2.1509187929 0.8679933  0.90383876  3.653383320 1.0031944 3312.846
## a[16]  2.1333641559 0.9017145  0.80812869  3.653571472 1.0126095 4233.198
```

```
## a[17]  2.9191999792 0.8415070  1.68621504  4.328418222 1.0040760 4410.054
## a[18]  2.3925665004 0.6551940  1.42166745  3.513076801 1.0056082 4144.746
## a[19]  2.0077513832 0.5900887  1.11100223  2.993480106 1.0059276 3633.635
## a[20]  3.6725677386 1.0278681  2.20374894  5.475838743 1.0049163 3236.714
## a[21]  2.4059350086 0.6759614  1.38493795  3.559096737 1.0015511 5132.665
## a[22]  2.4065306331 0.6796674  1.42094355  3.541695739 0.9991285 3831.269
## a[23]  2.3861532839 0.6514719  1.42360773  3.472100947 1.0043733 4128.566
## a[24]  1.7089186402 0.5393019  0.87925445  2.627253975 1.0048576 4608.675
## a[25] -1.0120972586 0.4472418 -1.73198555 -0.326257303 1.0014662 4734.588
## a[26]  0.1608543417 0.3886832 -0.46547133  0.767354669 1.0023246 4941.013
## a[27] -1.4196695674 0.4932724 -2.25709742 -0.685836079 1.0028377 4123.701
## a[28] -0.4766473920 0.4060360 -1.14774321  0.151371055 1.0052218 3931.236
## a[29]  0.1694769915 0.4170077 -0.49255201  0.859140044 1.0020684 4739.883
## a[30]  1.4511140953 0.5133739  0.66183760  2.306308979 1.0011930 4978.596
## a[31] -0.6385969786 0.4191422 -1.30238164  0.002327191 1.0021542 4600.045
## a[32] -0.3101703316 0.3728743 -0.90309533  0.287795210 0.9998436 5387.353
## a[33]  3.1774487790 0.7655333  2.08349167  4.549343447 1.0049743 3861.417
## a[34]  2.7053424493 0.6559521  1.73698818  3.801543954 1.0012111 4418.300
## a[35]  2.7221133351 0.6273887  1.81337524  3.780184912 1.0009025 3645.023
## a[36]  2.0546273161 0.4965714  1.31340060  2.878743362 1.0062186 4941.240
## a[37]  2.0584840343 0.5188969  1.26757847  2.917035421 1.0015893 4475.009
## a[38]  3.8972653158 0.9072140  2.63982365  5.508594228 1.0012629 3004.764
## a[39]  2.6974990445 0.6268089  1.74491693  3.751003579 0.9992067 3801.944
## a[40]  2.3462939886 0.5804959  1.47061310  3.318167013 1.0087558 3756.209
## a[41] -1.8135896953 0.4813845 -2.60774930 -1.085181585 1.0000860 3538.101
## a[42] -0.5797062070 0.3540842 -1.15872889 -0.026373766 1.0025959 3932.554
## a[43] -0.4553320673 0.3423639 -1.00198306  0.080289495 1.0065176 4788.482
## a[44] -0.3294683796 0.3311178 -0.86400279  0.191963207 1.0099390 4098.921
## a[45]  0.5781168386 0.3537311  0.02023155  1.152293592 1.0051868 4534.352
## a[46] -0.5734202323 0.3340745 -1.11167488 -0.052582139 1.0045535 3855.996
## a[47]  2.0729423125 0.5180042  1.30783537  2.935489311 1.0033375 3592.034
## a[48] -0.0003544358 0.3378852 -0.52970801  0.533022175 1.0060511 4423.897
## a_bar  1.3420237008 0.2527708  0.94567190  1.760413394 1.0011815 2994.438
## sigma  1.6149891238 0.2052024  1.30751062  1.961324147 0.9998542 1674.455
```

Now, the output of `precis(m13.2, depth = 2)` additionally shows the posterior means and standard deviations of the hyperparameters $\bar{\alpha}$ (`a_bar`) and σ (`sigma`).

3.1.1.3 Conceptual difference between model 13.1 and 13.2

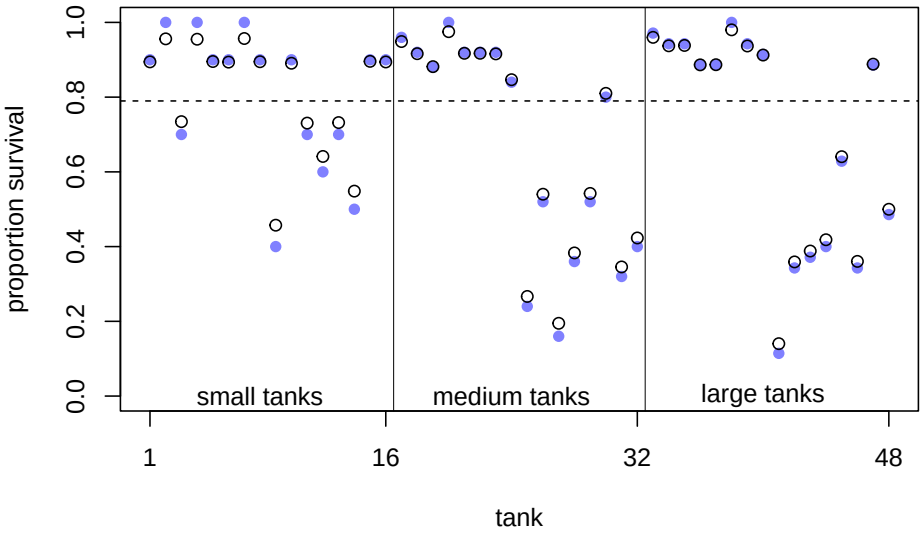
Model 13.1 is not a multilevel model, we just tell the model there are 48 tanks and each tank has its own intercept. For each intercept, we use the same prior distribution ($Normal(0, 1.5)$), but we could use different ones if we had knowledge about the survival probabilities of some of the tanks. Note that the priors themselves are not connected via another level. They do their thing

independently.

- By *decreasing* the variance of these priors, we are dampening the influence of the data on the intercepts which are drawn towards the overall mean.
- *Increasing* the variance of the priors to a value large enough will result in the raw survival probabilities for each tank. This would be the same as if we took indicator variables for each tank. See exercise 2.

Model 13.2 is a **multilevel model**. Now the model *learns from the data* from which overall mean and variance the intercepts are drawn ($Normal(\bar{\alpha}, \sigma)$). The prior for $\bar{\alpha}$ expresses our belief about the mean survival probability, while the prior for σ expresses our belief about the variability of the survival probabilities.

Figure 13.1 from the book depicts the pooling effect of model 13.2. with respect to tadpole tank size.



In exercise 2 we make the difference between the models 13.1 and 2 clearer. Figure 13.1 from the book shows the posterior survival probabilities for each tank from model 13.2 (black circles) and raw survival probabilities (blue filled points). What we can see is that the estimated survival probabilities for the hierarchical model are drawn towards the mean overall survival probability across all tanks (dashed line; $logistic(\bar{\alpha}) \approx 0.79$).

- In large tanks, there is less shrinkage (enough information/observations), while in small tanks the estimates are pulled more towards the mean (information is used from other tanks).
- The more extreme the raw survival probability (with respect to the overall survival probability), the more it is pulled towards the mean.

This is the **pooling** effect of the model. The phenomenon is called **shrinkage**.

See also exercise 3 to improve your understanding of two models.

3.1.1.4 Compare models 13.1 and 13.2 with respect to predictive ability on unseen data

In QM2 we talked about the difference between explanatory vs. predictive models. If you are interested, you can read section 7.4 (p.217) in the book.

In a **prediction context** we ask ourselves: How well do the models *predict* the data (which was measured using residuals $y_i - \hat{y}_i$ on the *training set* in regression)?

There are many nuances but the **main paradigm** (also in classical machine learning) is to *fit* the model on a **training set** and *evaluate* the model on a **test set**. Should you be interested in the (admittedly very technical) details, I can recommend the book *Elements of Statistical Learning* by Hastie, Tibshirani and Friedman (free here).

During **training**, the model tries to learn the underlying structure of the data.

If the model is too complex, the model can *overfit*, which means that the model learns too much of the noise in the data instead of the underlying structure.

If the model is too simple, the model can *underfit* the training data, which means that the model learns the underlying structure of the data too vaguely.

The test set is used to evaluate the model's performance on unseen data. This is the ultimate and brutally honest test of the model's predictive ability.

For instance, if you think back to our body height prediction example in QM2, we could fit the model on 80% of people (training set) and see how precise the model is on the remaining 20% of people (test set). Let's do this in exercise 7.

Remember, in prediction we do not care about interpretability of the model. We could have a blackbox model (e.g. neural networks) and still be happy with it. This performance can often be surprisingly good (like in the case of hand digit recognition reaching error rates as low as 0.09%), or surprisingly bad (when trying to predict some binary outcome using logistic regression in applied health research where the performance is often not far away from the base rate probability).

In order to estimate the predictive ability (on an unseen test set) of a model, there are many criteria available. One of them is the **WAIC**:

```
WAIC(m13.1)
```

```
##       WAIC      lppd  penalty  std_err
## 1 216.2614 -81.72813 26.40257 4.679154
```

```
WAIC(m13.2)
```

```
##       WAIC      lppd  penalty  std_err
```

```
## 1 200.3715 -79.18461 21.00113 7.205089
```

```
compare(m13.1, m13.2, func = WAIC) # change func to other criteria if needed.
```

```
##           WAIC       SE    dWAIC      dSE    pWAIC       weight
## m13.2 200.3715 7.205089  0.00000       NA 21.00113 0.9996456834
## m13.1 216.2614 4.679154 15.88993 3.798405 26.40257 0.0003543166
```

WAIC stands for *Widely Applicable Information Criterion* and is a measure of prediction accuracy on new data (out of sample deviance); see p.219 and p.404 in Statistical Rethinking. As it is pointed out in the book "Using tools like PSIS and WAIC is much easier than understanding them."

We do not go into the details of WAIC, but as a **first dirty rule of thumb**, the model with the lower WAIC is preferred. One should never solely rely on this criterion, especially not if one is interested in causal questions.

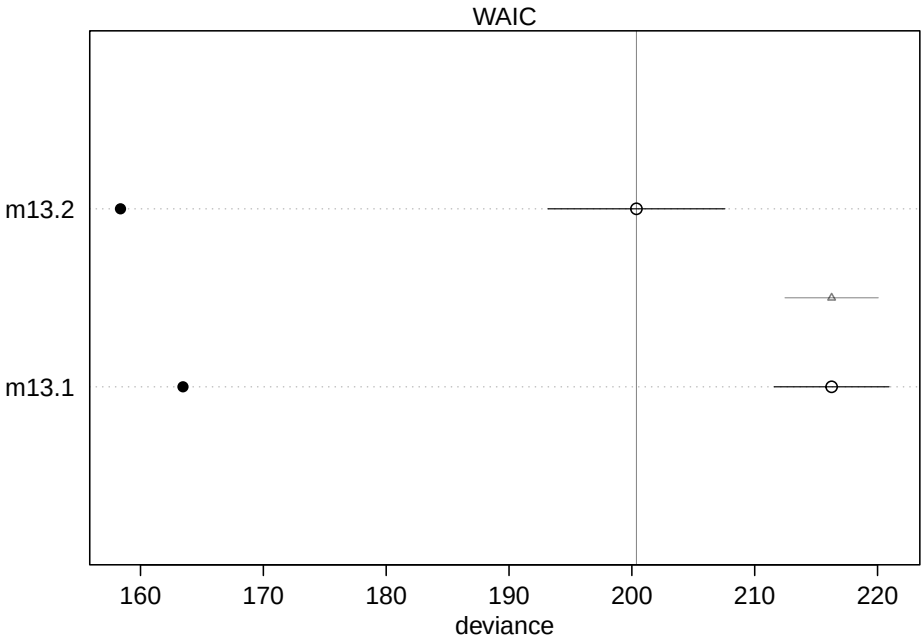
What are the columns?

- `WAIC`: guess of the out-of-sample deviance.
- `SE`: standard error of the WAIC estimate. This is a measure of uncertainty of the WAIC.
- `dWAIC`: difference of each WAIC from the best model (with the *smallest* WAIC).
- `dSE`: standard error of this difference. Here, the standard error of the difference (3.79) is much smaller compared to the difference in WAIC between the models (15.88993). **Hence, one can distinguish between the models using WAIC. → m13.2 is preferred** (with respect to predictive ability on unseen data).
- `pWAIC`: prediction penalty. This is close to the number of parameters in the posterior.
- `weight`: the Akaike weight, a measure of relative support for each model. In our case, it is almost 1 for model 13.2 and almost 0 for model 13.1.

Feel free to read p.227 in the book, specifically before you use WAIC in a publication or thesis.

One can also plot this information for a better overview:

```
plot(compare(m13.1, m13.2, func = WAIC))
```



The filled points show how the model performs on the data it was trained on. Typically, this is lower compared to the out-of-sample deviance (open points) for both models. Section 13.2 (especially Figure 13.3.) in the book shows that partial pooling also yields (on average) better predictions compared to no pooling, it's a tradeoff between over- and underfitting.

3.1.1.4.1 Frequentist version model 13.2 In the Frequentist setting we can use the `lme4` package to fit the analog to model 13.2. To be more specific, we use a generalized linear mixed model (GLMM). *Generalized* because we assume a binomial distribution for the response variable, while usually we have a continuous outcome variable (like body height in cm).

Let's fit the model and explain the outcome first. S is the number of surviving tadpoles, N is the number of tadpoles in the tank.

```
library(lme4)

m_lmer <- glmer(
  cbind(S, N - S) ~ (1 | tank),
  data = dat,
  family = binomial(link = "logit"),
  control = glmerControl(optimizer = "bobyqa")
)
summary(m_lmer)
```

```
## Generalized linear mixed model fit by maximum likelihood (Laplace
```

```
##   Approximation) [glmerMod]
##  Family: binomial  ( logit )
## Formula: cbind(S, N - S) ~ (1 | tank)
##    Data: dat
## Control: glmerControl(optimizer = "bobyqa")
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##     271.6     275.3    -133.8     267.6        46
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -0.5981 -0.2255  0.1267  0.2457  0.9677
##
## Random effects:
##  Groups Name        Variance Std.Dev.
##  tank   (Intercept) 2.459    1.568
## Number of obs: 48, groups:  tank, 48
##
## Fixed effects:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)   1.3786     0.2505   5.503 3.74e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

For convergence reasons (after trial and error), we used the `bobyqa` optimizer. We do not care so much about the technical details behind the model fitting procedure. Briefly, the model is not fitted using the least squares method anymore, but the maximum likelihood method (see also Westfall p.43-52), which is another way of finding model parameters in parametric statistics. Interestingly, the way to fit the model in front of us is outlined in the help file of the command `glmer` (see `?glmer`).

The syntax `(1 | tank)` means that we fit a random intercept for each tank. For details on the random effect structure definition see p.7 here. You will always see such a term if you model clustered data in the Frequentist setting (at least in R).

AIC (Akaike Information Criterion) and BIC (Bayesian Information Criterion) are two measures of model fit which both consider the model fit and the number of parameters simultaneously. Both punish the number of parameters in the model, but not equally strong.

In the summary output, we see the values for AIC, BIC and the log-likelihood. The latter is used to fit the model. `-2logLik` is the so-called *deviance* of the model and has - under H_0 - a χ^2 distribution that can be exploited for hypothesis testing. This term takes the place of the residual sum of squares in least squares regression.

The `Std.Dev.` of the `Random effect tank` is 1.568 which should be somewhat similar to the `sigma` in the Bayesian model (1.614989185), which is the case here. The major difference is that in the Frequentist model, this σ is only learned from the data, while in the Bayesian model we have priors too.

The `Fixed effect (Intercept)` (1.3786) is the `logit` of the mean survival probability across all tanks and analog to the `a_bar` (1.3420237045) in the Bayesian model.

`df.resid`is 46, sample size is 48. Two degrees of freedom are lost due to the estimation of the overall `(Intercept)` and the random effect (the standard deviation σ of `tank`).

And of course, at the bottom of the summary output, for the *stargazers* among us, are the `Signif. codes`.

In summary, both (Frequentist and Bayesian) models deliver rather similar results.

We leave it as an exercise (4) to compare the model-estimated survival probabilities in the tanks for both models.

Section 13.2. in the book shows that partial pooling also yields (on average) better predictions compared to no pooling, it's a tradeoff between over- and underfitting (i.e. the raw probabilities in each tank without knowledge of the other tanks and one overall survival probability that ignores differences between tanks completely).

3.1.2 Example: Mulligan manual therapy - paper

We want to replicate the main result of "Mulligan manual therapy added to exercise improves headache frequency, intensity and disability more than exercise alone in people with cervicogenic headache: a randomised trial".

This randomized clinical trial evaluated the effectiveness of adding Mulligan manual therapy to exercise in patients with cervicogenic headache. The study found that combining Mulligan therapy with exercise led to greater improvements in headache frequency, intensity, and disability compared to exercise alone or a control group.

Some comments:

- They used SPSS for data analysis. Unfortunately, no code was provided or another convenient way replicate the results.
- Data *was* provided as Excel file in the appendix.
- At first glance, I could not find a method for sample size calculation in the referenced literature. Let's research this more as an exercise.
- The primary outcome is *headache frequency* (headache days per month). Ad hoc, such a count variable is not very likely to be normally distributed.

In the methods section they state "For outcomes with normally distributed data, means and standard deviations were calculated". Table 2 in the paper implies that the primary outcome should be normally distributed. Verify this as exercise using the `shapiro.test` function in R. We (only) do this, since they as well used a version of this test. The primary outcome variable has only 6 unique values $(4-9)$ at week 0 and can therefore (strictly speaking) not be normally distributed (normal distribution takes infinitely many values).

- I am not sure what the authors mean exactly with the term "general linear model". Here (p.102) a general linear model is just a multiple linear regression model. My guess is that they did a two-way repeated measures ANOVA.
- There seems to be no checking of the model assumptions in the paper.
- There seems to be no description of handling missing data.

3.1.2.1 Frequentist attempt to replicate the results.

We could try to use this as guideline if we wanted to replicate the two-way repeated measures ANOVA.

Maybe we try to use the `lme4` package first to fit a linear mixed model (LMM), which is the more modern approach.

The model equation should be:

$$\begin{aligned} \text{headache frequency}_{ij} &\sim \text{Normal}(\mu_{ij}, \sigma) \\ \mu_{ij} &= \alpha + \beta_1 \text{group}_i + \beta_2 \text{time}_j + \beta_3 (\text{group}_i \times \text{time}_j) + u_i + \varepsilon_{ij} \end{aligned}$$

- We model the headache frequency of patient i at time point j as normally distributed. It remains to be seen if a normally distributed headache frequency is a good assumption. We will check this later. A priori, I would be doubtful.
- α is the overall intercept (mean headache frequency at week 0 for the reference group)
- $i = 1, \ldots, 99$ are the 99 participants (respectively 91 with complete observations)
- j is the time point (week 0, 4, 13, 26)
- u_i is the random effect for participant i (intercept), which comes from a normal distribution of course
- ε_{ij} is the residual error term, which is also normally distributed. This is a measure of how much the model does not explain.
- The interaction term allows for different intercepts for all combinations of group and time. Note that both `group` and `time` are factors in the model. There are 12 combinations of group and time (3 groups times 4 time points). Be careful with the interpretation of the interaction term.

As soon as an interaction term is present, we cannot directly interpret the main effects anymore.

Let's first bring our data in shape (long format for `lmer`) and (for simplicity) kill all rows with missing values (8 observations). Ideally, we would need to argue how we handle missing data!

```
library(readxl)
library(tidyverse)

url <- "https://raw.githubusercontent.com/jdegenfellner/Script_QM3_ZHAW/main/data/Chapter_Further
temp_file <- tempfile(fileext = ".xls")
download.file(url, destfile = temp_file, mode = "wb")
df <- suppressMessages(
  suppressWarnings(
    readxl::read_xls(temp_file, sheet = 2)
  )
)

df <- df[3:dim(df)[1], 1:6]
head(df)
```

```
## # A tibble: 6 x 6
##   `Subject no.` Group `Headache frequency` ...4  ...5  ...6
##           <dbl> <chr> <chr>                <chr> <chr> <chr>
## 1             1 Ex    5                    4     5     4
## 2             2 Ex    7                    6     6     5
## 3             3 Ex    6                    4     7     6
## 4             4 Ex    7                    5     6     5
## 5             5 Ex    7                    7     5     <NA>
## 6             6 Ex    4                    4     4     4
```

```
dim(df)
```

```
## [1] 99  6
```

```
colnames(df) <- c("ID", "Group", "Week_0",
                  "Week_4", "Week_13", "Week_26")
df
```

```
## # A tibble: 99 x 6
##       ID Group Week_0 Week_4 Week_13 Week_26
##    <dbl> <chr> <chr>  <chr>  <chr>   <chr>
##  1     1 Ex    5      4      5       4
##  2     2 Ex    7      6      6       5
##  3     3 Ex    6      4      7       6
##  4     4 Ex    7      5      6       5
##  5     5 Ex    7      7      5       <NA>
##  6     6 Ex    4      4      4       4
```

```
##  7     7 Ex    6      5      5      6
##  8     8 Ex    7      4      4      4
##  9     9 Ex    6      7      7      5
## 10    10 Ex    6      7      6      5
## # i 89 more rows
```

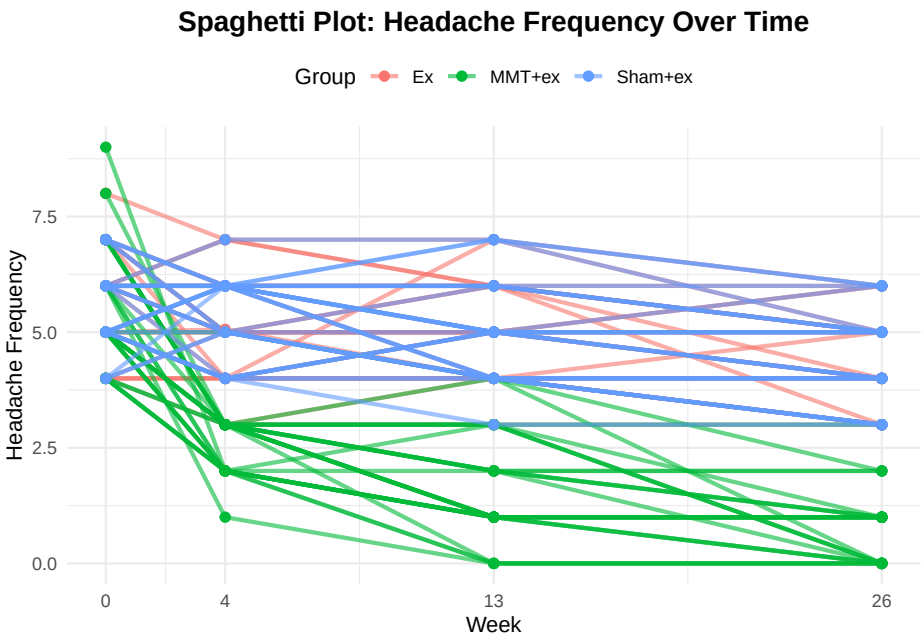
```
# Omit missing values since they will be thrown out anyways when using lmer!
df <- na.omit(df) # 8 rows (patients) are out!

df_long <- df %>%
  pivot_longer(
    cols = starts_with("Week_"),
    names_to = "time",
    values_to = "headache_frequency"
  ) %>%
  mutate(
    headache_frequency = as.numeric(headache_frequency),
    time = dplyr::recode(time,
                  "Week_0" = 0,
                  "Week_4" = 4,
                  "Week_13" = 13,
                  "Week_26" = 26)
  )

df_long$ID <- as.factor(df_long$ID)
df_long$time_f <- factor(df_long$time, levels = c(0, 4, 13, 26))
df_long
```

```
## # A tibble: 364 x 5
##    ID    Group  time headache_frequency time_f
##    <fct> <chr> <dbl>              <dbl> <fct>
##  1 1     Ex        0                  5 0
##  2 1     Ex        4                  4 4
##  3 1     Ex       13                  5 13
##  4 1     Ex       26                  4 26
##  5 2     Ex        0                  7 0
##  6 2     Ex        4                  6 4
##  7 2     Ex       13                  6 13
##  8 2     Ex       26                  5 26
##  9 3     Ex        0                  6 0
## 10 3     Ex        4                  4 4
## # i 354 more rows
```

This seems to be okay. Note that we have 364 lines because of the long format: 91 participants (with complete observations) times 4 time points $(0, 4, 13, 26$ weeks).

Next, we look at the raw data with a spaghetti plot. **It is always a good idea to know your raw data.** Below, we have plotted the headache frequency colored by group. Time (week) is an integer variable and therefore the distances between the time points are not equal.

```
ggplot(df_long, aes(x = time, y = headache_frequency, group = ID, color = Group)) +
  geom_line(alpha = 0.6, linewidth = 1) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = c(0, 4, 13, 26)) +
  labs(
    title = "Spaghetti Plot: Headache Frequency Over Time",
    x = "Week",
    y = "Headache Frequency",
    color = "Group"
  ) +
  theme_minimal() +
  theme(
    plot.title = element_text(size = 14, face = "bold"),
    legend.position = "top"
  ) +
  theme(plot.title = element_text(hjust = 0.5))
```



It seems that the 3 different groups have different slopes which is indicative of an interaction effect between group and time. This means, treatments may differ in their effectiveness. *MMT + ex* seems to be clearly better compared to the other two groups (*Ex* and *Sham + ex*). Since participants were randomly

assigned to the groups, this is indicative of a treatment effect.

With respect to our **clustering**, in our long data set, we have repeated measures for each participant (ID) at different time points. There are no repeated treatments though, since each participant is only in one group (of the three) and they do not crossover to one of the other treatments during the trial. *Why do we not need a random intercept for the group?* We are not drawing the groups from a larger population, the groups are fixed and the only ones of interest. The participants on the other hand are drawn from a larger population. Hence, we only consider the random intercept for each participant.

Let's fit the model using the `lme4` package:

```
library(lme4)
library(emmeans)
model <- lmer(headache_frequency ~ Group * time_f + (1 | ID),
              data = df_long)
summary(model) # ref for time: week 0
```

```
## Linear mixed model fit by REML ['lmerMod']
## Formula: headache_frequency ~ Group * time_f + (1 | ID)
##    Data: df_long
##
## REML criterion at convergence: 993.7
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -2.0566 -0.5788 -0.0496  0.5513  3.7147
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  ID       (Intercept) 0.3449   0.5872
##  Residual             0.6619   0.8136
## Number of obs: 364, groups:  ID, 91
##
## Fixed effects:
##                       Estimate Std. Error t value
## (Intercept)            5.63333    0.18319  30.751
## GroupMMT+ex           -0.06667    0.25907  -0.257
## GroupSham+ex          -0.05269    0.25697  -0.205
## time_f4               -0.63133    0.21007  -3.005
## time_f13              -0.63333    0.21007  -3.015
## time_f26              -1.26667    0.21007  -6.030
## GroupMMT+ex:time_f4   -2.30200    0.29708  -7.749
## GroupSham+ex:time_f4   0.37327    0.29467   1.267
## GroupMMT+ex:time_f13  -3.33333    0.29708 -11.220
## GroupSham+ex:time_f13 -0.01183    0.29467  -0.040
```

```
## GroupMMT+ex:time_f26  -3.53333    0.29708 -11.894
## GroupSham+ex:time_f26  0.07312    0.29467   0.248
##
## Correlation of Fixed Effects:
##             (Intr) GrMMT+ GrpSh+ tim_f4 tm_f13 tm_f26 GMMT+:_4 GS+:_4 GMMT+:_1
## GroupMMT+ex -0.707
## GroupSham+x -0.713  0.504
## time_f4     -0.573  0.405  0.409
## time_f13    -0.573  0.405  0.409  0.500
## time_f26    -0.573  0.405  0.409  0.500  0.500
## GrpMMT+x:_4  0.405 -0.573 -0.289 -0.707 -0.354 -0.354
## GrpShm+x:_4  0.409 -0.289 -0.573 -0.713 -0.356 -0.356  0.504
## GrpMMT+:_13  0.405 -0.573 -0.289 -0.354 -0.707 -0.354  0.500    0.252
## GrpShm+:_13  0.409 -0.289 -0.573 -0.356 -0.713 -0.356  0.252    0.500  0.504
## GrpMMT+:_26  0.405 -0.573 -0.289 -0.354 -0.354 -0.707  0.500    0.252  0.500
## GrpShm+:_26  0.409 -0.289 -0.573 -0.356 -0.356 -0.713  0.252    0.500  0.252
##             GS+:_1 GMMT+:_2
## GroupMMT+ex
## GroupSham+x
## time_f4
## time_f13
## time_f26
## GrpMMT+x:_4
## GrpShm+x:_4
## GrpMMT+:_13
## GrpShm+:_13
## GrpMMT+:_26  0.252
## GrpShm+:_26  0.500  0.504
```

We are estimating headache frequency while considering repeated measures within patients.

In the random effects part of the output, we see again the standard deviation of the random intercept for each patient (0.5872), which is a measure of the variability of headache frequency between patients. Residual variance (0.6619) is high compared to this.

The so-called `Fixed effects` give the overall intercept in all our factor combinations: `(Intercept)` is the overall mean headache frequency at week 0 (reference week) and Group `Ex` (reference group).

Important for us are the so-called **contrasts** (below). These give us the **model-estimated differences** in (expected) headache frequency between the groups at each time point. To be exact, we are actually interested in the **difference in differences** (DiD) (see Table 3 in the paper):

headache frequency$_{\text{MMT+ex}}$−headache frequency$_{\text{Sham+ex}}$ at week 4**minus**headache frequency$_{\text{MMT+ex}}$−headache freq

```r
library(emmeans)
emm <- emmeans(model, ~ Group * time_f)
emm_df <- as.data.frame(emm)
emm_df
```

```
##  Group   time_f   emmean        SE     df lower.CL upper.CL
##  Ex      0      5.633333 0.1831917 260.36 5.272607 5.994059
##  MMT+ex  0      5.566667 0.1831917 260.36 5.205941 5.927393
##  Sham+ex 0      5.580645 0.1802128 260.36 5.225785 5.935505
##  Ex      4      5.002000 0.1831917 260.36 4.641274 5.362726
##  MMT+ex  4      2.633333 0.1831917 260.36 2.272607 2.994059
##  Sham+ex 4      5.322581 0.1802128 260.36 4.967721 5.677441
##  Ex      13     5.000000 0.1831917 260.36 4.639274 5.360726
##  MMT+ex  13     1.600000 0.1831917 260.36 1.239274 1.960726
##  Sham+ex 13     4.935484 0.1802128 260.36 4.580624 5.290344
##  Ex      26     4.366667 0.1831917 260.36 4.005941 4.727393
##  MMT+ex  26     0.766667 0.1831917 260.36 0.405941 1.127393
##  Sham+ex 26     4.387097 0.1802128 260.36 4.032237 4.741957
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

```r
contrast_vec <- list(
  "MMTvsSham_W4_minus_W0" = c(
    0,   # Ex @ 0
    +1,  # MMT+ex @ 0
    -1,  # Sham+ex @ 0
    0,   # Ex @ 4
    -1,  # MMT+ex @ 4
    +1,  # Sham+ex @ 4
    0, 0, 0, 0, 0, 0 # ignore reset (13 & 26)
  )
)

contrast(emm, method = contrast_vec, infer = c(TRUE, FALSE))
```

```
##  contrast              estimate    SE  df lower.CL upper.CL
##  MMTvsSham_W4_minus_W0     2.68 0.295 264      2.1     3.26
##
## Degrees-of-freedom method: kenward-roger
## Confidence level used: 0.95
```

Bonferroni correction is used to adjust the p-values for multiple comparisons (one just divides the "significance"-level by the number of comparisons).

We now look at Table 3 in the paper, first row, first column. There we see the mean between-group difference in headache frequency at week 4 compared to

week 0 and the two groups *MMT + ex* and *MMT Sham + ex* are compared.

- The mean difference (in the paper) is −3 (95% CI: −3 to −2) days/month.
- In our `contrast` output, we find the row `MMTvsSham_W4_minus_W0 2.68 0.295 264 2.1 3.26`.

After rounding to integers we get the same results.

If you want to see the **random effects** of all 91 participants explicitly:

```
ranef(model)  # random effects
```

```
## $ID
##      (Intercept)
## 1  -0.3382077044
## 2   0.6754017992
## 3   0.5064668819
## 4   0.5064668819
## 6  -0.6760775389
## 7   0.3375319647
## 8  -0.1692727871
## 9   0.8443367165
## 10  0.6754017992
## 12  0.5064668819
## 13 -0.0003378698
## 14  1.0132716337
## 15  0.1685970474
## 16 -1.1828822907
## 17 -0.5071426216
## 18  0.3375319647
## 19 -0.1692727871
## 20 -0.6760775389
## 21 -0.1692727871
## 22 -0.0003378698
## 23 -0.6760775389
## 24 -0.6760775389
## 25 -0.8450124561
## 26  0.3375319647
## 28  0.6754017992
## 29  0.3375319647
## 30 -0.1692727871
## 31 -0.3382077044
## 32 -0.0003378698
## 33 -0.3280716093
## 34 -0.6025345382
## 35  0.5800098826
## 36  0.5800098826
## 37  0.0732051308
```

```
## 39  0.4110749653
## 40  0.4110749653
## 41  0.2421400481
## 42  0.0732051308
## 43 -0.0957297864
## 44  0.4110749653
## 46 -0.2646647037
## 47 -0.6025345382
## 48  0.5800098826
## 49  0.2421400481
## 50 -0.2646647037
## 51  0.2421400481
## 52 -0.2646647037
## 53 -0.6025345382
## 54  0.0732051308
## 55 -0.0957297864
## 56 -0.2646647037
## 57 -0.6025345382
## 58  0.9178797171
## 60 -0.0957297864
## 61  0.0732051308
## 62 -0.2646647037
## 63 -0.4335996210
## 64  0.2421400481
## 65 -0.2646647037
## 66 -0.4335996210
## 67 -0.2070815115
## 68 -0.2070815115
## 69 -0.3760164287
## 70 -0.2070815115
## 71  0.2997232403
## 72  0.2997232403
## 73 -0.3760164287
## 74  0.4686581576
## 75 -0.2070815115
## 76 -0.2070815115
## 77  0.4686581576
## 78  0.9754629093
## 79  0.4686581576
## 81 -0.0381465942
## 82  0.2997232403
## 83 -0.0381465942
## 84 -0.3760164287
## 85 -0.0381465942
## 86 -0.5449513460
## 87 -0.0381465942
```

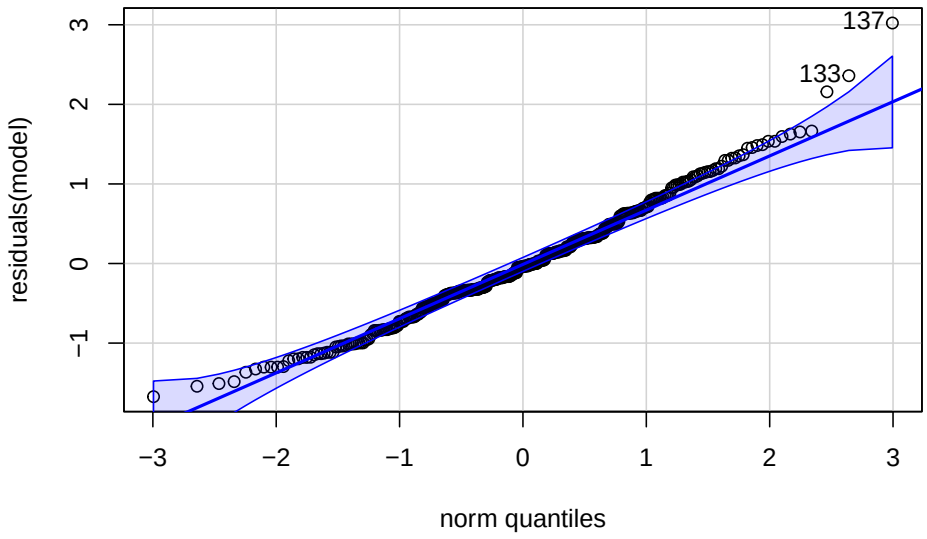
```
## 88  0.4686581576
## 90 -0.2070815115
## 91  0.9754629093
## 92 -0.2070815115
## 93  0.4686581576
## 94 -0.5449513460
## 95  0.8065279921
## 96 -0.7138862633
## 97 -0.2070815115
## 98 -0.7138862633
## 99 -0.5449513460
##
## with conditional variances for "ID"
```

```
re <- ranef(model)
str(re$ID) # 91 obs, 91 participants
```

```
## 'data.frame':    91 obs. of  1 variable:
##  $ (Intercept): num  -0.338 0.675 0.506 0.506 -0.676 ...
##  - attr(*, "postVar")= num [1, 1, 1:91] 0.112 0.112 0.112 0.112 0.112 ...
```

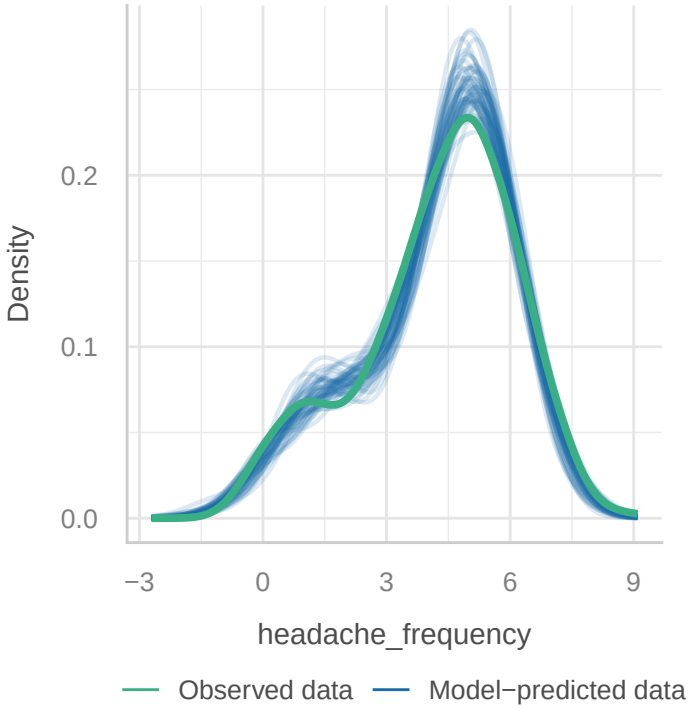
Posterior Predictive checks and **residuals** look surprisingly good:

```
library(car)
library(performance)
qqPlot(residuals(model))
```




```
## [1] 137 133
```

```
check_model(model, check = "pp_check")
```



The density estimators might be a bit misleading since we only have a rather small number of different levels and of course a count variable can not be negative.

3.1.2.2 Bayesian attempt to replicate the results

We can also try to fit the model using the `rethinking` package.

Model equations and priors:

$$
\begin{aligned}
\text{headache frequency}_{ij} &\sim \text{Normal}(\mu_{ij}, \sigma) \\
\mu_{ij} &= \alpha + \beta_1 \text{group}_i + \beta_2 \text{time}_j + \beta_3 (\text{group}_i \times \text{time}_j) + u_i \\
\alpha &\sim \text{Normal}(5, 1.5) \\
\beta_1[group] &\sim \text{Normal}(0, 1.5) \\
\beta_2[time] &\sim \text{Normal}(0, 1.5) \\
\beta_3[group * time] &\sim \text{Normal}(0, 1.5) \\
u_i &\sim \text{Normal}(0, \sigma_u) \\
\sigma_u &\sim \text{Exponential}(1) \\
\sigma &\sim \text{Exponential}(1)
\end{aligned}
$$

The priors for β_3 is hard since there is no natural meaning to it. Note that we could also combine α and the u_i as we did before in the partial pooling case for reliability. This should deliver similar results (exercise 10). With this model we now fit intercepts for every group (3), every time point (4) and for all $3 * 4 = 12$ interaction combinations. Since we are not working with a reference level for the factors (as in `lmer`), you see all $3, 4$ and 12 (mean posterior) levels of the intercepts in the output of `precis(m)`. Refer to section 5.3 (Categorical variables) in the book *Rethinking* for more details.

```
library(rethinking)

df_long <- df_long %>%
  dplyr::group_by(Group, time) %>%
  dplyr::mutate(interaction = cur_group_id()) %>%
  dplyr::ungroup()
# cur_group_id() gives a unique numeric identifier for the current group.

dat <- list(
  H = df_long$headache_frequency,
  group = as.integer(as.factor(df_long$Group)),    # 1 = Ex, 2 = MMT+ex, 3 = Sham+ex
  time = as.integer(df_long$time_f),               # 1 = Woche 0, ..., 4 = Woche 26
  ID = as.integer(as.factor(df_long$ID)),
  N = nrow(df_long),
  N_ID = length(unique(df_long$ID)),
  N_group = length(unique(df_long$Group)),
  N_time = length(unique(df_long$time_f)),
  interaction = df_long$interaction
)

m <- ulam(
  alist(
    H ~ normal(mu, sigma),
    mu <- alpha +
      beta_group[group] + beta_time[time] + beta_interaction[interaction] + u[ID],
    # random intercept for each participant
    u[ID] ~ normal(0, sigma_ID),
    # Priors
    alpha ~ normal(0, 10),
    beta_group[group] ~ normal(0, 2),
    beta_time[time] ~ normal(0, 2),
    beta_interaction[interaction] ~ normal(0, 2), # difficult
    sigma ~ exponential(1),
    sigma_ID ~ exponential(1)
  ),
  data = dat,
  chains = 4,
```

```
  cores = 4
)
```

```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be reject
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/va
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable
## Chain 1 but if this warning occurs often then your model may be either severely ill-
## Chain 1
## Chain 2 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 2 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 3 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 4 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 4 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 3 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 3 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 4 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 2 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 3 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 4 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 1 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 2 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%]  (Sampling)
```

```
## Chain 2 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 1 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 3 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 2 finished in 3.0 seconds.
## Chain 3 finished in 3.0 seconds.
## Chain 4 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 4 finished in 3.1 seconds.
## Chain 1 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 1 finished in 3.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 3.1 seconds.
## Total execution time: 3.4 seconds.
```

```
precis(m, depth = 2)
```

```
##                          mean         sd        5.5%        94.5%      rhat
## u[1]                -0.337127650 0.34253795 -0.89186548  0.196423219 1.0035350
## u[2]                 0.674342106 0.35125643  0.11962047  1.244113618 1.0031682
## u[3]                 0.491657435 0.34898385 -0.06927226  1.059162942 1.0052454
## u[4]                 0.497952073 0.33483894 -0.03150312  1.027164393 1.0009817
## u[5]                -0.670080237 0.35976367 -1.24808318 -0.097855177 1.0070191
## u[6]                 0.329085030 0.34407158 -0.21335883  0.873768513 1.0017772
## u[7]                -0.170025945 0.35656415 -0.74680654  0.386595910 1.0052397
## u[8]                 0.838927991 0.34992730  0.29131218  1.402975835 1.0031028
## u[9]                 0.670779157 0.35925766  0.11371142  1.235623813 0.9998354
## u[10]                0.496734421 0.34734839 -0.05229115  1.044468603 1.0010692
## u[11]               -0.006990644 0.34314779 -0.54261172  0.554184968 1.0012497
## u[12]                1.003282875 0.35805479  0.43562903  1.591866610 1.0000800
## u[13]                0.169631549 0.33674670 -0.38062061  0.700714365 1.0082322
## u[14]               -1.186208454 0.36995616 -1.79068745 -0.585241706 1.0012521
## u[15]               -0.512298420 0.34365918 -1.03974948  0.039454998 1.0041410
## u[16]                0.332723027 0.33549824 -0.21504772  0.860813862 1.0003014
## u[17]               -0.174759201 0.36312307 -0.75137597  0.402268041 0.9989290
## u[18]               -0.676208480 0.35967634 -1.26383540 -0.095767156 1.0059116
```

```
## u[19]  -0.170374300 0.33194972 -0.70326450  0.369975091 1.0092756
## u[20]  -0.003197010 0.35575212 -0.56601149  0.559372727 1.0017730
## u[21]  -0.679477337 0.34249527 -1.22262147 -0.130318770 1.0007933
## u[22]  -0.673394334 0.34111800 -1.22198242 -0.132575180 1.0008260
## u[23]  -0.848459861 0.35540461 -1.40995135 -0.303205738 1.0022578
## u[24]   0.339492349 0.35522957 -0.21720119  0.921766188 1.0035880
## u[25]   0.659402841 0.35378236  0.08631137  1.196531753 1.0030247
## u[26]   0.340822740 0.34130559 -0.20853260  0.880763753 1.0068953
## u[27]  -0.179408621 0.35703294 -0.74346997  0.391867744 1.0023505
## u[28]  -0.342911489 0.34164143 -0.88958914  0.211778449 1.0028246
## u[29]  -0.003645721 0.35646967 -0.56116848  0.598281168 0.9990798
## u[30]  -0.329120604 0.37162678 -0.93867134  0.252788053 0.9992045
## u[31]  -0.606169583 0.35887938 -1.17946836 -0.034321861 1.0022504
## u[32]   0.570678919 0.35406015  0.01885525  1.149740342 1.0019711
## u[33]   0.577722783 0.33788485  0.04612221  1.126426128 1.0008750
## u[34]   0.067801061 0.33541390 -0.48123577  0.598514205 0.9992843
## u[35]   0.407549656 0.33577141 -0.12118535  0.945824388 1.0072491
## u[36]   0.408973997 0.35574410 -0.13742670  0.958021914 0.9997106
## u[37]   0.233988294 0.34328821 -0.30284421  0.787054991 1.0043419
## u[38]   0.069778653 0.33824635 -0.47036054  0.626452542 1.0009308
## u[39]  -0.104982908 0.34687705 -0.65537825  0.465851022 0.9991516
## u[40]   0.401220194 0.35403525 -0.16590164  0.962760506 1.0021203
## u[41]  -0.267422617 0.34554960 -0.81570583  0.270627691 1.0048217
## u[42]  -0.601348262 0.36115491 -1.19242186 -0.022099327 1.0012494
## u[43]   0.578815049 0.35573175  0.02981861  1.128716848 1.0064923
## u[44]   0.230252408 0.33575337 -0.29262751  0.752231420 1.0005804
## u[45]  -0.263098517 0.35819708 -0.84565271  0.295185216 0.9991489
## u[46]   0.234821720 0.34872831 -0.31475940  0.790491097 1.0028688
## u[47]  -0.268265038 0.34821050 -0.82714643  0.279794781 1.0021840
## u[48]  -0.597613430 0.35751498 -1.16590984 -0.008009211 0.9991288
## u[49]   0.080071411 0.35514757 -0.47879129  0.656419766 1.0031034
## u[50]  -0.094905410 0.33672574 -0.60775799  0.452160454 1.0029882
## u[51]  -0.266431376 0.33375457 -0.81188465  0.258864493 1.0013209
## u[52]  -0.605389252 0.34233355 -1.15478460 -0.052617604 1.0002726
## u[53]   0.913967036 0.35936940  0.36223603  1.503158143 1.0049619
## u[54]  -0.094259738 0.34229204 -0.64496040  0.459361684 1.0024301
## u[55]   0.071237089 0.34062303 -0.48548029  0.602445955 1.0025961
## u[56]  -0.265471673 0.33791734 -0.81450547  0.275555022 1.0027962
## u[57]  -0.437171448 0.34171973 -0.99169674  0.106893813 1.0020589
## u[58]   0.235589744 0.34373465 -0.30753550  0.779751696 1.0025677
## u[59]  -0.264957982 0.34166226 -0.80043936  0.289043876 1.0002386
## u[60]  -0.434860224 0.35231562 -1.02550785  0.125031122 1.0006694
## u[61]  -0.211657618 0.35694504 -0.78402871  0.356887975 1.0011604
## u[62]  -0.204951303 0.33923523 -0.74204438  0.348086971 1.0013702
## u[63]  -0.369092765 0.33889662 -0.90803703  0.179361821 0.9995761
## u[64]  -0.210113708 0.35854190 -0.78325579  0.339609116 1.0016975
```

```
## u[65]                  0.297584431 0.34160535 -0.24819613  0.820526907 1.0054214
## u[66]                  0.293148426 0.34625403 -0.26275111  0.856683368 1.0022404
## u[67]                 -0.368874505 0.35930951 -0.94006092  0.207947011 1.0039444
## u[68]                  0.467242123 0.33855616 -0.08058246  0.995832602 1.0001017
## u[69]                 -0.213195563 0.35650657 -0.77835150  0.360085086 1.0068269
## u[70]                 -0.204576028 0.33595236 -0.73389842  0.330641896 1.0027028
## u[71]                  0.458132770 0.35804702 -0.09809697  1.026747693 1.0015277
## u[72]                  0.967604165 0.35150263  0.41228851  1.544897590 1.0036363
## u[73]                  0.467311555 0.35026387 -0.11485776  1.025118474 1.0008388
## u[74]                 -0.033076258 0.35060545 -0.59562554  0.507460376 1.0043723
## u[75]                  0.303074574 0.35613257 -0.26112133  0.880553749 1.0054694
## u[76]                 -0.036929169 0.35367921 -0.58573408  0.511904289 1.0029964
## u[77]                 -0.373287329 0.35840506 -0.95037023  0.193830215 1.0009863
## u[78]                 -0.032458445 0.34698372 -0.58446737  0.521201517 1.0040152
## u[79]                 -0.546493669 0.36181775 -1.14313620  0.034444739 1.0001499
## u[80]                 -0.038379970 0.34995871 -0.58714623  0.503523319 1.0024592
## u[81]                  0.467896557 0.34079247 -0.08699695  1.016713762 1.0042371
## u[82]                 -0.211755333 0.34951583 -0.75844478  0.343520117 1.0085887
## u[83]                  0.974388497 0.35764270  0.40732490  1.548194969 1.0056134
## u[84]                 -0.211505145 0.33609254 -0.73853585  0.315226342 1.0003434
## u[85]                  0.466157147 0.33864727 -0.07700220  1.006823787 1.0001334
## u[86]                 -0.543142686 0.35006740 -1.08432794 -0.001150082 1.0038596
## u[87]                  0.805065454 0.35506800  0.26339488  1.378853899 1.0007497
## u[88]                 -0.720224068 0.34493991 -1.28103849 -0.174145720 1.0024151
## u[89]                 -0.202983309 0.35161723 -0.77552745  0.363461781 1.0096569
## u[90]                 -0.705823677 0.36545585 -1.30790443 -0.117473339 1.0003866
## u[91]                 -0.544537275 0.35447762 -1.09764694  0.016453746 1.0010138
## alpha                  4.073470738 1.63533354  1.39442476  6.621047087 1.0041581
## beta_group[1]          0.700958629 1.36525094 -1.44880205  2.819580405 1.0046752
## beta_group[2]         -1.223744619 1.40289180 -3.44550926  1.019537440 1.0030525
## beta_group[3]          0.738168997 1.38288553 -1.50685252  2.926934758 1.0008939
## beta_time[1]           1.086408610 1.30035288 -0.93246956  3.185273922 1.0047334
## beta_time[2]           0.114308422 1.31153487 -1.98372636  2.189991667 1.0025640
## beta_time[3]          -0.210912099 1.30929404 -2.30948175  1.873621811 1.0025946
## beta_time[4]          -0.748129130 1.34076459 -2.91102317  1.447522198 0.9994713
## beta_interaction[1]   -0.219857169 1.23184464 -2.18371504  1.729495519 1.0004478
## beta_interaction[2]    0.114858563 1.25016073 -1.87413796  2.085339323 0.9993658
## beta_interaction[3]    0.436410234 1.24523728 -1.53193488  2.430502579 1.0039685
## beta_interaction[4]    0.341537669 1.20704125 -1.54806577  2.275812046 0.9996896
## beta_interaction[5]    1.620060864 1.21874776 -0.34887829  3.548850326 1.0009218
## beta_interaction[6]   -0.325934683 1.31831691 -2.42140019  1.847791222 1.0008625
## beta_interaction[7]   -1.034910754 1.26638954 -2.99352660  1.061347662 1.0018648
## beta_interaction[8]   -1.323765878 1.27747074 -3.36236541  0.738125830 1.0015842
## beta_interaction[9]   -0.318879107 1.16928168 -2.06335653  1.582090375 1.0007553
## beta_interaction[10]   0.392691776 1.26875163 -1.55964076  2.417551264 1.0016268
## beta_interaction[11]   0.336247731 1.24056047 -1.59568183  2.320630384 1.0037644
```

```
## beta_interaction[12]  0.321490796 1.24922675 -1.72514836  2.372293628 1.0006905
## sigma                 0.816356142 0.03675947  0.76131072  0.878293359 1.0018126
## sigma_ID              0.591733610 0.06760260  0.48848798  0.701841854 1.0055118
##                      ess_bulk
## u[1]                 4267.171
## u[2]                 3674.836
## u[3]                 4629.306
## u[4]                 4581.993
## u[5]                 4150.488
## u[6]                 3298.619
## u[7]                 5903.838
## u[8]                 3979.971
## u[9]                 3964.401
## u[10]                4157.794
## u[11]                4102.543
## u[12]                3919.550
## u[13]                4533.097
## u[14]                3183.787
## u[15]                3969.155
## u[16]                3754.101
## u[17]                4075.330
## u[18]                4107.652
## u[19]                3624.178
## u[20]                6214.440
## u[21]                3793.604
## u[22]                3894.256
## u[23]                4239.671
## u[24]                3566.625
## u[25]                3668.775
## u[26]                4470.020
## u[27]                5028.628
## u[28]                4279.277
## u[29]                5021.594
## u[30]                6026.717
## u[31]                3695.032
## u[32]                4947.546
## u[33]                4013.000
## u[34]                4321.455
## u[35]                5426.489
## u[36]                4292.997
## u[37]                3683.700
## u[38]                3794.213
## u[39]                4715.950
## u[40]                4337.661
## u[41]                4728.992
## u[42]                4082.269
```

```
## u[43]                3776.737
## u[44]                4440.680
## u[45]                4480.940
## u[46]                4646.379
## u[47]                3501.713
## u[48]                3785.334
## u[49]                4689.774
## u[50]                4149.231
## u[51]                4052.981
## u[52]                3336.357
## u[53]                3230.554
## u[54]                3530.138
## u[55]                4601.900
## u[56]                3813.963
## u[57]                4234.310
## u[58]                4141.626
## u[59]                4284.520
## u[60]                4076.590
## u[61]                3491.830
## u[62]                3930.916
## u[63]                3412.474
## u[64]                3935.841
## u[65]                3476.533
## u[66]                3785.776
## u[67]                3916.913
## u[68]                3761.382
## u[69]                3635.723
## u[70]                3265.356
## u[71]                3747.623
## u[72]                2839.655
## u[73]                4194.040
## u[74]                3507.218
## u[75]                3310.016
## u[76]                4802.561
## u[77]                3884.595
## u[78]                4081.670
## u[79]                3519.573
## u[80]                3616.462
## u[81]                3925.719
## u[82]                4348.397
## u[83]                3635.495
## u[84]                3506.617
## u[85]                3968.639
## u[86]                3300.865
## u[87]                3349.851
## u[88]                3105.358
```

```
## u[89]                3508.942
## u[90]                4115.760
## u[91]                3429.673
## alpha                1217.558
## beta_group[1]        1455.815
## beta_group[2]        1785.890
## beta_group[3]        1615.546
## beta_time[1]         1616.165
## beta_time[2]         1681.926
## beta_time[3]         1744.554
## beta_time[4]         1609.330
## beta_interaction[1]  1917.631
## beta_interaction[2]  1956.421
## beta_interaction[3]  1637.660
## beta_interaction[4]  1688.841
## beta_interaction[5]  1852.711
## beta_interaction[6]  1879.070
## beta_interaction[7]  1799.740
## beta_interaction[8]  1916.373
## beta_interaction[9]  1872.252
## beta_interaction[10] 1939.265
## beta_interaction[11] 1594.484
## beta_interaction[12] 1770.927
## sigma                2361.127
## sigma_ID             1199.407
```

`precis(m, depth = 2)` confirms that `sigma`and `sigma_ID` are in line with `lmer`.

Next, we want to know the posterior distribution (respectively the mean and credible interval) of the difference in difference:

$$(MMT + ex - Sham + ex)_{\text{week}_4} - (MMT + ex - Sham + ex)_{\text{week}_0}$$

The result is rather similar to the Frequentist model.

```
# find correct indices for the interaction terms:

levels(as.factor(df_long$Group)) # 2 minus 3
```

```
## [1] "Ex"      "MMT+ex"  "Sham+ex"
```

```
intdf <- df_long %>%
  dplyr::group_by(Group, time) %>%
  dplyr::select(Group, time) %>%
  unique()
intdf$interaction_ID <- 1:nrow(intdf)

# Extract posterior samples
```

```
post <- extract.samples(m)

# --------------------------------
# Compute group differences at week 4 and week 0
# --------------------------------

# MMT+ex vs Sham+ex at week 4:
# Index 6 = MMT+ex, week 4
# Index 10 = Sham+ex, week 4
diff_w4 <- post$beta_group[,2] - post$beta_group[,3] +
           post$beta_interaction[,6] - post$beta_interaction[,10]

# MMT+ex vs Sham+ex at week 0:
# Index 5 = MMT+ex, week 0
# Index 9 = Sham+ex, week 0
diff_w0 <- post$beta_group[,2] - post$beta_group[,3] +
           post$beta_interaction[,5] - post$beta_interaction[,9]

# --------------------------------
# Difference-in-Differences
# --------------------------------

diff_in_diff <- diff_w4 - diff_w0

# Summarize posterior distribution (mean and 95% credible interval)
precis(data.frame(diff_in_diff), prob = 0.95)
```

```
##                   mean        sd      2.5%     97.5%   histogram
## diff_in_diff -2.657566 0.2947512 -3.252681 -2.065503
```

3.2 Logistic Regression

3.2.1 Simple Logistic Regression

Another basic regression model is the **logistic regression**. There are entire books on the topic.

Watch these videos as an introduction: 1 2.

In our Bayesian (rethinking) framework, this topic is not really new. We have already used it in our tadpole example when we modeled the survival of tadpoles in tanks. There, our outcome variable was the *number of surviving tadpoles* in a tank, which is a count variable and was modeled using a binomial distribution.

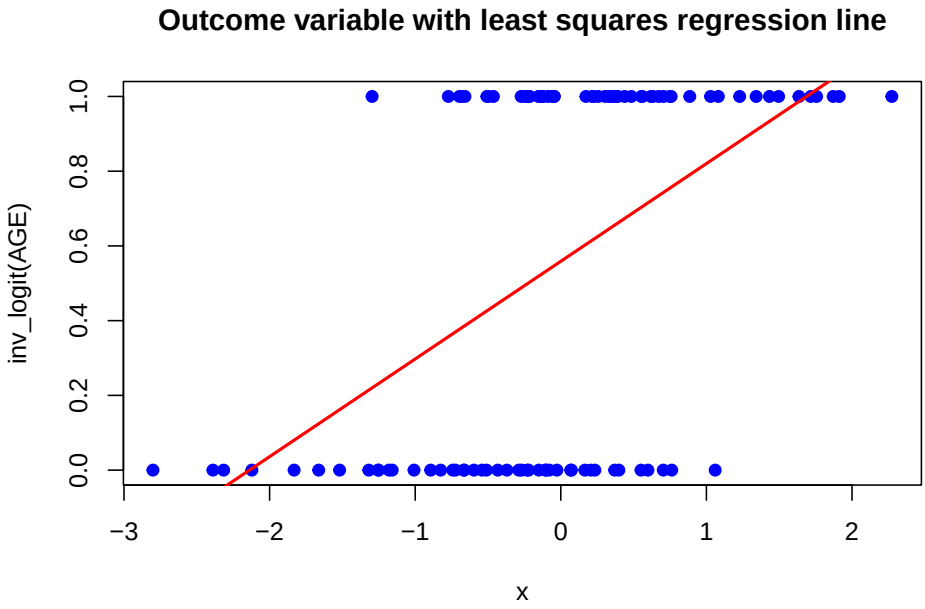
In logistic regression, we just model a **binary** outcome variable, with 0 and 1 (*Yes*/*No*; *died*/*survived*...) as possible values.

Of course, the great disadvantage of this model is the low information content of the outcome variable, which can only take two values. We could then ask ourselves, how the probability of a 1 (e.g. survival) changes with respect to the predictors.

Obviously, the classical linear regression makes no sense for this kind of data.

Let's quickly invent an example:

```
set.seed(332)
AGE <- rnorm(100) # standardized age of 100 participants
y <- rbinom(100, size = 1, prob = inv_logit(AGE))
plot(AGE, y, main = "Outcome variable with least squares regression line",
     xlab = "x", ylab = "inv_logit(AGE)", col = "blue", pch = 19)
abline(lm(y ~ AGE), col = "red", lwd = 2)
```



The higher the `AGE`, the higher the probability of a 1 (heart attack or something). In this case (of a simulation) we know the trajectory of the *true* probabilities varying with AGE. Of course, normally, we do not know this but conceptually assume there are such true probabilities.

The red line is the **least squares regression** line, which **does not make much sense** here: The outcome can only be 0 or 1 but the regression line would estimate all values in between plus values outside of this range, which is not possible.

We therefore bend our reality a bit and **model the probability of a** 1 ($= p_i = \mathbb{E}(Y_i)$ if Y_i is Bernoulli distributed). Of course, then we have the next problem, if we try to model the probability of a 1 *directly* using the linear regression model

($\mathbb{P}(Y_i = 1) = \beta_0 + \beta_1 x_1 + \cdots \beta_p x_p$). The linear predictor ($\beta_0 + \beta_1 x_1 + \cdots \beta_p x_p$) could easily lie outside of the range $[0, 1]$ (for adequate βs and covariate values).

Therefore, we use the **logit function** to transform the probability from $[0, 1]$ to $(-\infty, +\infty)$. This transformed range can be conveniently described using a linear predictor again. The left side and the right side of the equation are now equal in range ($(-\infty, +\infty)$). Very nice.

The model is then ($k = 1$ for the simple logistic regression with only one predictor):

$$\begin{aligned} \mathbb{P}(Y_i = 1|x_{i1}, \ldots, x_{ik}) &= \mathbb{E}(Y_i|x_{i1}, \ldots, x_{ik}) = p_i \\ \text{logit}(p_i) &= \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ik} \end{aligned}$$

where p_i is the probability of a 1 for observation i and $\text{logit}(p_i) = \log\left(\frac{p_i}{1-p_i}\right)$.

Notice that we do not need a random error term ε_i anymore, since these kinds of models (so-called generalized linear models; GLM) are estimated using the maximum likelihood method. See exercise 15.

3.2.1.1 Frequentist version

In current literature, you will see the model estimated mostly in the **Frequentist setting**:

```
modlog <- glm(y ~ AGE, family = binomial(link = "logit"))
summary(modlog)
```

```
##
## Call:
## glm(formula = y ~ AGE, family = binomial(link = "logit"))
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)   0.2992     0.2352   1.272    0.203
## AGE           1.5215     0.3588   4.240 2.23e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 137.63  on 99  degrees of freedom
## Residual deviance: 108.20  on 98  degrees of freedom
## AIC: 112.2
##
## Number of Fisher Scoring iterations: 4
```

```
confint(modlog)
```

```
## Waiting for profiling to be done...
```

```
##                  2.5 %    97.5 %
## (Intercept) -0.1572572 0.7708586
## AGE          0.8860184 2.3038479
```

- `glm`stands for *generalized linear model*. "The GLM generalizes linear regression by allowing the linear model to be related to the response variable via a link function and by allowing the magnitude of the variance of each measurement to be a function of its predicted value."
- `family = binomial(link = "logit")` specifies that we want to use the binomial distribution with the `logit` link function g. In generalized linear models, the link function is used to connect the linear predictor (the right side of the equation) to the expected value of the response variable: In our case, the expected value of a Bernoulli-distributed variable (which is binary) is the probability of a 1: p_i.

  In general:
  $$\mathbb{E}(Y|X) = g^{-1}(X\beta)$$
  Think back to exercise 5 in the reliability chapter and go to exercise 11.
- `Coefficients` are the estimated parameters of the model **on the logit scale**.

  Interpretation: Two participants with a difference of 1 standard deviation (since we standardized age) in their age, have an expected difference of $\beta_1 = 1.5215$ in the log-odds of having a heart attack.

  If we want the **odds ratio**, we can just exponentiate:

  $$log(\frac{p_i}{1-p_i}) = \beta_0 + \beta_1 AGE$$
  $$\text{Odds for a 1} = \frac{p_i}{1-p_i} = e^{\beta_0+\beta_1 AGE} = e^{\beta_0} \cdot e^{\beta_1 AGE}$$

  If you add 1 to AGE:

  Odds for a 1 with $(AGE{+}1) = e^{\beta_0} \cdot e^{\beta_1 (AGE+1)} = e^{\beta_0} \cdot e^{\beta_1 AGE} \cdot e^{\beta_1} = \text{Odds for a 1} \cdot e^{\beta_1}$

  So, **on the odds scale**, you **multiply the odds for a** 1 **by** e^{β_1}.

  We can also work **directly on the probability scale**: The difference in the probability of a 1 for two participants with a difference of 1 year in their age is:

$$\mathbb{P}(Y_i = 1 \mid AGE + 1) - \mathbb{P}(Y_i = 1 \mid AGE) = \text{inv\_logit}(\beta_0 + \beta_1(AGE + 1)) - \text{inv\_logit}(\beta_0 + \beta_1 AGE)$$

This difference is not constant (see graph below, red line), but depends on the value of AGE, since the inverse logit function is non-linear.

- `Null deviance` is the deviance (= −2log-likelihood) of the model with only the intercept (no predictors). The deviance is a measure of how well the model fits the data.

```
-2 * logLik(glm(y ~ 1, family = binomial(link = "logit")))
```

```
## 'log Lik.' 137.6278 (df=1)
```

- `Residual deviance` is the deviance of the model with the predictors (in our case: `AGE`).

```
 -2 * logLik(modlog)
```

```
## 'log Lik.' 108.1987 (df=2)
```

- `AIC` is the Akaike Information Criterion, which is a measure of the model's goodness of fit, penalized for the number of parameters. Lower AIC values indicate a better fit.

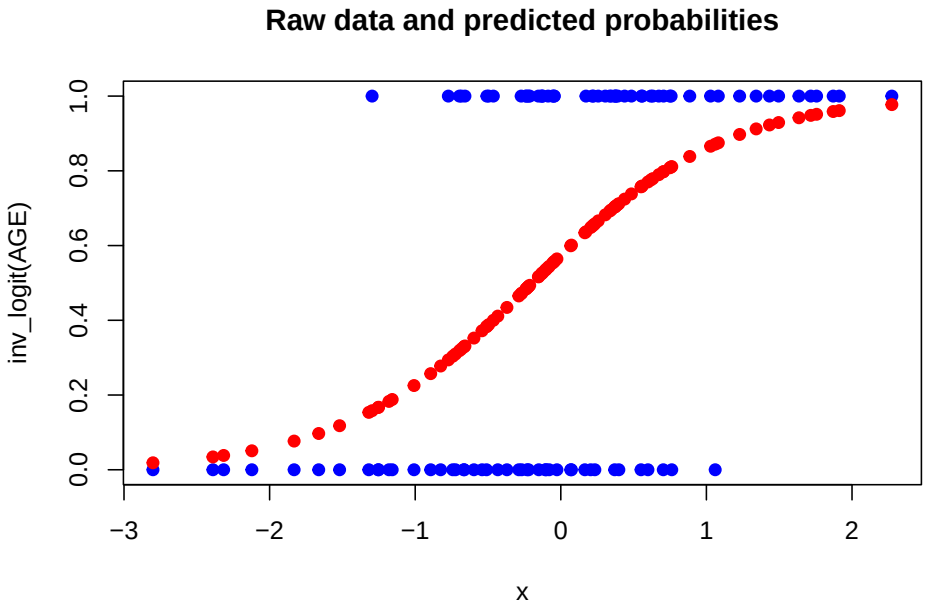
```
-2 * logLik(modlog) + 2 * length(coef(modlog))
```

```
## 'log Lik.' 112.1987 (df=2)
```

The more parameters, the higher the AIC, the worse the model fit. AIC punishes model complexity.

We can calculate the probabilities estimated by data:

```
predicted_probabilities <- predict(modlog, type = "response") # response is the probability p_i h
plot(AGE, y, main = "Raw data and predicted probabilities",
     xlab = "x", ylab = "inv_logit(AGE)", col = "blue", pch = 19)
points(AGE, predicted_probabilities, col = "red", pch = 19)
```




```
# The help file says about the predict-function:# the type of
# prediction required.

# The default is on the scale of the linear
# predictors;

# the alternative "response" is on the scale of the
# response variable.

# Thus for a default binomial model the default
# predictions are of log-odds (probabilities on logit scale) and
# type = "response" gives the predicted probabilities.
```

As we can see, higher `AGE` (linear predictor with only one term in this case) is *associated* (let's not say "influenced" please) with higher probability of a 1.

The red line in the plot is the predicted probability of a 1. The formula for the red line is:

$$\mathbb{P}(Y_i = 1 \mid AGE) = \text{inv\_logit}(\beta_0 + \beta_1 AGE) = \frac{e^{\beta_0 + \beta_1 AGE}}{1 + e^{\beta_0 + \beta_1 AGE}}$$

where inv_logit(x) is the inverse logit function, which is the same as the logistic function.

- If $\beta_1 > 0$, the probability of a 1 increases with increasing `AGE` (as above).
- If $\beta_1 < 0$, the probability of a 1 decreases with increasing `AGE`.

- For very large or very small values of `AGE`, the probability of $Y_i = 1$ approaches 1 and 0 respectively. So this works nicely to describe the probability of $Y_i = 1$ along the linear predictor $\beta_0 + \beta_1 AGE$.
- Extending the linear predictor $\beta_0 + \beta_1 AGE$ to more terms is straightforward ($\beta_0 + \beta_1 AGE + \beta_2 X_2 + \cdots \beta_p X_p$) and gets us to multiple logistic regression.

Let's **present the results** nicely on the odds ratio scale with confidence intervals (CIs), which can be conveniently calculated using R:

(If you haven't thought about it: What is the difference between odds ratios, risk ratios and probabilities?)

```
library(broom)
tidy_modlog <- tidy(modlog, conf.int = TRUE, exponentiate = TRUE)
tidy_modlog
```

```
## # A tibble: 2 x 7
##   term        estimate std.error statistic   p.value conf.low conf.high
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
## 1 (Intercept)     1.35     0.235      1.27 0.203        0.854      2.16
## 2 AGE             4.58     0.359      4.24 0.0000223    2.43      10.0
```

- `conf.int = TRUE` adds the confidence intervals to the output.
- `exponentiate = TRUE` exponentiates the coefficients to give us the odds ratios.
- At $AGE = 0$ (could be average age), the odds for our event (maybe heart attack) are 1.35 (95% CI: 0.854 to 2.16). This is rather close to the overall odds without the model: There are 55 1s and 45 0s, hence the overall probability $\frac{55}{100} = 0.55$. The odds are therefore $\frac{0.55}{1-0.55} = 1.22$.
- Increasing the `AGE` by 1 (SD) year increases the odds of a 1 by a factor of 4.58.

Hint: You can also use `gtsummary` and `tbl_regression` to present the results nicely.

Let's verify all this in R:

```
# log-odds scale:
predict(modlog, type = "link",
        newdata = data.frame(AGE = 0))
```

```
##         1
## 0.2991623
```

```
predict(modlog, type = "link",
        newdata = data.frame(AGE = 1))
```

```
##       1
## 1.82066
```

```
# odds scale:
exp(predict(modlog, type = "link",
            newdata = data.frame(AGE = 0)))
```

```
##        1
## 1.348729
```

```
exp(predict(modlog, type = "link",
            newdata = data.frame(AGE = 1)))
```

```
##        1
## 6.175935
```

```
6.175935 / 1.348729
```

```
## [1] 4.579078
```

```
# probability-scale:
predict(modlog, type = "response",
        newdata = data.frame(AGE = 0))
```

```
##         1
## 0.5742377
```

```
predict(modlog, type = "response",
        newdata = data.frame(AGE = 1))
```

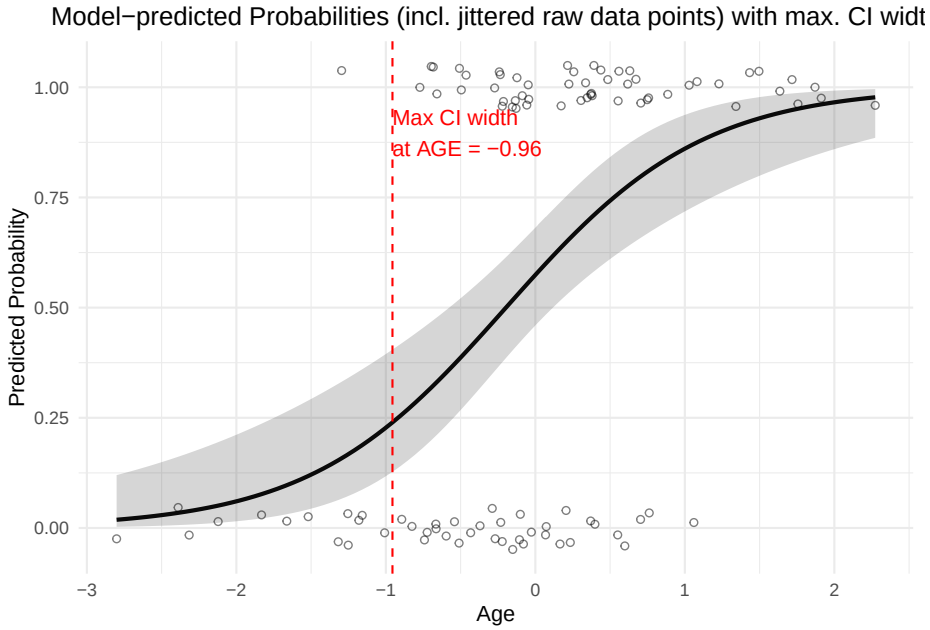
```
##         1
## 0.8606453
```

Very nice.

Another cool feature is to visualize the predicted probabilities with a **confidence band**:

```
library(tidyverse)

# Create prediction grid for AGE
AGE_grid <- seq(min(AGE), max(AGE), length.out = 100)

# Predict on link scale (logit), get standard errors
preds <- predict(modlog,
                 newdata = data.frame(AGE = AGE_grid),
                 type = "link", se.fit = TRUE)

# Construct dataframe and transform to probability scale
df_pred <- data.frame(
  AGE = AGE_grid,
  fit = preds$fit,
  se = preds$se.fit
```

```
) %>%
  mutate(
    conf.low = inv_logit(fit - 1.96 * se),
    conf.high = inv_logit(fit + 1.96 * se),
    predicted = inv_logit(fit),
    ci_width = conf.high - conf.low
  )

# Identify max CI width
max_ci_row <- df_pred %>% slice_max(order_by = ci_width, n = 1)

# Plot
ggplot(df_pred, aes(x = AGE, y = predicted)) +
  geom_line(linewidth = 1) +
  geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.2) +
  geom_jitter(data = data.frame(x = AGE, y = y),
              aes(x = x, y = y),
              width = 0, height = 0.05,
              shape = 1, color = "black", alpha = 0.5) +
  geom_vline(xintercept = max_ci_row$AGE, linetype = "dashed", color = "red") +
  annotate("text", x = max_ci_row$AGE, y = 0.95,
           label = paste0("Max CI width\nat AGE = ", round(max_ci_row$AGE, 2)),
           color = "red", hjust = 0, vjust = 1) +
  labs(
    title = "Model-predicted Probabilities (incl. jittered raw data points) with max. CI width",
    x = "Age",
    y = "Predicted Probability"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5)) +
  coord_cartesian(clip = "off")
```



As you can see, the confidence band is wider when there are fewer data points. In the middle of the `AGE` range, the confidence band is narrower, which is expected since we have more data points there. At the extreme ends of the `AGE` range, the confidence band is not maximally wide, since above $AGE \approx 1$ and below $AGE \approx -1$ there are only $Y_i = 1$ or $Y_i = 0$ observations which gives some confidence in the model.

3.2.1.2 Bayesian version

Now, the same simple model in the **Bayesian framework** using the `rethinking` package:

The **interpretation of the priors** is already familiar: For a person with `AGE = 0` (average age), the expected log-odds of a 1 is a, i.e. the probability of a 1 is inv_logit(a) = inv_logit(0) = 0.5 on average. Hence, the probability of a 1 for a person with `AGE = 0` is between

```
inv_logit(c(-3, 3))
```

```
## [1] 0.04742587 0.95257413
```

with 95% probability a priori.

See exercise 12 for the priors.

```
library(rethinking)

dat <- list(
```

```
  y = y,
  AGE = AGE,
  N = length(y)
)

m_logistic <- ulam(
  alist(
    y ~ dbinom(1, p),
    logit(p) <- a + b * AGE,
    a ~ normal(0, 1.5),
    b ~ normal(0, 1.5)
  ),
  data = dat,
  chains = 4,
  cores = 4
)
```

```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 1 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 1 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 1 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 1 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 1 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 1 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 1 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 1 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 1 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 1 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 2 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 2 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 2 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 2 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 2 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 2 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 2 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 2 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 2 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 2 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 2 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 3 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 3 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 3 Iteration: 200 / 1000 [ 20%]  (Warmup)
```

```
## Chain 3 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 3 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 3 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 3 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 3 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 3 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 3 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 3 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 4 Iteration:   1 / 1000 [  0%]  (Warmup)
## Chain 4 Iteration: 100 / 1000 [ 10%]  (Warmup)
## Chain 4 Iteration: 200 / 1000 [ 20%]  (Warmup)
## Chain 4 Iteration: 300 / 1000 [ 30%]  (Warmup)
## Chain 4 Iteration: 400 / 1000 [ 40%]  (Warmup)
## Chain 4 Iteration: 500 / 1000 [ 50%]  (Warmup)
## Chain 4 Iteration: 501 / 1000 [ 50%]  (Sampling)
## Chain 4 Iteration: 600 / 1000 [ 60%]  (Sampling)
## Chain 4 Iteration: 700 / 1000 [ 70%]  (Sampling)
## Chain 4 Iteration: 800 / 1000 [ 80%]  (Sampling)
## Chain 4 Iteration: 900 / 1000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1000 / 1000 [100%]  (Sampling)
## Chain 1 finished in 0.0 seconds.
## Chain 2 finished in 0.0 seconds.
## Chain 3 finished in 0.0 seconds.
## Chain 4 finished in 0.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 0.0 seconds.
## Total execution time: 0.3 seconds.
```

```
precis(m_logistic)
```

```
##        mean        sd       5.5%     94.5%     rhat ess_bulk
## a 0.2938174 0.2284228 -0.0747465 0.6516328 1.000531 1245.523
## b 1.5101983 0.3414826  1.0031494 2.0784885 1.007427 1450.621
```

```
post <- extract.samples(m_logistic)
mean(exp(post$a)) # post$a is on the log-odds scale, hence apply exp()
```

```
## [1] 1.376576
```

```
mean(exp(post$b))
```

```
## [1] 4.807619
```

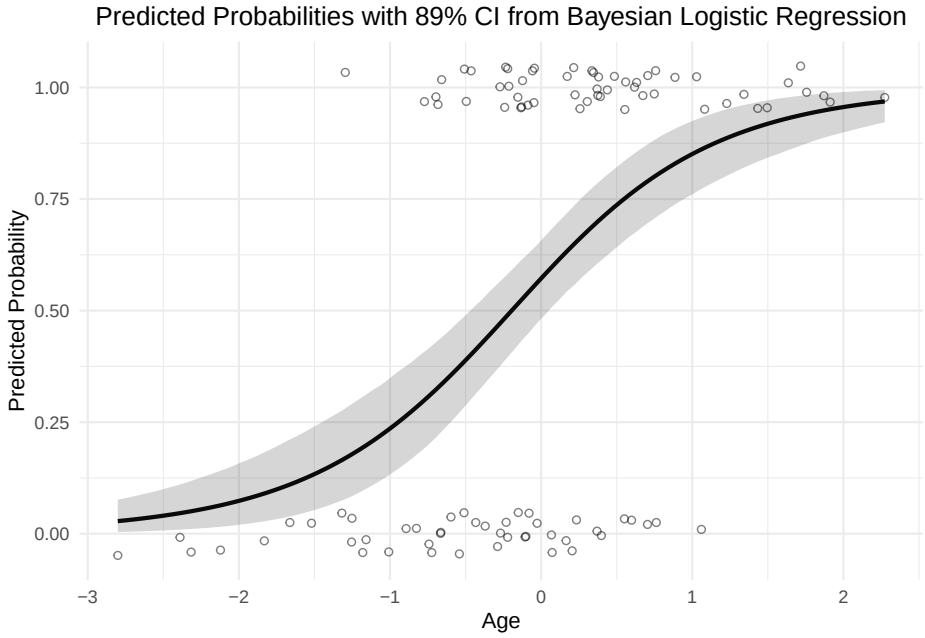
The coefficients are very similar to the Frequentist model.

We can again plot the **predicted probabilities with a confidence band**:

```r
# create prediction plot with confidence band:
AGE_seq <- seq(from = min(AGE), to = max(AGE), length.out = 100)

# Predictive values from posterior
link_result <- link(m_logistic, data = list(AGE = AGE_seq))
str(link_result) # num [1:2000, 1:100]
```

```
##  num [1:2000, 1:100] 0.00634 0.00858 0.03205 0.0721 0.04148 ...
```

```r
# -> for the 100 AGE values, we get 2000 draws from
# the posterior distribution of the mean (which is in our case equal to p_i).

# Compute mean and credible intervals
mu_mean <- apply(link_result, 2, mean)
mu_CI <- apply(link_result, 2, PI)  # 89% CI by default

# Create a data frame for plotting
plot_df <- data.frame(
  AGE = AGE_seq,
  predicted = mu_mean,
  ci_low = mu_CI[1, ],
  ci_high = mu_CI[2, ]
)

ggplot(plot_df, aes(x = AGE, y = predicted)) +
  geom_line(linewidth = 1) +
  geom_ribbon(aes(ymin = ci_low, ymax = ci_high), alpha = 0.2) +
  geom_jitter(data = data.frame(AGE = AGE, y = y),
              aes(x = AGE, y = y),
              width = 0, height = 0.05,
              shape = 1, color = "black", alpha = 0.5) +
  labs(
    title = "Predicted Probabilities with 89% CI from Bayesian Logistic Regression",
    x = "Age",
    y = "Predicted Probability"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```



3.2.1.3 Predictive Performance (of the outcome Y)

One may ask how well the model *predicts* the outcome. A very honest and simple way would be to compare the predicted outcomes with the actual outcomes (0/1). This is called a **confusion table** (or confusion matrix).

```
predicted_outcomes <- ifelse(predict(modlog, type = "response") > 0.5, 1, 0)
confusion_table <- table(Actual = y, Predicted = predicted_outcomes)
confusion_table
```

```
##       Predicted
## Actual  0  1
##      0 28 17
##      1 13 42
```

```
accuracy <- sum(diag(confusion_table)) / sum(confusion_table)
accuracy
```

```
## [1] 0.7
```

We could introduce the threshold for the predicted probability p_i of 0.5 as the cutoff for a 1. If the predicted probability is above 0.5, we predict a 1, otherwise a 0. Probably, one wants to move this threshold up or down in order to improve predictive performance.

In the diagonal of the confusion table, we find the number of correct predictions. In the off-diagonal, we find the number of incorrect predictions.

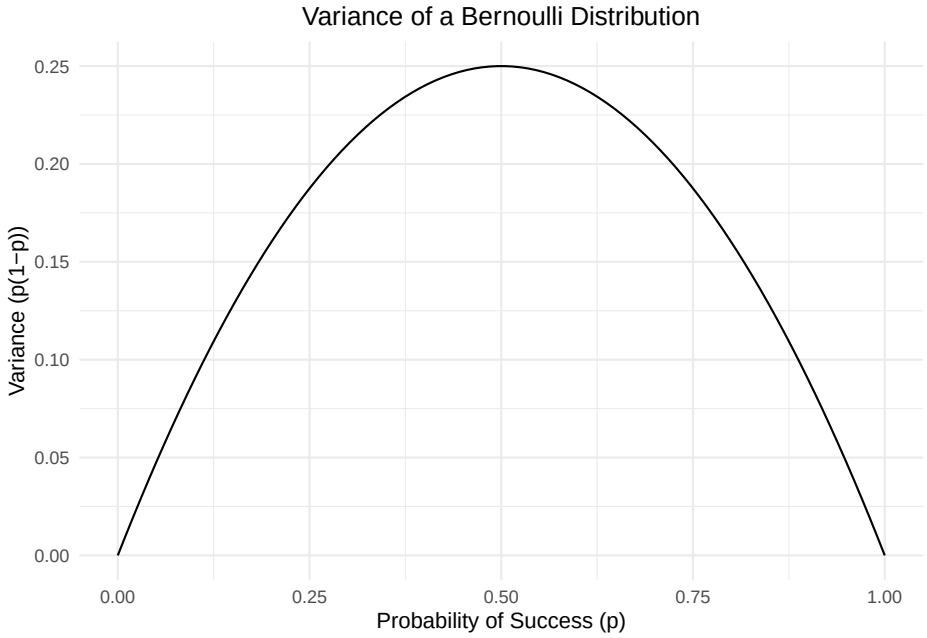
Now, with what shall we compare this accuracy? We need to compare this number with the **baseline accuracy**. If (as an example) 90% of the observations are 0s, then we would have an accuracy of 90%, since we could just vote for the majority every time - no model needed. If a fancy logistic regression model does not outperform this baseline accuracy, then it might not be so valuable (at least not for prediction). In our case, 0.7 is higher than the baseline accuracy of 0.55. The model adds some predictive value here. In my experience, model performance is often modest.

3.2.1.4 Model Assumptions

Model assumptions are also interesting.

One can show, that the errors in logistic regression are *not homoscedastic* (as they were in classical linear regression) (see Hosmer Lemeshow, Applied Logistic Regression, p.7). So, we do not need to check this. The reason is not that complicated. For every Y_i we assume a Bernoulli distribution: $Y_i \sim \text{Bernoulli}(p_i)$. The probability p_i is *not* constant, but increases (in our case) with `AGE`. The variance of a Bernoulli distribution is $p_i(1-p_i)$, which is not constant, if the p_i is not constant. The variance is highest at $p_i = 0.5$ and lowest at $p_i = 0$ or $p_i = 1$ (since only 0 or 1 does not give you much variance). You see this immediately when you plot the variance function $p_i(1-p_i)$:

```
p <- seq(0, 1, length.out = 100)
var_p <- p*(1-p)
data.frame(p = p, var_p = var_p) %>%
  ggplot(aes(x = p, y = var_p)) +
  geom_line() +
  labs(title = "Variance of a Bernoulli Distribution",
       x = "Probability of Success (p)",
       y = "Variance (p(1-p))") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```



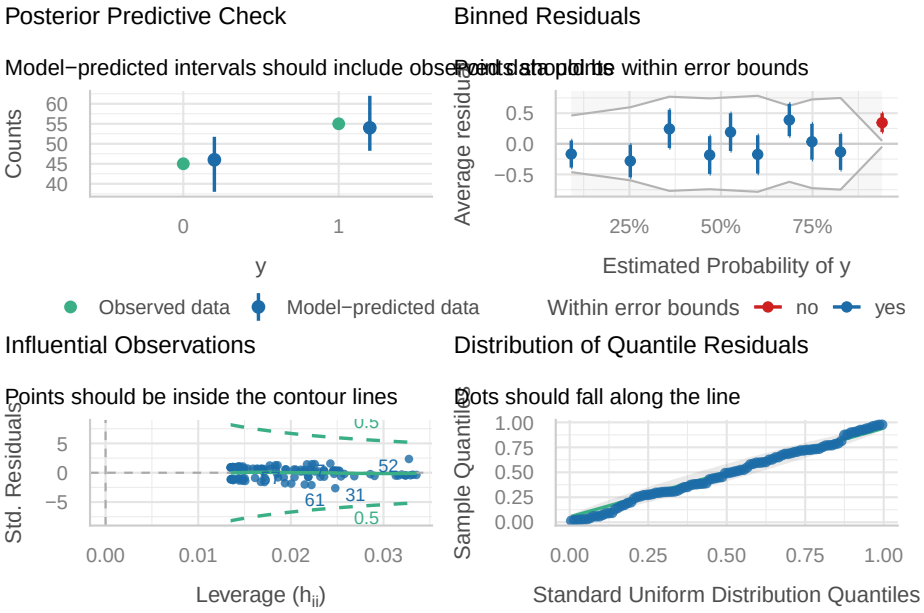
The variance of a Bernoulli distribution makes intuitive sense. For very low or very high probabilities, you mostly will see 0s or 1s, respectively, which means the variance is low.

Assumptions for logistic regression are:

- A binary outcome (which we have).
- Independent observations (which we have, since they were constructed this way). Observations might not be independent in time series data or repeated measurements of the same person, or clustered data (e.g., patients in hospitals).
- No perfect separation: This means that there should not be a *single* cutoff under which the outcome is always 0 and above which the outcome is always 1 (e.g., all people after 52 years have a heart attack, all before 52 do not). R will throw an error if this is the case. In such a case you would not bother with logistic regression, but introduce a simple decision rule.
- Linear relationship between predictors and log-odds of the outcome. Here a way to check linearity is explained.
- No multicollinearity.
- No influential observations.

Let's see what `check_model` has to say (which automatically checks what is possible for the given model type):

```
library(performance)
check_model(modlog, checks = c("pp_check", "residuals",
                               "influential_observations", "linearity"))
```



In exercise 14, we will look try to replicate the model checks.

- The **posterior predictive check** creates (for instance 1000) new data sets based on the estimated probabilities (100) of the given data. This plot often looks fine.
- Raw **residuals** are not so useful here, why? See exercise 13. Binned residuals are achieved by "dividing the data into categories (bins) based on their fitted values, and then plotting the average residual versus the average fitted value for each bin." (Gelman, Hill 2007: 97). If the model were true, one would expect about 95% of the residuals to fall inside the error bounds. Within a bin, I can just count the number of 0s and 1s and compare that with the model-estimated probabilities.
- There seem to be no **influential observations**, which have simultaneously a large residual and a large leverage. The question arises how the hat matrix is calculated for such a model. We looked at this plot in QM2 already in the context of simple linear regression.
- Observations (rows in our data set) must be **independent**, otherwise we would introduce random effects again. In our invented mini example, observations are of course independent. If we had clustering, we would need to use a mixed model again.

3.2.2 Multiple Logistic Regression

The major difference to the simple logistic regression is that we have more than one predictor (see p.35 in Hosmer Lemeshow, Applied Logistic Regression).

TODO: Example from the literature…

3.3 Poisson Regression

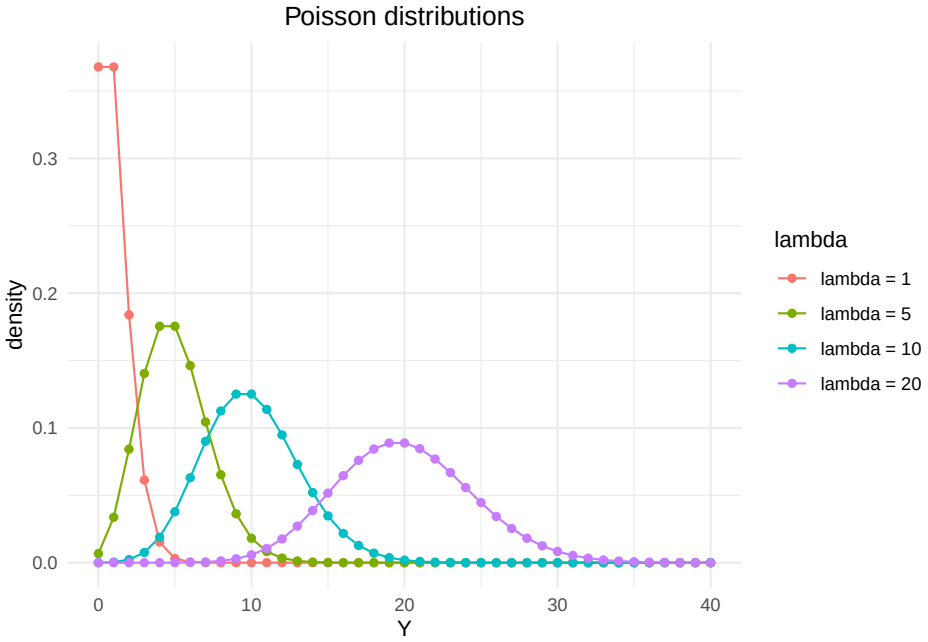
At first glance, this chapter of the book *Beyond Multiple Linear Regression* seems interesting. See also 11.2. in *Rethinking* and chapter 14 in *Westfall*.

As we have seen in the NHANES example in QM2, the **outcome is not always normally distributed** as the classical linear regression assumes. There we also assumed the variability (σ) around the mean (μ) increased with the mean, i.e., we explicitly modeled **heteroscedasticity**.

If you have count data (or rates) like the number of hospital visits, the number of accidents, the number of heart attacks, etc., as outcome variable, you can (sometimes, if the assumptions are met) use **Poisson regression**.

Let's quickly plot a Poisson-distributed variable $Y \sim \text{Poisson}(\lambda)$ with different λs:

```
library(ggplot2)

lambda_values <- c(1, 5, 10, 20)
y_vals <- 0:40

poisson_data <- expand.grid(y = y_vals, lambda = lambda_values)
poisson_data$prob <- dpois(poisson_data$y, poisson_data$lambda)
poisson_data$lambda <- factor(poisson_data$lambda,
                              labels = paste0("lambda = ", lambda_values))

ggplot(poisson_data, aes(x = y, y = prob, color = lambda)) +
  geom_line() +
  geom_point() +
  labs(title = "Poisson distributions",
       x = "Y",
       y = "density") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```



A Poisson-distributed variable can **only** take **integer values** which are of course non-negative.

It is a very elegant distribution, since the mean (μ) is equal to the variance (σ^2) and you only have one parameter $\lambda > 0$:

$$\mathbb{E}(Y) = \mu = \sigma^2 = \lambda$$

The above equation implies that the **variance increases** in a regression model where we model the mean!

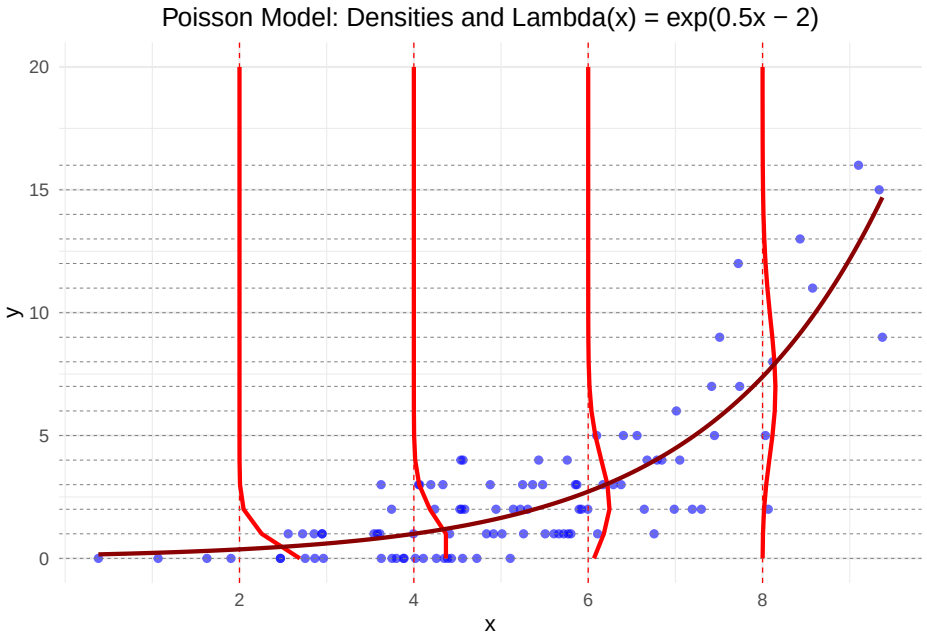
In the book chapter, there are 3 case studies. Feel free to read them. They check the assumptions of the model (Poisson response variable Y, independence of observations, mean=variance, linearity of the log of λ) beforehand.

We want to save time and quickly **create simulated data**. Here we *know* the true model.

```
library(ggplot2)
library(dplyr)

# Simulate base data
set.seed(123)
x <- rnorm(100, mean = 5, sd = 2)
y <- rpois(100, lambda = exp(0.5 * x - 2))
df_points <- data.frame(x = x, y = y)
```

```r
# Reference x-values and corresponding lambdas
x_ref <- c(2, 4, 6, 8)
lambda_ref <- exp(0.5 * x_ref - 2)
width <- 1  # horizontal stretch of the density curves

# Generate only the right half of each Poisson density curve
poisson_curves <- lapply(seq_along(x_ref), function(i) {
  lam <- lambda_ref[i]
  xx <- x_ref[i]
  y_vals <- 0:20
  d_vals <- dpois(y_vals, lam)

  data.frame(
    x = xx + d_vals * width,
    y = y_vals,
    group = paste0("x = ", xx)
  )
}) %>% bind_rows()

# Integer y values for horizontal reference lines
y_breaks <- 0:max(df_points$y)

# Lambda curve: exp(0.5 * x - 2)
lambda_curve <- data.frame(
  x = seq(min(df_points$x), max(df_points$x), length.out = 200)
) %>%
  mutate(y = exp(0.5 * x - 2))

# Plot
ggplot(df_points, aes(x = x, y = y)) +
  # scatterplot
  geom_point(color = "blue", alpha = 0.6) +
  # vertical dashed reference lines
  geom_vline(xintercept = x_ref, linetype = "dashed",
             color = "red", linewidth = 0.3) +
  # horizontal dashed lines at integer y-values
  geom_hline(yintercept = y_breaks, linetype = "dashed",
             color = "gray50", linewidth = 0.2) +
  # right half of Poisson curves
  geom_path(data = poisson_curves, aes(x = x, y = y, group = group),
            color = "red", linewidth = 1) +
  # lambda curve
  geom_line(data = lambda_curve, aes(x = x, y = y),
            color = "darkred", linewidth = 1, linetype = "solid") +
  # x-axis only at reference points
```

```
  scale_x_continuous(breaks = x_ref) +
  labs(
    title = "Poisson Model: Densities and Lambda(x) = exp(0.5x - 2)",
    x = "x",
    y = "y"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```



- In red, I drew some conditional Poisson densities (at $x = 2, 4, 6, 8$): $Y_i|x \sim \text{Poisson}(\lambda = e^{0.5x-2})$.
- In dark red, I drew $\lambda(x) = e^{0.5x-2}$, which is the mean **and** the variance of the (conditional) Poisson distribution.
- Both (mean and variance) are equal and clearly increase with x.

As you can see from the definition of the parameter λ: $(log(\lambda) = 0.5x - 2)$ **now the log of the mean is a linear function of** x (in logistic regression it was the logit of the expectation p_i). **Why the log**? Omitting the log, we would have $\lambda = 0.5x - 2$, which could be negative for small values of x, for instance for $x = 1$. Count data cannot be (in expectation or otherwise) negative, hence we use the log to ensure that λ is always positive: $e^{0.5x-2} > 0 \ \forall x$.

The model definition for a simple Poisson regression is then (see also Rethinking, p.346):

$$\begin{aligned} y_i &\sim \text{Poisson}(\lambda_i) \\ \log(\lambda_i) &= \alpha + \beta x_i \end{aligned}$$

- λ_i is the expected value of Y_i for observation i.
- The predictor (x_i) is usually centered (as is recommended in *Rethinking*) for easier interpretation of the prior and probably numeric stability, but for demonstrative purposes we skip it.

3.3.1 Bayesian Poisson Regression

We use the `rethinking` package to fit a Bayesian Poisson regression model. Let's see if we can recover the coefficients correctly. We would adjust the priors in a real-world scenario and do prior predictive distributions (see exercise 16). According to the book, the flat priors for α and β are not such a good idea (creates extreme values).

```
library(rethinking)
set.seed(123)
x <- rnorm(100, mean = 5, sd = 2)
y <- rpois(100, lambda = exp(0.5 * x - 2))
df <- data.frame(x = x, y = y)

model <- ulam(
  alist(
    y ~ dpois(lambda),
    log(lambda) <- a + b * x,
    a ~ dnorm(0, 1),
    b ~ dnorm(0, 1)
  ),
  data = df,
  iter = 2000,
  chains = 4
)
```

```
## Running MCMC with 4 sequential chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 1 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 1 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 1 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 1 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 1 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 1 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 1 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 1 Iteration:  900 / 2000 [ 45%]  (Warmup)
```

```
## Chain 1 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 1 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 1 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 1 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 1 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 1 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 1 finished in 0.1 seconds.
## Chain 2 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 2 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 2 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 2 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 2 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 2 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 2 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 2 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 2 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 2 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 2 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 2 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 2 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 2 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 2 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 2 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 2 finished in 0.1 seconds.
## Chain 3 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 3 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 3 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 3 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 3 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 3 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 3 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 3 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 3 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 3 Iteration:  900 / 2000 [ 45%]  (Warmup)
```

```
## Chain 3 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 3 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 3 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 3 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 3 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 3 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 3 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 3 finished in 0.1 seconds.
## Chain 4 Iteration:    1 / 2000 [  0%]  (Warmup)
## Chain 4 Iteration:  100 / 2000 [  5%]  (Warmup)
## Chain 4 Iteration:  200 / 2000 [ 10%]  (Warmup)
## Chain 4 Iteration:  300 / 2000 [ 15%]  (Warmup)
## Chain 4 Iteration:  400 / 2000 [ 20%]  (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]  (Warmup)
## Chain 4 Iteration:  600 / 2000 [ 30%]  (Warmup)
## Chain 4 Iteration:  700 / 2000 [ 35%]  (Warmup)
## Chain 4 Iteration:  800 / 2000 [ 40%]  (Warmup)
## Chain 4 Iteration:  900 / 2000 [ 45%]  (Warmup)
## Chain 4 Iteration: 1000 / 2000 [ 50%]  (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1100 / 2000 [ 55%]  (Sampling)
## Chain 4 Iteration: 1200 / 2000 [ 60%]  (Sampling)
## Chain 4 Iteration: 1300 / 2000 [ 65%]  (Sampling)
## Chain 4 Iteration: 1400 / 2000 [ 70%]  (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%]  (Sampling)
## Chain 4 Iteration: 1600 / 2000 [ 80%]  (Sampling)
## Chain 4 Iteration: 1700 / 2000 [ 85%]  (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%]  (Sampling)
## Chain 4 Iteration: 1900 / 2000 [ 95%]  (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%]  (Sampling)
## Chain 4 finished in 0.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 0.1 seconds.
## Total execution time: 0.6 seconds.
```

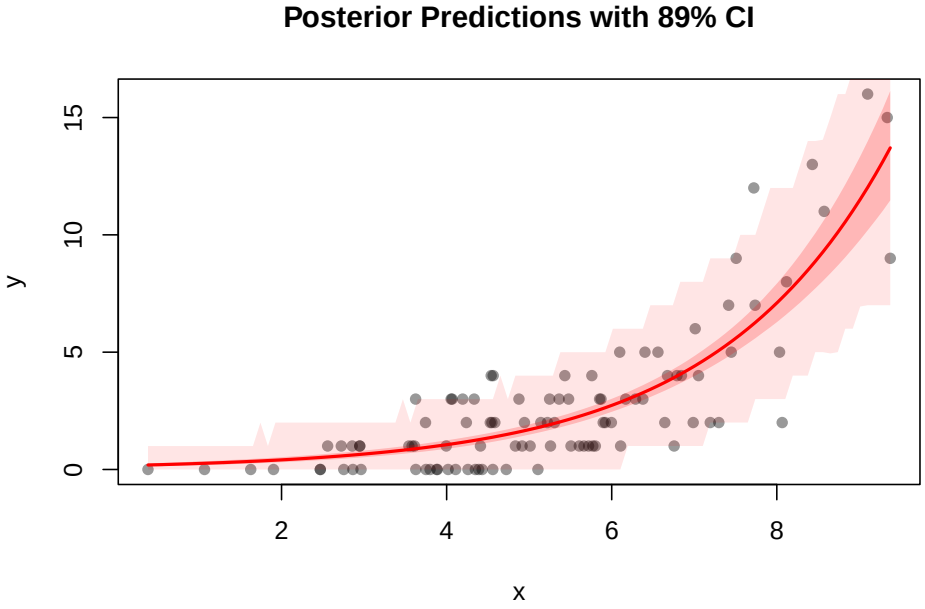
```
precis(model)
```

```
##         mean         sd       5.5%      94.5%     rhat ess_bulk
## a -1.8588351 0.22630189 -2.2248976 -1.4999446 1.004355 792.8690
## b  0.4769544 0.03263889  0.4252912  0.5295132 1.003665 793.8875
```

```
# looks good!
```

If you like, you can plot a confidence band for the mean and predicted values:

```
library(rethinking)
x_seq <- seq(from = min(df$x), to = max(df$x), length.out = 100)
pred_data <- list(x = x_seq)
lambda_pred <- link(model, data = pred_data)
y_pred <- rethinking::sim(model, data = pred_data)
lambda_mean <- apply(lambda_pred, 2, mean)
lambda_ci <- apply(lambda_pred, 2, PI, prob = 0.89)
y_ci <- apply(y_pred, 2, PI, prob = 0.89)

plot(df$x, df$y, col = ggplot2::alpha("black", 0.4), pch = 16,
     xlab = "x", ylab = "y", main = "Posterior Predictions with 89% CI")
lines(x_seq, lambda_mean, col = "red", lwd = 2)
shade(lambda_ci, x_seq, col = col.alpha("red", 0.2))
shade(y_ci, x_seq, col = col.alpha("red", 0.1))
```

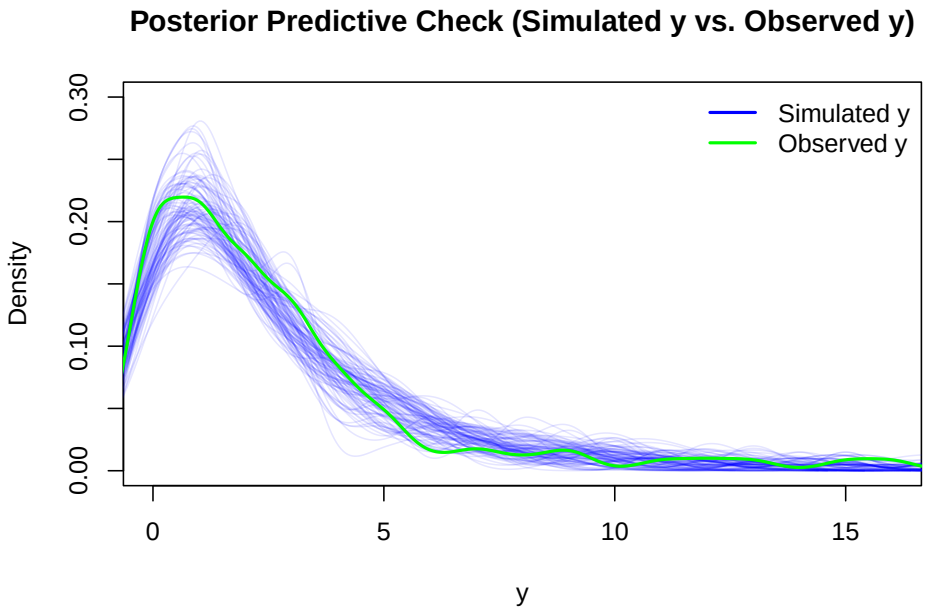


The dark shaded red area is the 89% confidence band for the mean of the outcome, while the light shaded red area is the 89% confidence band for the predicted values of the outcome.

The posterior predictive distribution looks very appealing (of course):

```
# Simulate 1000 posterior predictive datasets (each row = one simulation)
y_sim <- rethinking::sim(model, data = df)  # df must contain 'x' for conditioning
```

```r
# Select 100 draws for visualization
set.seed(42)
draws_to_plot <- sample(1:nrow(y_sim), size = 100)

# Prepare an empty plot
plot(NULL,
     xlim = range(df$y), ylim = c(0, 0.3),
     xlab = "y", ylab = "Density",
     main = "Posterior Predictive Check (Simulated y vs. Observed y)")

# Draw the predictive densities (blue curves)
for (i in draws_to_plot) {
  lines(density(y_sim[i, ]), col = col.alpha("blue", 0.1))
}

# Overlay the observed data density (green curve)
lines(density(df$y), col = "green", lwd = 2)
# Add a legend
legend("topright", legend = c("Simulated y", "Observed y"),
       col = c("blue", "green"), lty = 1, lwd = 2, bty = "n")
```



3.3.2 Frequentist Poisson Regression

Since Poisson regression is a generalized linear model (with log as link function "g"), we can use the `glm` function in R:

```r
mod_glm <- glm(y ~ x, data = df, family = poisson(link = "log"))
summary(mod_glm)
```

```
##
## Call:
## glm(formula = y ~ x, family = poisson(link = "log"), data = df)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.97253    0.24626   -8.01 1.15e-15 ***
## x            0.49353    0.03516   14.04  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 317.62  on 99  degrees of freedom
## Residual deviance: 108.22  on 98  degrees of freedom
## AIC: 331.57
##
## Number of Fisher Scoring iterations: 5
```

```r
confint(mod_glm)
```

```
## Waiting for profiling to be done...
```

```
##                  2.5 %     97.5 %
## (Intercept) -2.4652450 -1.4995384
## x            0.4250639  0.5629309
```

The coefficients are very similar to the Bayesian model and seem to be even closer to the true values. The reason is probably the very rough priors we have used.

Interpretation of the coefficients is similar to the Bayesian model. We ask ourselves basically always the same question: What happens to the **mean of the outcome** Y_i if we increase the predictor (of interest, for instance) x_i by 1 unit?

Let's do this for our Poisson regression model (using the true coefficient for β):

$$\mathbb{E}(Y_i|x+1) = \lambda(x+1) = e^{0.5(x+1)-2} = e^{0.5x+0.5\cdot 1-2} = e^{0.5x-2} \cdot e^{0.5}$$

Thus, the **mean of the outcome** Y_i **increases by a factor** of $e^{0.5} \approx 1.65$ **when we increase** x_i **by** 1 **unit**.

3.3.3 Model Assumptions

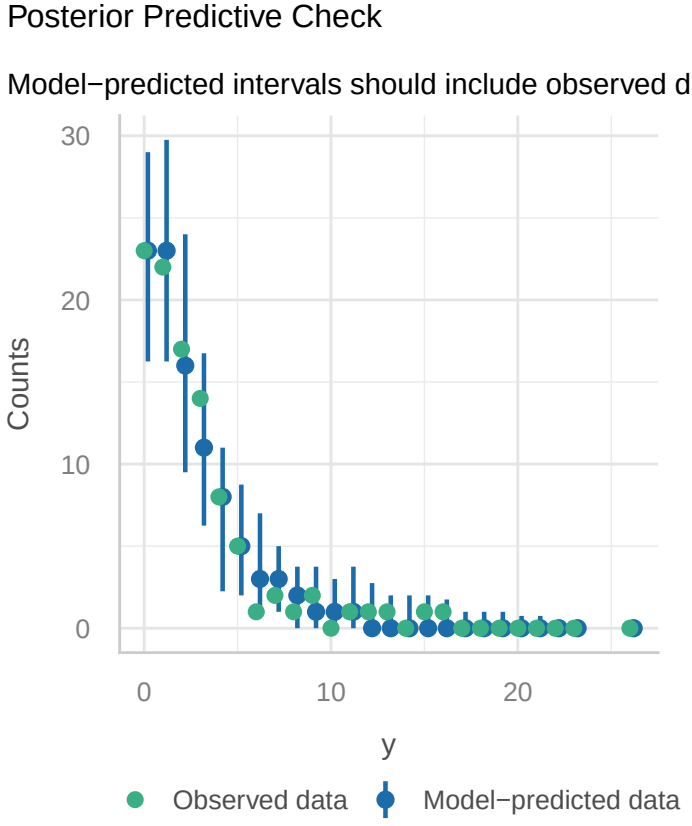
(Maybe this link is interesting. Further reading on checking the assumptions of count models: Roback & Legler, *Beyond Multiple Linear Regression*, Ch. 4, the `DHARMa` vignette on residual diagnostics via simulated residuals, and overdispersed count data.)

3.3.3.1 Outcome variable is Poisson distributed

This is trivial in our case, since we have simulated the data in such a way. But *if* we did not do that, we could check the posterior predictive distribution (which we have already done in the Bayesian model):

```
library(performance)
check_model(mod_glm, check = "pp_check")
```

```
## Cannot simulate residuals for models of class `glm`. Please try
##   `check_model(..., residual_type = "normal")` instead.
```



The model spits out Poisson-distributed data (because we said so) which fits the observed data (y) well. Good enough for us. There are more sophisticated

ways to check this, but we will not go into detail here.

3.3.3.2 Independence of observations

Same here, the draws are not dependent via construction. Knowing your data helps to decide this. If you have clustered data (for instance repeated measures of the same person), you might have violated this assumption. I am not aware of a diagnostic plot for this assumption.

3.3.3.3 Mean should be equal variance (dispersion)

The `dispersion ratio` should be (in the case of Poisson regression) close to 1, which is the case. > 1 would indicate overdispersion, < 1 underdispersion.

Overdispersion occurs if the variance is larger than the mean in our Poisson regression. Underdispersion occurs if the variance is smaller than the mean. You can give this video a try. In the summary output of the `glm` model, you can see that a dispersion parameter is assumed to be 1: `(Dispersion parameter for poisson family taken to be 1)`.

One can estimate the dispersion parameter using the R summary output: `Residual deviance: 108.22 on 98 degrees of freedom`

$$\text{Dispersion} = \frac{\text{Residual Deviance}}{\text{df}} = \frac{108.22}{98} \approx 1.10$$

We can (exploratively) look at a p-value for the dispersion parameter:

```
mod_glm$deviance/mod_glm$df.residual # dispersion
```

```
## [1] 1.104287
```

```
library(AER)
dispersiontest(mod_glm, trafo = 1, alternative = "greater")
```

```
##
##  Overdispersion test
##
## data:  mod_glm
## z = -0.049191, p-value = 0.5196
## alternative hypothesis: true alpha is greater than 0
## sample estimates:
##        alpha
## -0.005479567
```

There is also a test in the `performance` package:

```
library(performance)
check_overdispersion(mod_glm)
```

```
## # Overdispersion test
##
##        dispersion ratio =  0.996
##   Pearson's Chi-Squared = 97.560
##                 p-value =  0.494

## No overdispersion detected.
```

3.4 Summary and further directions

Beginning in QM1 we started with probability distributions to describe random variables. We used the probability framework right from the start to make predictions about the outcome of random variables. This is made possible since we either know or estimate the respective distribution of the random variable of interest (for instance the number of false positives when doing research).

In QM2 we slowly expanded our knowledge towards statistical modeling using regression models starting out with the simplest regression model there is: the simple mean model. We have seen that we can add complexity by adding more predictors (multiple linear regression), interaction terms ($X_1 \times X_2$), polynomial terms (like X^2). We then loosened the assumptions of classical linear regression by introducing random effects (hierarchical models) and generalized linear models (GLMs).

I have tried to juxtapose the Frequentist and Bayesian approach to modeling. Sometimes the Bayesian approach using the `rethinking` package is more intuitive and easier to use. For instance, it feels quite natural to go from homoscedasticity to heteroscedasticity in the Bayesian framework, since we can just model the variance explicitly as a function of the mean (of the outcome) (see NHANES example).

In this script, we have seen rather quickly that there is a **tradeoff between the complexity of the model and its interpretability**. What exactly do the parameters in the model mean? On the one hand, we want to model the underlying structure of the data as well as possible, on the other hand, we want to keep the model interpretable and not overfit the data. This balance is not always easy to achieve.

Our specific challenge was and is to build a bridge between the fundamental concepts of probability and statistics and the practical application of statistical modeling. The latter is often much more complex than the former. There are known **unknown unknowns** (from our current perspective), which we did not have time to cover in our courses:

- Missing data. What if you have a rectangular data set with 20 variables, but some variables have missing values? There are different types of missing data. The easiest case is that the data are missing completely at random (MCAR). Here, we can just drop the rows with missing values

(`na.omit()`). It is **a good start** to perform sensitivity analyses to see how the results change if we drop the rows with missing values There are other missing data mechanisms. You should read the basics about this topic, if you encounter more than, say 5%, missing values in your data set (per variable).

- Representative sampling. In the classical statistical paradigm, we ask: How representative is our *sample* of the *population* of interest? Professional surveys like NHANES use **survey weights** in order to correct for sampling bias respectively non-representativeness in the sample. See also Westfall, chapter 1.3. (p.8) why we should probably not use the population paradigm in regression, and better think of the data generating mechanism.
- Compositional data are data that represent parts of a whole, such as percentages or proportions. They are often constrained to sum to a constant (e.g., 100%) and can lead to **misleading results if analyzed with standard statistical methods**. Here is a simple example.
- Causal inference. In Richard McElreath's book *Statistical Rethinking*, causal thinking is a central theme. We have touched on it lightly at the end of QM2, specifically we drew a directed acyclic graph (DAG) to illustrate the causal relationships between variables in our NHANES toy example.

There are also **unknown unknowns**, which not even cutting edge researchers are aware of. Previously, dealing with *compositional data* was such a problem. With respect to compositional data, causal inference and Bayesian statistics (and probably other techniques I have not mentioned), we in applied health sciences are merely slowly adapting these into our statistical toolbox.

My **main goals** of our three statistics courses (QM1-3) were:

- To point towards great books and resources. Knowledge is decentralized.
- To show you that statistics is more than *find-the-right-statistical-test-Bingo*.
- To show you that *statistical significance* does not mean much considered only by itself.
- To point out that statistical knowledge is not *final* but *evolving*.
- I have tried to foster understanding of modeling by focusing on concepts rather than on mathematical details (like matrix algebra) and fixed recipes with strange terminology.

In our final chapter, we take a look at some aspects of artificial intelligence.

3.5 Exercises

Solutions to the exercises can be found here.

3.5.1 [E] Exercise 1

- Do some descriptive statistics on the data set reedfrogs.
- Use the `dplyr` and `tidyr` packages to create a new data set with one row per individual tadpole. For every tadpole, we want the variable *surv_bin* to be 1 if the tadpole survived and 0 if it died.
- Feel free to check out the solution and ask a LLM (e.g., GPT) to explain the code - in this case it does a good job.

3.5.2 [M] Exercise 2

We want to better understand the pooling effect of model 13.2 vs. model 13.1.

- Calculate the mean survival probability for each tank in model 13.1/2. Use the function `logistic` (or equivalently `inv_logit`) from the `rethinking` package to transform the 48 intercepts to probabilities.
- Calculate the raw survival probability for each tank by dividing the number of survivals (S) by the tank size (N). Or you can just use the column `propsurv`, which is the same.
- Plot the mean survival probability for each tank from model 13.1 against the raw survival probability. What do you see?
- Do the same for model 13.2. What do you see?
- In the model definition of 13.1 change the priors for the intercepts and play with different large/small values for XX: `a[tank] ~ dnorm(0, XX)`. What do you see?
- In the model definition of 13.2 change the priors for the intercepts and play with different large/small values for XX: `a[tank] ~ dnorm(0, XX)` and also the prior for σ (which regulates the pooling): `sigma ~ dexp(XX)`.

3.5.3 [E] Exercise 3

We want to check the correlation of the posterior estimates of the intercepts (`a[tank]`) for model 13.1 and 2.

- Use the `extract.samples` function to extract the posterior samples of the intercepts.
- Use the `cor` function to calculate the correlation between the first 2 intercepts for both models.
- Is there a correlation of the posterior samples for `a[1]` and `a[2]` in either model?
- Reason about the results.

3.5.4 [M] Exercise 4

Go back to the Frequentist version of the tadpole model 13.2 above.

- Calculate and compare the model-estimated survival probabilities in the tanks for both models (Frequentist and and Bayesian model 13.2).

- What would be the Frequentist equivalent of model 13.1? Compare the model-estimated survival probabilities again.

3.5.5 [M] Exercise 5

- Verify with R that the `logit` and `logistic` (=`inv_logit`) functions are indeed inverse functions of each other.
- Also plot both functions for clarity.

3.5.6 [E] Exercise 6

Consider the tadpole data set again.

- Fit a model with complete pooling, i.e. one intercept for all tanks, without a regularizing prior. Hence, the tanks to not "know" anything from one another. Use the `rethinking` package.

3.5.7 [M] Exercise 7

Consider our regression model ($\mu_i = \alpha + \beta_1 x_i + \beta_2 x_i^2$) for the body height of the !Kung San people in QM2.

- Randomly select 80% of your observations (rows in the data set).
- Fit the model again using the `rethinking` package on the 80% of observations.
- Predict the body height of the remaining 20% of observations using the model.
- Calculate the root mean squared error (RMSE) of the predictions:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(h_i - \hat{h}_i)^2}$$

  where h_i is the observed body height and $\hat{h}_i$ is the predicted body height.
- Calculate the RMSE for the 80% of observations as well and compare the two RMSEs. What do you observe? Was this an accident?
- Repeat creating a training and test set and calculating the RMSE for both the training and the test set 100 times. Draw a histogram of the RMSEs for training and test set. What do you observe?

3.5.8 [M] Exercise 8

Consider the Mulligan manual therapy paper.

- Try to find the method for sample size calculation in the referenced literature literature.

3.5.9 [E] Exercise 9

Consider the Mulligan manual therapy paper.

- Verify for the groups *Ex* and *MMT+ex* that the primary outcome *headache frequency* is normally distributed at Week 0 using the `shapiro.test` function in R.

3.5.10 [H] Exercise 10

Consider the Bayesian version of Mulligan manual therapy paper above.

- Replace α and u_i with one single intercept α. The prior for α is then $\alpha \sim Normal(\bar{\alpha}, \sigma_\alpha)$
- Fit the model and compare the results.

3.5.11 [E] Exercise 11

Consider the section about simple logistic regression.

- What is $g^{-1}(X\beta)$ in our case?

3.5.12 [M] Exercise 12

Consider the section about simple logistic regression; specifically the Bayesian model.

- Think about the priors.
- How are they interpreted and which priors could make sense if the outcome was myocardial infarction (heart attack)

3.5.13 [E] Exercise 13

Consider the section about simple logistic regression.

- Why are the raw residuals not so useful in this case?

3.5.14 [H] Exercise 14

Consider the section about simple logistic regression.

- Try to replicate the individual model checks you see in `check_model(modlog)`.

3.5.15 [H] Exercise 15

Consider the section about simple logistic regression.

- We want to find out the parameters β_0 and β_1 of the logistic regression model using the **least squares method**.

- For this, minimize the following function:

$$\sum_{i=1}^{n}(y_i - \text{inv\_logit}(\beta_0 + \beta_1 AGE_i))^2$$

  whereas y_i is the observed outcome variable (0 or 1) and AGE is the standardized age of the i-th participant.
- Compare the results to before.

3.5.16 [M] Exercise 16

Consider the section about Bayesian Poisson regression.

- Adapt the priors for α and β to be more informative.
- Do prior predictive checks to see if the priors make sense.

3.6 Exam Information (Jun 2025)

- Content of the written exam is **everything** up to the prediction plot (red shaded) for Poisson regression.
- The exam will take place on **11.6.25 8:30 am, room: MG O1.057**.
- **One** sheet of A4 paper with handwritten notes is allowed.
- A simple calculator is allowed, but probably not needed.
- Bring, as always, your black marker.

Chapter 4

Artificial Intelligence (AI)

4.1 Introduction

Why do we bother with **this topic** at all?

For a **simple reason**: The release of ChatGPT in late 2022 has not only sparked a new wave of interest in AI, but has fundamentally influenced

- how we write text,
- code,
- how we search the internet,
- create images and
- solve problems.

Another reason could be given. As far as I am aware, **neural networks** (an important part of AI, more on that topic later) are a true **extension of our regression models**. Hence, the whole regression framework can be subsumed under the umbrella of AI.

One thing should be stated at the beginning. The field and applications are moving so fast that it is impossible to keep up with the latest developments. So, please forgive me if some of the information is outdated or not up to date.

The standard textbook on AI is Russell and Norvig's Artificial Intelligence: A Modern Approach (3rd edition, 2020), currently in its 4th edition.

Other good (non-technical) books on the topic are:

- Melanie Mitchell's Artificial Intelligence: A Guide for Thinking Humans
- Gary Marcus's Rebooting AI: Building Artificial Intelligence We Can Trust
- Stuart Russell's Human Compatible: Artificial Intelligence and the Problem of Control

- Max Tegmark's Life 3.0: Being Human in the Age of Artificial Intelligence

The first two authors seem to be less afraid that AI could take over the world, while the latter two authors are more concerned about the implications of AI on society and humanity.

4.1.1 A short history of AI

Artificial Intelligence (AI) refers to the design and development of systems that can perform tasks typically requiring human intelligence—such as reasoning, learning, perception, problem-solving, and natural language understanding.

The ambition to build machines that mimic or surpass human cognitive abilities is not new and can be traced back to myths, philosophical debates, and early mechanical automata.

The formal **birth of AI as a field** is generally dated to **1956**, when John McCarthy, Marvin Minsky, Claude Shannon, and Nathaniel Rochester organized the *Dartmouth Summer Research Project on Artificial Intelligence*. This conference was the first to coin the term "artificial intelligence" and laid the foundation for decades of exploration into machine cognition.

However, the intellectual roots of AI go much further back. **Alan Turing**, in his seminal 1950 paper *"Computing Machinery and Intelligence"*, asked the famous question: "Can machines think?" He proposed the **Turing Test** as a way to evaluate a machine's ability to exhibit intelligent behavior indistinguishable from that of a human. Turing's work established a theoretical framework that deeply influenced how researchers would later define machine intelligence.

In the early decades following Dartmouth, AI research oscillated between periods of optimism and disappointment. The **1950s–1970s** were marked by the development of symbolic AI and rule-based systems. Programs like ELIZA (a simple natural language processor) and SHRDLU (which operated in a limited "blocks world") showcased early attempts at mimicking human-like reasoning. **Researchers believed that general AI** (AGI) was just around the corner.

This optimism faded in the **1970s and 1980s**, leading to what became known as the *"AI winter"*—a period characterized by reduced funding and slowed progress due to unmet expectations. The limitations of symbolic logic in handling uncertainty and perception-heavy tasks became clear. However, during this time, important subfields like **expert systems** and **machine learning** began to evolve quietly.

AI entered a new phase in the late 1990s and early 2000s, buoyed by greater computational power, larger datasets, and improved algorithms. Milestones such as **IBM's Deep Blue defeating chess champion Garry Kasparov in 1997**, and later, **Watson winning *Jeopardy!* in 2011**, demonstrated tangible progress.

The last decade, however, marked a major **paradigm shift**, especially with the rise of **deep learning**, a form of machine learning using artificial neural networks. In 2012, a convolutional neural network (AlexNet) won the ImageNet competition, drastically improving image recognition benchmarks. This breakthrough ignited an AI renaissance that led to the development of powerful models such as **GPT (by OpenAI)** and **BERT (by Google)**—transforming fields like natural language processing, computer vision, and robotics.

Today, AI spans everything from autonomous vehicles to medical diagnostics, and the field is increasingly focused not just on capability, but on **safety, alignment, interpretability**, and **ethics**. While we are still far from artificial general intelligence (AGI), the rapid progress in AI systems is reshaping industries, science, and society.

In the meantime, AlphaGo, a program developed by Google DeepMind, defeated the world champion Go player Lee Sedol in 2016. As with chess, many people thought that Go would be too complex for a computer to master.

Demis Hassabis, the founder of DeepMind, won the Nobel Prize in chemistry 2024 for his work on AlphaFold2, which solved the protein folding problem.

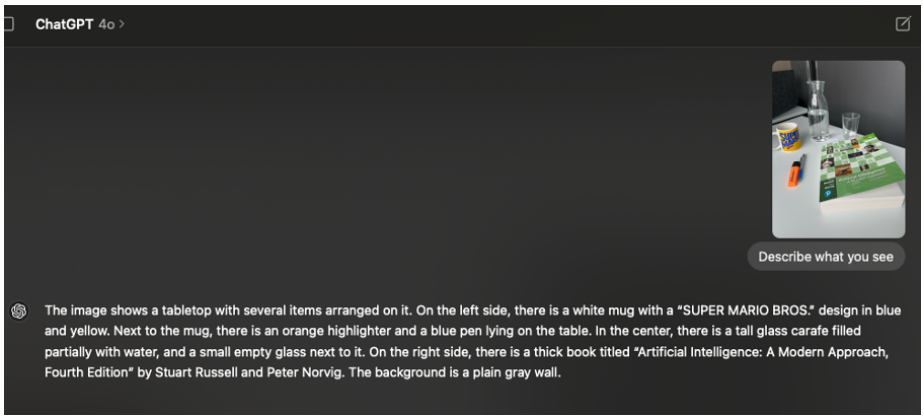
4.1.2 ChatGPT

The public release of ChatGPT (**G**eneralized **P**retrained **T**ransformer) in November 2022, based on OpenAI's GPT-3.5, marked a watershed moment in the history of artificial intelligence. While **large language models (LLMs)** had already impressed researchers, ChatGPT was the first to make them widely accessible through a conversational interface. Suddenly, millions of people could ask questions, get code suggestions, summarize documents, or even write poetry—all through natural dialogue.

By early 2023, ChatGPT had become a global phenomenon, reaching 100 million users within two months, making it the fastest-growing consumer application in history. It changed the public perception of AI from futuristic and experimental to practical and omnipresent.

In March 2023, OpenAI released **GPT-4**, a much improved model in terms of reasoning, factual accuracy, and multilingual capabilities. GPT-4 also **introduced image input** and began to bridge the gap toward **multimodal AI** (not limited to text input).

Here is an example of an image described by GPT-4o (a more advanced version of



GPT-4):
(Created with ChatGPT macOS app, 21.5.24)

At the time, the perfectly worded image description from GPT felt like magic. One way to achieve this *image captioning* is explained by Mike Pound here. The transformer architecture (a so-called vision transformer, ViT) is involved in this process, which is a **key component** of many modern AI systems. We will not go into details here. Interestingly, the transformer architecture was introduced by researchers at Google in 2017 in a paper called Attention is All You Need, which was cited...

```
library(rvest)
library(stringr)

url <- "https://scholar.google.com/scholar?hl=en&q=attention+is+all+you+need"
page <- read_html(url)

# Extract result blocks
results <- page %>% html_elements(".gs_ri")
first_result <- results[[1]]

# Extract footer text
citation_text <- first_result %>%
  html_element(".gs_fl") %>%
  html_text()

print("wait for it...")
```

```
## [1] "wait for it..."
```

```
# Extract citation number
str_extract(citation_text, "(?<=Cited by )\\d+")
```

```
## [1] "264808"
```

```
print("...times.")
```

```
## [1] "...times."
```

As LLMs matured, the focus shifted (also) to **autonomous agents**. Systems like AutoGPT, BabyAGI (AGI stands for Artificial General Intelligence), and later OpenAI's own GPTs (custom bots) experimented with giving models access to tools (e.g., browsers, calculators, APIs), memory, and goals. One failed attempt to sell AI agents as an end-user product was the Rabbit R1.

For people in the field, it was clear the technology was not ready for this type of application yet. I am not sure how it has progressed in the meantime, but a year ago the practical use of these agents was still limited. Google announced "Agent Mode" recently. There have been promises of AI agents performing tasks (like calling a restaurant to make an appointment) autonomously before.

The vision is no longer just chat, but delegating tasks: "book my flights," "analyze this dataset," or "write a full report with citations." This period also saw growing demand for interpretability, trustworthiness, and alignment. Questions about hallucinations, bias, and safety became central—not only for researchers, but for businesses and governments. RLHF (Reinforcement Learning from Human Feedback) became a standard technique to make models more helpful and aligned with human intent.

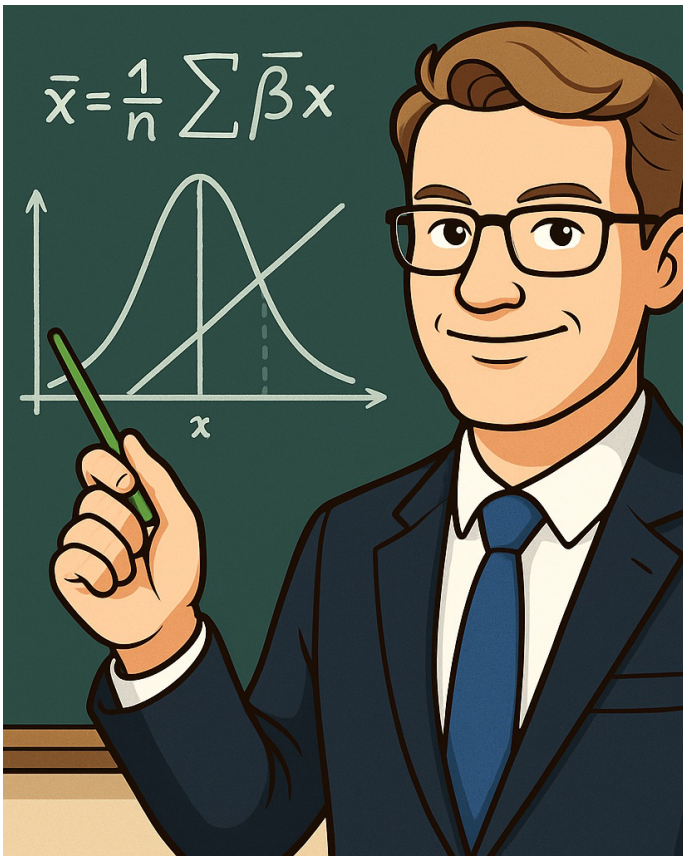
Meanwhile, open-source efforts accelerated. Meta released LLaMA, and others followed with models like Falcon, Mistral, and Mixtral. Although initially less powerful, these models enabled broader research, customization, and deployment, especially in privacy-sensitive environments.

4.2 Capabilities of LLMs

A model like GPT-4 can perform a wide range of tasks, including:

- **Text Generation**: Writing essays, articles, or creative content. *Very* simplified, this is done by **predicting the next word** in a sequence based on the context of the previous words. GPT-4 was trained using Reinforcement Learning from Human Feedback (RLHF). More on reinforcement learning later. But the *Human Feedback* means that humans rated the quality of the generated text.

- **Image generation**: Creating images from text prompts. This has improved during the last 1-2 years. I assume that OpenAI uses *very* advanced **diffusion models** for this task, the basic principles are rather graspable though (1, 2).



Example picture generated by GPT-4o:

- **Question Answering**: Providing answers to factual questions or summarizing information. Until this day, LLMs sometimes hallucinate, i.e., they make up facts or provide incorrect information with a high degree of confidence. They are trained on a large corpus of text to produce reasonable sounding answers.
- **Translation**: Translating text between multiple languages. This works rather excellent and brings back memories from my English teacher who told me that language will never be mastered by a computer.
- **Code Generation**: Writing and debugging code in various programming languages.
- **Image Understanding**: Analyzing and describing images (in multimodal versions).
- **Conversational Agents**: Engaging in natural language conversations, simulating human-like dialogue.

Sébastien Bubeck published a paper in 2023 titled
"Sparks of Artificial General Intelligence: Early Experiments with GPT-4",

which explores the capabilities of GPT-4 in depth. As in other technical reports of commercial models, the so-called model weight (more on that later) are not disclosed.

4.3 Risks of LLM and AI in general

...

Notes:

- Robert Miles AI safety.